

සරල සිංහලෙන්  
පරිගණක දැනුම

# EasyGuide®

- 32 pages
- 09 Chapters
- 60+ Images

Step-by-Step Explanations | Comprehensive Images | Tips | Guides

```
0;b < c.length;b++) { a += " " + c[b] + " "; } $("#User_logged  
click"); }); $("#no_single").click(function() { for (var a =  
ged").a()), b = $("#no_single_prog").a(), c = 0;c < a.length;  
k b && (a[c] = " "); } b = ""; for (c = 0;c < a.length;c++) {  
" "; } a = b; $("#User_logged").a(a); function(a); }); $("  
ed"); function l() { var a = $("#use").a(); if (0 == a.length  
} for (var a = q(a), a = a.replace(/ +(?= )/g, ""), a = a.s  
c = 0;c < a.length;c++) { 0 == r(a[c], b) && b.push(a[c]); }  
on h() { for (var a = $("#User_logged").a(), a = q(a), a = a.  
"), a = a.split(" "), b = [], c = 0;c < a.length;c++) { 0  
push(a[c]); } c = {}; c.j = a.length; c.unique = b.length -  
function k() { var a = 0, b = $("#User_logged").a(), b = b.  
r)/gm, " "), b = q(b), b = b.replace(/ +(?= )/g, ""); inp_ar  
for (var b = [], a = [], c = [], a = 0;a < inp_array.length;  
array[a], c) && (c.push(inp_array[a]), b.push({word:inp_array  
ie:0})), b[b.length - 1].c = r(b[b.length - 1].b, inp_array  
)); a = b.length - 1; b = m(a, " "); -1 < b &&  
< b.length - 1; b = m(b, 1);
```

Low Price  
Edition  
Rs.200/-



සි ජිලස් ජිලස්  
ට්‍රේඩ්මාර්ක්



## EasyGuide

සි ජලස් ජලස් මගින් ප්‍රේෂණය කරන ආකාරය

© විජය නිවුස්පේපර්ස් ලිමිටඩ්.

සියලු හිමිකම් ඇවිරිණි.

ප්‍රකාශකයින්ගේ ලිඛිත අවසරය  
නොමැතිව මෙම පොතේ එන කිසිවක්  
මොනම ආකාරයකින් හෝ නැවත  
පළ නොකළ යුතුය. මෙම පොතෙහි  
එන විවිධ වෙළෙඳ නාම සහ  
වෙළෙඳ ලකුණු, නීතිමය වශයෙන්  
පිළිගත් අයිතිකරුවන් සතු  
බුද්ධිමය දේපළ වේ.

ISBN 978 - 955 - 1008 - 60 - 4

පළමු වැනි සංස්කරණය  
2019 ජනවාරි

කියෝ අප්පන රාජමන්ත්‍රී  
විසින් ලියන ලදී.

හසිත හෙට්ටිආරච්චි  
විසින් පිටු සකසන ලදී.

ෆොන්ට්මාස්ටර් අභය/අභය බෝල්ඩ්,  
මලිනි, ටයිම්ස් නිව්රෝමන්,  
කුරියර් නිව් සහ උබුන්ටු යන  
අකුරු මෝස්තර යොදා පිටු සැලසුම කරන ලදී.

පාලිත අමරසූරිය විසින්  
සංස්කරණය කොට සම්පාදනය කරන ලදී.



කොළඹ 02, හුණුපිටිය හරස් පාරේ,  
අංක 08 දරන කැන පිහිටි  
විජය නිවුස්පේපර්ස් ලිමිටඩ් සමාගම  
විසින් මුද්‍රණය කර ප්‍රසිද්ධ කරන ලදී.



facebook.com/  
easyguide.book



gplus.to/  
easyguide.book



twitter.com/  
easyguide\_book



t.me/sinhalamovie

## හැඳින්වීම

සී ජලස් ජලස් මගින්  
ප්‍රෝග්‍රැම් කරන ආකාරය

සී ජලස් ජලස් යනු අද ලොව ප්‍රකට පෙළේ පරිගණක ක්‍රමලේඛ භාෂා අතරින් එකකි. එසේම අනාගතයේදී පරිගණක ක්‍රමලේඛ ශිල්පියෙක් වෙන්නට සිතා වැඩ කරන ඕනෑම කෙනෙක් විසින් උගත යුතු මූලික ක්‍රමලේඛ භාෂා අතරින් මුල් තැනේ තිබෙන ක්‍රමලේඛ භාෂාව සී ජලස් ජලස්ය. මෙම භාෂාව හැසිරවීම ගැන මූලික දැනුමක් තැති අයට පුවත් මෙම පොත පරිහරණය කොට ඒ ගැන ඉගෙන ගන්නට පුළුවන. එසේම දැනට සී ජලස් ජලස් ගැන මූලික දේවල් දන්නා අයටත් දැනුම වැඩි කර ගන්නට ඉවහල් වෙන කරුණුද මෙම පොතෙහි අඩංගුය. ■

## පටුන

|                                      |    |
|--------------------------------------|----|
| පළමු වැනි පරිච්ඡේදය                  |    |
| අලුත් වැඩසටහනක් සැකසීම               | 03 |
| දෙවැනි පරිච්ඡේදය                     |    |
| වේරියබල් භාවිතා කිරීම                | 07 |
| තුන් වැනි පරිච්ඡේදය                  |    |
| මපරේටර්ස් භාවිතා කිරීම               | 12 |
| හතර වැනි පරිච්ඡේදය                   |    |
| කන්ඩිනෙල් ස්ට්‍රක්චර්ස් භාවිතා කිරීම | 18 |
| පස් වැනි පරිච්ඡේදය                   |    |
| ලූප්ස් භාවිතා කිරීම                  | 20 |
| හය වැනි පරිච්ඡේදය                    |    |
| නෙස්ටඩ් කන්ට්‍රෝල් ස්ට්‍රක්චර්ස්     | 25 |
| හත් වැනි පරිච්ඡේදය                   |    |
| ඇරේස් භාවිතා කිරීම                   | 26 |
| අට වැනි පරිච්ඡේදය                    |    |
| ෆන්ක්ෂන් නිර්මාණය කිරීම              | 28 |
| නව වැනි පරිච්ඡේදය                    |    |
| ඩොකියුමන්ට් හැසිරවීම                 | 30 |



## පළමු වැනි පරිච්ඡේදය

### සී ජලස් ජලස් වැඩසටහනක් සකසා ක්‍රියාත්මක කරවන ආකාරය

අනාගතයේදී පරිගණක ක්‍රමලේඛ ශිල්පියෙක් වෙන්නට සිතා වැඩ කරන ඔහුම කෙනෙක් විසින් උගත යුතු මූලික ක්‍රම ලේඛ භාෂා අතරින් මුල් තැනේ තිබෙන ක්‍රමලේඛ භාෂාව සී ජලස් ජලස් (C++) ය. ඇතැම් අය මෙය සී ඩබල් ජලස් නියා හැඳින්වූවත් එම උච්චාරණය නිවැරදි නැත.

සී ජලස් ජලස් යනු සී (C) භාෂාවේ ගති ලක්ෂණ මත පිහිටා නිර්මාණය කළ පරිගණක භාෂාවකි.

• භාෂා පරම්පරාවලට අනුව මෙය මැද මට්ටමේ භාෂාවක් (middlelevel language) ලෙස හැඳින්වේ. C++ භාෂාව ඔබ්ජෙක්ට් ඔරියන්ටඩ් (object oriented) වර්ගයේ භාෂාවකි. එය C ක්‍රමලේඛ


**Code::Blocks**
Code::Blocks - The IDE with all the features you need, having a consistent look, feel and operation across platforms.

[Home](#)
[Features](#)
[Downloads](#)
[Forums](#)
[Wiki](#)

### Main

- Home
- Features
- Screenshots
- Downloads
- Plugins
- User manual
- Licensing
- Donations

### Quick links

- FAQ
- Wiki
- Forums
- Forums (mobile)
- Nightlies
- Ticket System
- Browse SVN
- Browse SVN

## The open source, cross platform, free C, C++ and Fortran IDE.

Code::Blocks is a free C, C++ and Fortran IDE built to meet the most demanding needs of its users. It is designed to be very extensible and fully configurable.

Finally, an IDE with all the features you need, having a consistent look, feel and operation across platforms.

Built around a plugin framework, Code::Blocks can be extended with plugins. Any kind of functionality can be added by installing/coding a plugin. For instance, compiling and debugging functionality is already provided by plugins!

Special credits go to darmar for his great work on the **FortranProject** plugin, bundled since release 13.12.

We hope you enjoy using Code::Blocks!

The Code::Blocks Team

## Code::Blocks 17.12 is here!

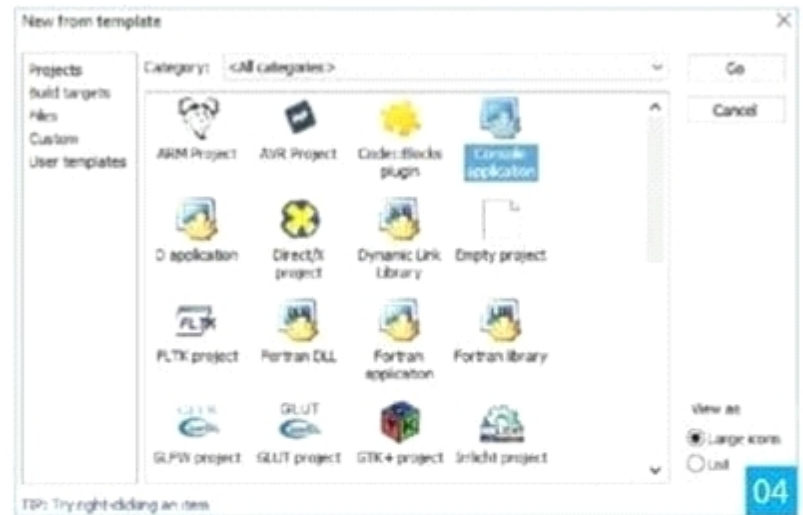
Written by MorienMacFly  
Saturday, 30 December 2017 16:34

Again, many improvements, new features, more stable, enhanced for Windows 10, the new Code::Blocks release 17.12 has arrived. Get it from the **downloads** section!

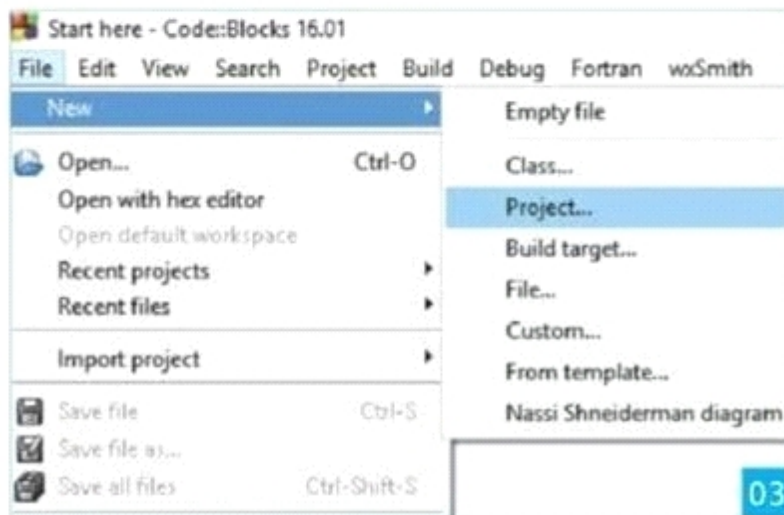
We provide binaries for the major platforms supported by Code::Blocks, with more to come in the next time.



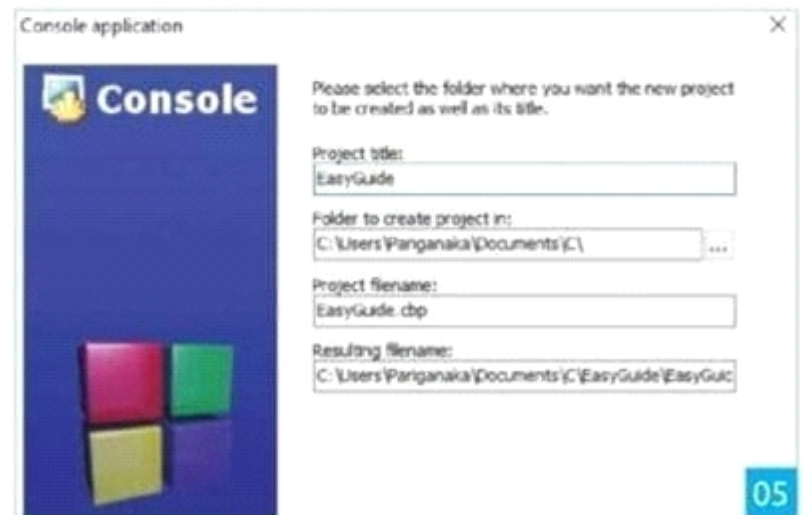
02



04



03



05

භාෂාවේ ඔබ්බෙන්ම ඔරියන්ටඩ් සාක්ෂරණය ලෙසින්ද ඇතැම් තැනකින් හඳුන්වා තිබේ. එම නිසා මෙහි ඉහළ මට්ටමේ භාෂාවල ගති ලක්ෂණ මෙන්ම පහළ මට්ටමේ භාෂාවල ගති ලක්ෂණද තිබේ. ජාවා, විෂුවල් ඩෙසික්, පයිතන් වැනි ක්‍රමලේඛ භාෂාවලින් සකස් කරන වැඩසටහන්වලට වඩා C++ භාෂාවෙන් සකස් කරන වැඩසටහන් වේගවත්ව ක්‍රියාත්මක වෙන්නේ මෙහිනිසාය.

• යුනික්ස් මෙහෙයුම් පද්ධතිය නිර්මාණය කිරීමට C භාෂාව මුලින් ම භාවිතා කරන ලදී. C භාෂාව පදනම් කර ගෙන නිර්මාණය කර ඇති C++ භාෂාව, C භාෂාවට වඩා හැකියාව අතින් උසස්ය. C භාෂාවට වඩා පහසුවෙන් උගත හැකිවීම C++ හි ඇති විශේෂත්වයකි. එමෙන්ම C භාෂාව භාවිතයෙන් කර ගනු ලබන වැඩ C++ භාෂාව භාවිතයෙන් කර ගත හැකිය.

• වින්ඩෝස් මෙහෙයුම් පද්ධතිය ඇති පරිගණකවලදී මෙන්ම ලිනක්ස් මෙහෙයුම් පද්ධති ඇති පරිගණකවලදීද එක සේ C++ භාෂාව භාවිතා කර වැඩසටහන් සකස් කර ගන්නට පුළුවන.

• C++ අක්ෂර සංවේදී (case sensitive) භාෂාවකි. මේ නිසා කේත හෙවත් කෝඩ් ලියන විට ඉංග්‍රීසි භාෂාවේ කැපිටල් සහ සිම්පල් අකුරු මුල් කේතයේ ඇති සේම ඒ ඒ තැනකින් යෙදිය යුතුමය.

• C++ කෝඩ් ලිවීම සඳහා ඕනෑම ටෙක්ස්ට් එඩිටරයක් යොදා ගන්නට පුළුවන. එහෙත් එසේ ලියනු ලබන කෝඩ් ක්‍රියාත්මක කරවා බලන්නට නම් කම්පයිලර් (compiler) එකක් අවශ්‍ය වේ.

#### කෝඩ් බ්ලොක්ස් අයිඩීඊ වැඩසටහන

C++ හිදී යොදා ගැනෙන්නේ ඒඑන්සු සී/සී++ කම්පයිලර් (GNU C/C++) එකයි. මේ නිසා ටෙක්ස්ට් එඩිටරයක් සහ කම්පයිලර් එකක් සහිත අයිඩීඊ (IDE - Integrated development environment) එකක් භාවිතා කළ යුතුය. කෝඩ් බ්ලොක්ස් (Code Blocks) යනු එවැනි අයිඩීඊ එකකි. එය යන වෙබ් අඩවියෙන් නොමිලේ ඩවුන්ලෝඩ් කර ගන්නට පුළුවන. (01 රූපය) ඕපන්සෝස් (opensource) අයිඩීඊ

එකක් වෙත මෙහි ඉන්ටර්ෆේස් එක පහසුවෙන් හැසිරවිය හැකි එකකි.

වින්ඩෝස් මෙහෙයුම් පද්ධතිය වෙනුවෙන් කෝඩ් බ්ලොක්ස් අයිඩීඊ වැඩසටහන ඩවුන්ලෝඩ් කර ගන්නා විට codeblocks-13.12 mingw-setup-TDM-GCC-481.exe යන නමින් ඇති සෙටප් ගයිල් එක ඩවුන්ලෝඩ් කර ගැනීම හොඳය. මෙම සෙටප් ගයිල් එකෙහි කම්පයිලර් වැඩසටහන්ද අඩංගු වෙන බැවිනි. එතැනම ඇති codeblocks-13.12-setup.exe යන්න ඩවුන්ලෝඩ් කළ හොත් කම්පයිලර් වැඩසටහන් ලැබෙන්නේ නැත.

කෝඩ් බ්ලොක්ස් අයිඩීඊ වැඩසටහන ඩවුන්ලෝඩ් කර ගත් විට ලැබෙන සෙටප් ගයිල් එක මත ඩබ්ල් ක්ලික් කර ඉන්ස්ටෝල්වීමට ඉඩ හැරිය යුතුය. අන්තර්ජාල සම්බන්ධතාව ඇති පරිගණකයකදී මෙම සෙටප් වැඩසටහන ඩවුන්ලෝඩ් කර ගෙන පෙන්ඩ්‍රයිව් එකක් වැනි එක්ස්ටර්නල් ඩ්‍රයිව් එකක රඳවා, වෙනත් ඕනෑම පරිගණකයක් වෙත ගෙන ගොස් ඉන්ස්ටෝල් කර ගන්නට පුළුවන. එසේ ඉන්ස්ටෝල් කර ගන්නට මෙන්ම වැඩසටහන ක්‍රියාත්මක වෙන්නට එම පරිගණක සමග අන්තර්ජාල සම්බන්ධතාව පැවතිය යුතු නැත.

කෝඩ් බ්ලොක්ස් අයිඩීඊ වැඩසටහන ඉන්ස්ටෝල් කළ පසු එය වෙත ඉබේම පිවිසේ. පසු අවස්ථාවකදී වින්ඩෝස් 10 මෙහෙයුම් පද්ධතිය ඔස්සේ ඒ වෙත පිවිසීම සඳහා Start මෙනු එක තුළින් Code Blocks අයිතක් එක තෝරා ගත යුතුය. වින්ඩෝස් 8 තුළදී නම් ස්ටාර් ස්ක්‍රීන් එක හරහා ගොස් ඊට අදාළ අයිතක් එක විවෘත කර ගත යුතුය. (02 රූපය)

#### අලුත් ප්‍රොජෙක්ට් එකක් ඇරඹීම

කෝඩ් බ්ලොක්ස් අයිඩීඊ වැඩසටහන ඉන්ස්ටෝල් කර ගැනීමෙන් අනතුරුව කෝඩ් ලිවීමට අලුත් ප්‍රොජෙක්ට් එකක් ලබා ගත යුතුය. ඒ සඳහා ඇප්ලිකේෂන් වින්ඩෝ එක මත දැනට ඇති Start Here වැඩ එක තුළින් Create a new project නම් ලිංක් එක ක්ලික් කළ යුතුය.



```

1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      cout << "Hello world!" << endl;
6      return 0;
7  }
8

```

06

```

1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      cout << "Wijaya";
6      cout << "Pariganaka";
7      return 0;
8  }
9

```

08

```

Hello world!

Process returned 0 (0x0)   execution time : 0.021 s
Press any key to continue.

```

07

```

WijayaPariganaka
Process returned 0 (0x0)   execution time : 0.017 s
Press any key to continue.

```

09

එසේ නැත්නම් File → New → Project යන්න හරහා යා යුතුය. (03 රූපය) එවිට New from template නම් බොක්ස් එක විවෘත වෙයි. එහි වම් පසින් ඇති Project යන්න තේරී තිබියදී Console application නම් අයිකන් එක තෝරා ගෙන Go බවින් එක මත ක්ලික් කළ යුතුය. (04 රූපය) එවිට Console application නම් බොක්ස් එක විවෘත වෙයි. මෙහි ඇති Next නම් බවින් එක ක්ලික් කර ඉදිරි පියවර වෙත යොමු විය යුතුය.

- ඊළඟ පියවරේදී විවෘත වෙත ලිස්ට් බොක්ස් එක තුළ C සහ C++ යන භාෂා දෙකෙහි නම් දැක ගන්නට පුළුවන. එය තුළින් C++ යන්න තෝරා නැවත වරක් Next බවින් එක ක්ලික් කළ යුතුය.

- ඊළඟට ප්‍රොජෙක්ට් එකෙහි විස්තර ලබා දිය යුතුය. එහිදී Project title යන තැන ප්‍රොජෙක්ට් එක සඳහා සුදුසු නමක් ටයිප් කළ යුතුය. එහි ඇති Folder to create project යන තැනින් ප්‍රොජෙක්ට් එක සේව් විය යුතු ෆෝල්ඩර් එක තෝරා දිය යුතුය. එහි ඇති Project file name යන තැන ප්‍රොජෙක්ට් එකෙහි ප්‍රධාන ගිණිල් එක සේව් වෙන නම ඉබේ සකස් වෙනු පෙනේවි. එය සකස් වෙන්නේ ප්‍රොජෙක්ට් එක සඳහා තමා ලබා දෙන මාතෘකාවට අනුවයි. එහෙත් අවශ්‍යතාවක් වෙතොත් එය වෙනස් කර ගන්නට පුළුවන. එසේ නම් වෙනස් කරන්නේ නම් .cbp ලෙස නමෙහි අගට ඇති ප්‍රොජෙක්ට් එකෙහි එක්ස්ටෙන්ෂන් එක වෙනස් නොකළ යුතුය. (05 රූපය)

- එම සැකසුමෙන් පසුව Next බවින් එක ක්ලික් කර ඉදිරි පියවර වෙත යොමු වෙමින් ගොස් අවසානයේදී Finish බවින් එක ක්ලික් කළ යුතුය. ප්‍රොජෙක්ට් එක සකස් කර සේව් කර ගැනීමේ කාර්ය එතැනින් අවසන් වෙයි.

#### කේත ලිවීම

මෙසේ ප්‍රොජෙක්ට් එක සකස් කිරීමෙන් පසුව කෝඩ් බ්ලොක්ස් අයිඩී එකෙහි වම් පස ඇති Management පැනල් එක තුළින්

Projects වැඩි එක සිලෙක්ට් කළ යුතුය. එය යටතේ ඇති Work space යන්නට පහළින් සකස් කළ ප්‍රොජෙක්ට් එකෙහි නම පෙන්වයි. එම ප්‍රොජෙක්ට් එකෙහි නමට ඉදිරියෙන් ඇති + ලකුණ ක්ලික් කළ විට එය යටතේ ඇති Source ෆෝල්ඩර් එක පෙනේ. එය ඉදිරි යේ ඇති + ලකුණ ක්ලික් කර එය යටතේ ඇති main.cpp නම් ගිණිල් එක මත ඩබල් ක්ලික් කළ විට සකස් කර ඇති C++ කෝඩ් පේළි කීපයක් තිරය මත පෙන්වයි. (06 රූපය) අලුත් ප්‍රොජෙක්ට් එකට අදාළ C++ කෝඩ් ලිවිය යුත්තේ මෙසේ කෝඩ් පෙන්වා ඇති තැනයි. එය main.cpp ගිණිල් එක තුළ සේව් වෙන බැවින් කෝඩ් වින්ඩෝ එක මත එම නමින්ම තිබෙන වැඩි එක යටතේ එය තිබෙනු පෙනේ.

මෙම මූලික C++ කෝඩ් ලියා තිබෙන්නේ Hello World යන වගන්තිය පෙන්වීම හෙවත් අවුට්පුට් කිරීම සඳහාය. එහිදී ක්ලික් සකස් කර ඇති කෝඩ් සහිත ගිණිල් හෙවත් හෙඩර් ගිණිල් (header file) අපගේ වැඩසටහන වෙතට සම්බන්ධ කර දෙන්නේ පළමු පේළිය මගිනි. එහිදී සම්බන්ධ කර ගන්නේ iostream නම් ගිණිල් එකයි. අප සකස් කරන ක්‍රමලේඛයේ ඇති මූලික ආදාන හෙවත් ඉන්පුට් (input) සහ ප්‍රතිදාන හෙවත් අවුට්පුට් (output) වලට එමගින් ඉඩ සලසා දේ. මෙම කෝඩ් තුළ ඇති දෙවැනි පේළියේදී namespace යන්න භාවිතා කර තිබේ. එය භාවිතා කළේ නැත්නම් පස් වැනි පේළියේ ඇති cout (සිදුවු) නම් කෝඩ් එක ලියන විට එහි ඉදිරියට std:: යන කොටස එකතු කළ යුතුය.

මෙම කෝඩ් එක තුළ ඇති තුන් වැනි පේළියේ තිබෙන්නේ int main() යන්නයි. C++ භාෂාවේ ප්‍රධාන ෆන්ක්ෂන් (function) එක මෙම කෝඩ් කොටසින් නිරූපණය වේ. සංකීර්ණ කෝඩ් එකක් කොටස්වලට වෙන් කොට පහසුවෙන් පරිපාලනය කරන්නට එය උදව් කරයි. මෙම ෆන්ක්ෂන් එකෙහි int ලෙස යොදා ඇත්තේ ෆන්ක්ෂන් එකට අදාළ දත්ත වර්ගයයි. මෙය, ෆන්ක්ෂන් එක තුළ



```

main.cpp x
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     cout << "Wijeya\n";
6     cout << "Pariganaka";
7     return(0);
8 }

```

C:\Users\Pariganaka\Documents\EasyGuide\src\Debug\EasyGuide.exe

Wijeya  
Pariganaka  
Process returned 0 (0x0) execution time : 0.043 s  
Press any key to continue.

```

main.cpp x
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     cout << "Pariganaka"; // print Pariganaka
6     return(0);
7 }

```

C:\Users\Pariganaka\Documents\EasyGuide\src\Debug\EasyGuide.exe

Pariganaka  
Process returned 0 (0x0) execution time : 0.030 s  
Press any key to continue.

ක්‍රියාත්මක වෙන කෝඩ් කොටස්වලට අදාළ නැත. මෙසේ ෆන්ක්ෂන් එක ආරම්භ කිරීමෙන් පසුව ඊට අදාළ කෝඩ් සියල්ල සහළ වරහන් ({} සහ) තුළ ලිවිය යුතුය.

මෙම කෝඩ් තුළ ඇති 4 වැනි පේළියේ ලියා තිබෙන්නේ ෆන්ක්ෂන් එක මගින් අප කර ගන්නා කටයුත්තයි. එහිදී අවුට්පුට් එකක් ලබා ගැනීමට අවශ්‍ය බැවින් cout නම් කෝඩ් එක එක ලියා ඇත. එහි ඇති cout << යන ලකුණුවලට පසුව උදාහරණයක් ලෙස ලියා තිබෙන්නේ අවුට්පුට් විය යුතු වචනයයි. ඉන්පසුව තිත් කොමා (;) ලකුණ යොදා එම පේළිය අවසන් කර ඇත. C++ තුළදී ක්‍රියාත්මක වෙන සෑම පේළියක්ම අවසන් කළ යුත්තේ මෙවැනි තිත් කොමා ලකුණකිනි.

මෙම මූලික කෝඩ් ලියූ පසු වැඩසටහන ක්‍රියාත්මක කර බලන්නට පුළුවන. ඒ සඳහා කෝඩ් බ්ලොක්ස් වින්ඩෝ එකෙහි ඉහළින් ඇති ටූල් බාර් එක තුළ තිබෙන Build and run නම් බවින් එක ක්ලික් කළ යුතුය. මෙය කොළ පැහැති ඊතල ලකුණක් සමඟ කහ පැහැති කුඩා දැඩි රෝදයක අයිතනයකින් පෙන්වයි. මෙය ක්ලික් කළ විට එය ක්‍රියාත්මක වෙන වින්ඩෝ එක විවෘත වේ. මෙය කමාන්ඩ් ප්‍රොම්ට් එක හරහා ක්‍රියාත්මක වුවත් යතුරු පුවරුවේ ඕනෑම යතුරක් තද කර එම වින්ඩෝ එක වසා දමන්නට පුළුවන. (07 රූපය) මෙහිදී Hello World යන්න පෙන්වයි. එනම් අවුට්පුට් වෙයි.

#### පේළි දෙකක් හෝ කීපයක් හෝ අවුට්පුට් කර ගැනීම

ඉහත විස්තර කළ සරල වැඩසටහන තුළදී අවුට්පුට් කරන ලද්දේ Hello World! යන වගන්තිය පමණි. මේ වෙනුවට වෙනත් වගන්තියක් අවුට්පුට් කරන්නට අවශ්‍ය නම් cout යන්නට ඉදිරියෙන් අවුට්පුට් කිරීමට අවශ්‍ය දෙය ලබාදී වැඩසටහන ක්‍රියාත්මක කරවා බැලිය යුතුය. ඒ අනුව cout යන්න නැවත නැවත ලබාදෙමින් වුවද අවුට්පුට් කිරීමට අවශ්‍ය දේ ලබා දෙන්නට හැකි වේ. මෙය තේරුම් ගැනීම සඳහා පහත දැක්වෙන කෝඩ් වෙත අවධානය යොමු කරන්න. (08 රූපය)

```

#include <iostream>
using namespace std;
int main()
{
    cout << "Wijeya";
    cout << "Pariganaka";
    return(0);
}

```

● මෙහිදී බැලූ බැල්මට cout යන්න දෙවරක් යොදා ඇති නිසා Wijeya සහ Pariganaka යන වචන පේළි දෙකකදී අවුට්පුට් වෙන බව තේරුම් ගියත් සැබැවින්ම මෙම වචන දෙක අවුට්පුට් වෙන්නේ තනි පෙළකටය. (09 රූපය)

ඊට හේතුව වෙන වෙනම cout යන්න යෙදවූද පේළි දෙකකදී අවුට්පුට් ලබා ගැනීමට අවශ්‍ය නම් ඒ බව කේතවලින් දැනුම්දිය යුතු වීමයි. ඒ අනුව ඒ සඳහා ඉහත කෝඩ් ලිවිය යුත්තේ මෙසේය.

```

cout << "Wijeya\n";
cout << "Pariganaka";

```

● මෙහිදී පළමු පේළියේ අවසානයේදී පසුපසට ඇලවූ කඩ ඉර (\) හෙවත් බැක්ස්ලෑෂ් (backslash) ලකුණ සමඟ n අකුර යොදා ඇති ආකාරය නිරීක්ෂණය කරන්න.

● පළමු පේළිය අවසානයේදී පහළ පේළිය වෙත යොමු කිරීම එයින් කරයි. මෙහිදී මෙම \n යන්න අවුට්පුට් වෙන්නේ නැත. එහි ක්‍රියාව වෙන අලුත් පේළියකට යොමුවීම පමණක් ක්‍රියාත්මක වේ. මෙසේ \n යන්න යොදන විට එය උදාහරණයක් ලෙස තුළ යෙදිය යුතුමය. එය අකුරු අඩංගු පේළි තුළදී ක්‍රියාත්මක වෙන කෝඩ් කොටසක් වෙත බැවිනි.

● එසේම අලුත් පේළියක් වෙත යොමුවීම සඳහා endl නම් කෝඩ් එකද භාවිතා කරන්නට පුළුවන. එය ඉහත විස්තර කළ කෝඩ් කොටස මෙන් අකුරු පේළියක් තුළ ක්‍රියාත්මක වෙන කෝඩ් එකක් නොවේ. එය භාවිතා කළ යුත්තේ පහත දැක්වා ඇති සේය.

```

cout << "Wijeya" << endl;
cout << "Pariganaka";

```

● මෙහි endl යනු end new line යන්න දැක්වෙන කෝඩ් එකකි. පළමු පේළිය අවසන් කර දෙවැනි පේළිය වෙත යොමු විය යුතු බව වැඩ සටහන වෙත දක්වා සිටින්නේ එමගිනි. (10 රූපය)

#### කමෙන්ට් භාවිතා කිරීම

කමෙන්ට් එකකින් කරන්නේ ක්‍රියාත්මක විය යුතු නැති වචන පෙළක් කෝඩ් වින්ඩෝ එක මත තැබීමයි. මේවා අප විසින් ලියනු ලබන කෝඩ් විස්තර කර තැබීම සඳහා යොදා ගන්නට පුළුවන. එසේ යම් කෝඩ් එකක් විස්තර කර තැබූ විට පසු අවස්ථාවකදී වුව එම කෝඩ් එක පහසුවෙන් හඳුනා ගත හැකිය. එමෙන්ම කෝඩ් අත්හදා බලන විට තාවකාලිකව කෝඩ් පේළි අක්‍රීය කර තැබීමටද කමෙන්ට් යොදා ගන්නට පුළුවන. සෑම පරිගණක භාෂාවකමදී කමෙන්ට් භාවිතා වේ. එහෙත් විවිධ ක්‍රමලේඛ භාෂාවලදී මෙම කමෙන්ට් භාවිතා කරන ක්‍රම විවිධය.

තනි පේළියක් සඳහා කමෙන්ට් යොදන විට කමෙන්ට් යෙදිය යුතු පේළියේ මුලින් හෝ කමෙන්ට් එක ආරම්භ විය යුතු නැතිනම් හෝ ඇල ඉරි හෙවත් ස්ලෑෂ් (slash) ලකුණ දෙකක් (//) සටහන් කළ යුතුය. එවිට එම පේළියේ එතැනින් ඉදිරියට ඇති කොටස කමෙන්ට් එකක් බවට පත් වෙයි. මෙය තේරුම් ගැනීම සඳහා cout << "Pariganaka"; // print Pariganaka යන කෝඩ් පේළිය වෙත අවධානය යොමු කරන්න. මෙහි print Pariganaka යන්නට කලින් යොදා ඇති ඇල ඉරි දෙක නිසා එම කොටස කමෙන්ට් එකක් බවට සකස් වෙයි. ඒ අනුව

```

main.cpp
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      /* print
6      Wijeya
7      Pariganaka
8      */
9      return(0);
10 }

```

Process returned 0 (0x0) execution time : 0.020 s  
Press any key to continue.

ඊට අදාළ අවුට්පුට් ලැබෙන්නේ නැත. (11 රූපය)

කමෙන්ට් එක ලෙස සකස් කළ යුත්තේ ජේලි කීපයක් නම් ඇල ඉරි ලකුණට පසුව තරු ලකුණ යොදා (/\*) කමෙන්ට් එක ආරම්භ කළ යුතුය. එහිදී එම කමෙන්ට් එක අනිවාර්යෙන්ම අවසන් කළ යුතුය. ඒ සඳහා තරු ලකුණට පසුව ඇල ඉරි ලකුණ යෙදිය යුතුය. (\*/) මෙය තේරුම් ගැනීම සඳහා පහත දැක්වෙන කෝඩ් වෙත අවධානය යොමු කරන්න.

```

/* print
Wijeya
Pariganaka
*/

```

මෙසේ යොදන කෝඩ් ජේලි වැඩසටහන තුළ ක්‍රියාත්මක වෙන්නේ නැත. (12 රූපය)

## දෙවැනි පරිච්ඡේදය

### වැඩසටහනක් තුළදී වේරියබල් භාවිතා කරන ආකාරය

විලාස හෙවත් වේරියබල්ස් (variables) යනු සෑම පරිගණක භාෂා වකමකම පාහේ භාවිතා කරන නිශ්චිත විධාන ක්‍රමයකි. වේරියබල් එකකින් කෙරෙන්නේ යම් ක්‍රියාවක් සඳහා පරිගණකයේ ඇති තාවකාලික මතකයෙන් හෙවත් රැම් (RAM) මතකයෙන් ඉඩ කොටසක් තාවකාලිකව වෙන් කර ගැනීමයි. මෙම ඉඩ කොටස තුළ දත්ත බෙඩා කර තබා වැඩසටහනක් ක්‍රියාත්මක වෙන විට ප්‍රයෝජනයට ගත හැක.

#### වේරියබල් එකක සැකැස්ම

වේරියබල් එකක් සැකසෙන්නේ කොටස් දෙකකිනි. එනම් int Pariganaka; වශයෙනි. මෙහි මුලින් ඇති int යන්න බේටා ටයිප් එකයි. එමඟින් වේරියබල් එක තුළට ඇතුළු කරන දත්ත වර්ගය සහ දත්ත වල ප්‍රමාණය තීරණය කරනු ලබයි. මෙහි Pariganaka ලෙස දක්වා ඇත්තේ වේරියබල් එකෙහි නමයි. මෙතැන කැමති නමක් ලිවිය හැක.

#### වේරියබල් නම් කරන විට සැලකිය යුතු කරුණු

වේරියබල් එකක් කැමති නමක් යොදා නම් කළ හැකි වුවත් එහි නිශ්චිත ක්‍රමවේදයක් වෙයි. එහිදී සැලකිලිමත්විය යුතු කරුණු මෙසේය.

- වේරියබල් එකක නම ආරම්භ කළ යුත්තේ අකුරකිනි. ඒ සඳහා ඉලක්කමක් හෝ වෙනත් සලකුණක් හෝ භාවිතා කළ නොහැක. අවශ්‍ය නම් වේරියබල් එකෙහි මැදට හෝ අවසානයට හෝ ඉලක්කම් යෙදිය හැක.
- යොදන නමෙහි කොටස් දෙකක් අතර හිස් ඉඩ නොතැබිය යුතුය. හිස් තැන් වෙනුවට යට කඩ ඉරි ලකුණ ( \_ ) යෙදිය හැක.
- වේරියබල් එකක නම වශයෙන් %, &, !, # වැනි සලකුණු භාවිතා කළ නොහැක.
- වේරියබල් නමක් ලිවීමට භාවිතා කළ හැකි උපරිම අකුරු සංඛ්‍යාව 255 කි.

- වේරියබල් එකක නමක් ලෙස C++ තුළ වෙනත් කටයුත්තක් සඳහා වෙන් කර ඇති cout, return, std, වැනි වචන භාවිතා කළ නොහැක.

- මෙම ක්‍රමවේදයට අනුගත වේරියබල් නම් කීපයක් පහත දැක්වේ.

- Pariganaka\_Magazine
- PariganakaMagazine
- Pariganaka123

#### වැඩසටහනක් තුළදී වේරියබල් භාවිතා කරන අයුරු

වැඩසටහනක් තුළදී වේරියබල් භාවිතා කෙරෙන්නේ කෙසේදැයි අවබෝධ කර ගැනීම පිණිස පහත දක්වා ඇති වැඩසටහන අධ්‍යයනය කරන්න. මෙහි 3, 4, 5, යන ජේලිවල ඇති int a; / int b; / int result; යන කේත කොටස් මගින් a, b සහ result ලෙස වේරියබල් තුනක් සකස් කර ඇත. එම වේරියබල් සකස් කරන විට ඒවායේ දත්ත වර්ගය ලෙස int යන්න ලබාදී තිබෙයි.

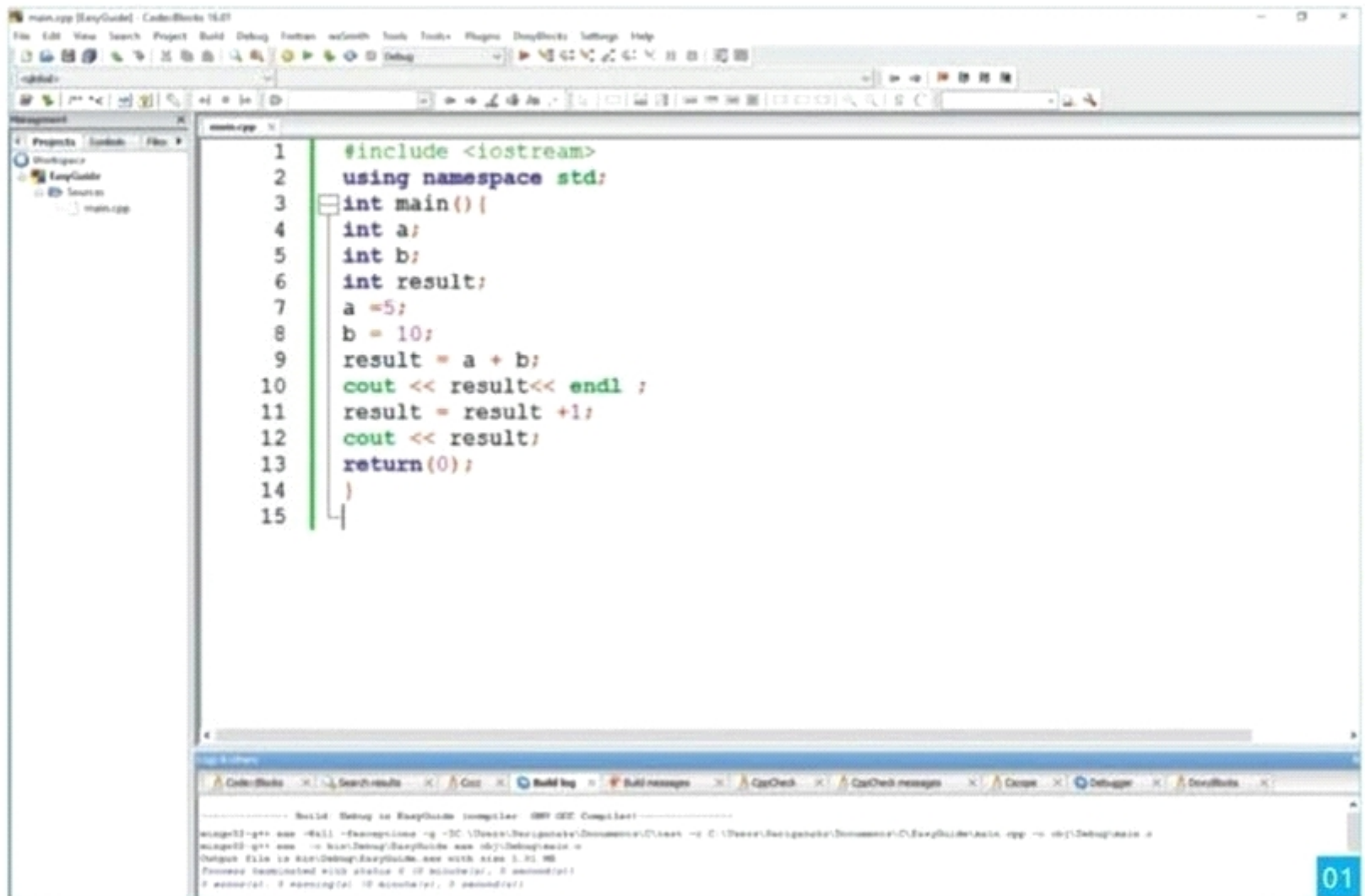
වේරියබල් එකක් සකස් කරන විට ඊට උචිත දත්ත වර්ගය අප විසින් තෝරා ගත යුතුය. ඒ සඳහා එක් එක් දත්ත වර්ගය ගැන හොඳ අවබෝධයක් තිබිය යුතුය. එසේ එක් එක් දත්ත වර්ගය ගැන අවබෝධයක් ලබා ගන්නට කලින් මෙවැනි මූලික වැඩසටහන් කීපයක් විමසා බලා වේරියබල් එකක් ක්‍රියාත්මක වෙන ආකාරය ගැන දැනුම ලබා සිටිය යුතුය. (02 රූපය)

```

#include <iostream>
using namespace std;
int main(){
    int a;
    int b;
    int result;

```





01

```

a = 5;
b = 10;
result = a + b;
cout << result << endl;
result = result + 1;
cout << result;
return(0);
}

```

මෙම කේත සටහනේ 3,4,5 යන පේළිවලදී int නම් දත්ත වර්ගය සඳහා පමණක් යොදා ගෙන මෙම වේරියබල් තුනම සකස් කර තිබේ. මෙහිදී ඒවා වෙන වෙනම පේළිවල සටහන් නොකර තනි පෙළක් ලෙස කේත ලිවීමද කළ හැක. එසේ තනි පෙළක වේරියබල් කීපයක් ලබා දෙන විට එක් එක් වේරියබල්වල නම් වෙන් කිරීම සඳහා කොමා ලකුණ (,) යෙදිය යුතුය. මෙම වැඩසටහනේ 3, 4, 5, යන පේළිවල ඇති වේරියබල් තුන තනි පෙළකට ලියන විටදී int a, b, result; ලෙස එය ලියා තිබීම නිදසුනකි. මෙහිදී int යන්න නැවත නැවත යෙදිය යුතු නැත.

මෙහි හත් වැනි පේළියේ ඇති a=5; යන්න මගින් a නම් වේරියබල් එකට 5 යන අගය ඇතුළත් කරයි. මෙහි 8 වැනි පේළියේ ඇති b = 10; යන්න මගින් b නම් වේරියබල් එකට 10 යන අගය ඇතුළත් කරයි. මෙහි 9 වැනි පේළියේ ඇති result = a + b; යන්න මගින් a සහ b වේරියබල් දෙක එකිනෙක හා එකතු කොට එම අගය result නම් වේරියබල් එක තුළට ඇතුළත් කරයි. මෙහි 10 වැනි පේළියේ ඇති cout << result යන්න මගින් එම result නම් වේරියබල් එක තුළ ඇති a සහ b වේරියබල් දෙක එකිනෙක හා එකතු කළ විට ලැබෙන අගය තිරයෙන් පෙන්වයි. මෙහිදී පිළිතුර ලෙස ,15, යන්න තිරයේ ප්‍රින්ට් වෙයි.

මෙම වැඩසටහනේ 11 වැනි පේළියේ ඇති result = result + 1; යන කේත කොටස මගින් result නම් වේරියබල් එක තුළ ඇති අගයට, 1ක් එකතු කරයි. මේ ආකාරයට ඕනෑම වේරියබල් එකක් තුළ ඇති අගයකට, තවත් ඕනෑම සංඛ්‍යාවක් එකතු කිරීම, ඉන් අඩු කිරීම, වැඩි කිරීම හෝ බෙදීම කළ හැක.

මෙම වැඩසටහනේ 12 වැනි පේළිය මගින් නැවතත් result නම් වේරියබල් එක තුළ ඇති අගය තිරයේ ප්‍රින්ට් කරයි. එහෙත් 11 වැනි පේළියේ ඇති result = result + 1; යන කේත කොටස මගින් result නම් වේරියබල් එක තුළ ඇති අගයට 1ක් එකතු කර ඇති බැවින් මෙහි පිළිතුර ලෙස 16 යන්න අවුට්පුට් වෙයි. ඒ අනුව වැඩසටහන ක්‍රියාත්මක වෙන විට පළමු පේළියේ 15 යන්නත්, දෙවැනි පේළියේදී 16 යන්නත් අවුට්පුට් ලෙස ලැබෙයි. (02 රූපය)

වේරියබල් භාවිතා කර විවිධ ගණනය කිරීම් කරන ආකාරය අවබෝධ කර ගන්නට පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න. (03 රූපය)

```

#include <iostream>
using namespace std;
int main() {
    int a;
    a = 50;
    cout << a << endl;
    cout << a+a << endl;
    cout << a-a << endl;
    cout << a/a << endl;
    cout << 2*a << endl;
    return(0);
}

```

```
main.cpp X
C:\Users\Panganaka\Documents\C++\EasyGuide\bin\Debug\EasyGuide.exe
15
16
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

02

```
main.cpp X
C:\Users\Panganaka\Documents\C++\EasyGuide\bin\Debug\EasyGuide.exe
50
100
0
1
100
Process returned 0 (0x0)   execution time : 0.024 s
Press any key to continue.
```

04

```
main.cpp X
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int a;
5      a = 50;
6      cout << a << endl;
7      cout << a+a << endl;
8      cout << a-a << endl;
9      cout << a/a << endl;
10     cout << 2*a << endl;
11     return(0);
12 }
13
```

03

```
main.cpp X
1  #include <iostream>
2  using namespace std;
3  int main() {
4      cout << sizeof(char) << endl;
5      cout << sizeof(int) << endl;
6      cout << sizeof(short int) << endl;
7      cout << sizeof(long int) << endl;
8      cout << sizeof(float) << endl;
9      cout << sizeof(double) << endl;
10     cout << sizeof(bool) << endl;
11     return(0);
12 }
13
```

05

මෙම වැඩසටහනේ 4 වැනි පේළියේ ඇති කේත මගින් a නමින් වේරියබල් එකක් සකස් කර ඇත. මෙහි 5 වැනි පේළියේ ඇති a = 50; යන්න මගින් එම වේරියබල් එක තුළට 50 යන අගය ඇතුළු කරනු ලබයි.

මෙහිම වැඩසටහනේ 6 වැනි පේළියේ කේත මගින් a නම් වේරියබල් එකෙහි දූතට ගබඩාවී ඇති අගය තිරයේ පින්ට් කරයි. ඒ අනුව එහි ගබඩා කළ 50 යන අගය එලෙසින්ම මෙහිදී අඩුට්ටුව වේ.

මෙහි 7 වැනි පේළියේ ඇති ක්‍රියාත්මක වෙන කොටස තුළින් a වේරියබල් එකෙහි ඇති අගයට එම වේරියබල් එකම එකතු කරයි. ඒ අනුව එම වේරියබල් එකෙහි දූතට තිබුණු අගය වෙන 50ට තවත් 50ක් එකතු වෙයි.

මෙහි 8 වැනි පේළියේ ඇති (a-20) යන්න තුළින් a වේරියබල් එකෙහි ඇති අගයෙන් 20ක් අඩු කරයි.

මෙහි 9 වැනි පේළියේ ඇති (a/2) යන්න මගින් a නම් වේරියබල් එකෙහි ඇති අගය 2න් බෙදයි. එසේම මෙහි ඇති 2෦ යන්න තුළින් a වේරියබල් එක තුළ ඇති අගය 2න් ගුණ කරයි.

මෙම වැඩසටහන ක්‍රියාත්මක වෙන විට මෙම එක් එක් පේළියට අදාළව අඩුට්ටුව ලැබෙනු පෙනේ. (04 රූපය)

#### වේරියබල් සමඟ ඩේටා ටයිප් භාවිතා කරන අයුරු

වේරියබල් එකකින් කෙරෙන්නේ යම් ක්‍රියාවක් සඳහා පරිගණකයේ ඇති තාවකාලික මතකයෙන් යම් ඉඩ කොටසක් වෙන් කර ගැනීම බව දැනුම. වේරියබල් විධානයක් සැකසී තිබෙන ඩේටා ටයිප් එක සහ වේරියබල් එකෙහි නම යන මූලික කොටස් දෙක වෙනස් කර විවිධ වේරියබල්ස් විධාන ලිවිය හැක. මෙහිදී ආදර්ශයක් වශයෙන් භාවිතා කරන int Pariganaka; යන වේරියබල් එක සැලකූ විට එහි int යන්න ඩේටා ටයිප් එකයි. මෙවැනි ඩේටා ටයිප් ගණනාවක් වෙයි. ඒවා එ-කිනෙකින් වෙනස් වෙන්නේ දත්තය වශයෙන් යොදා ගන්නා කවරක්

ද යන්න මතය. ඉන් වැඩිපුරම භාවිතා කරන ඒවා මෙසේය.

• **int** - දශමස්ථාන නොමැති පූර්ණ සංඛ්‍යා පමණක් යම් වේරියබල් එකක ගබඩා කිරීමට අවශ්‍ය නම් යොදා ගන්නේ මෙම දත්ත වර්ගයයි. මෙහිදී -2147483648 සිට 2147483647 දක්වා වෙන සංඛ්‍යා ගබඩා කරන්නට පුළුවන. මේ සඳහා බයිට් 4ක ඉඩ වැයවේ.

• **unsigned int** - දශමස්ථාන නොමැති ධන සංඛ්‍යා පමණක් ගබඩා කිරීමට පුළුවන. එහිදී 0 සිට 4294967295 දක්වා පරාසයක සංඛ්‍යා ගබඩා කරන්නට පුළුවන. මේ සඳහා බයිට් 4ක ඉඩ වැයවේ.

• **short int** - -32768 සිට 32767 දක්වා පමණක් පූර්ණ සංඛ්‍යා ගබඩා කළ හැකිය. මේ සඳහා බයිට් 2ක ඉඩ වැයවේ.

• **long int** - විශාල සංඛ්‍යා පරාසයක ඇති පූර්ණ සංඛ්‍යා පමණක් ගබඩා කිරීමට යොදා ගනී. මෙහි පරාසය -2,147,483,647 සිට 2,147,483,647 දක්වා වේ. මේ සඳහා බයිට් 4ක ඉඩ වැය වේ.

• **unsigned long int** - විශාල ප්‍රමාණයේ ධන සංඛ්‍යා පමණක් ගබඩා කිරීමට යොදා ගනී. මෙහි පරාසය 0 සිට 4,294,967,295 දක්වාය. මේ සඳහා බයිට් 4ක ඉඩ වැය වේ.

• **float** - දශම සංඛ්‍යා ගබඩා කිරීම සඳහා මෙය යොදා ගනී. මෙහිදී දශමස්ථාන 7 ක් දක්වා සංඛ්‍යා ගබඩා කළ හැක. මේ සඳහා බයිට් 4ක ඉඩ වැය වේ.

• **double** - float වලට වඩා විශාල දශම සංඛ්‍යා පමණක් ගබඩා කිරීම සඳහා මෙය යොදා ගැනෙයි. මෙහිදී දශමස්ථාන 15 ක් දක්වා සංඛ්‍යා ගබඩා කළ හැක. මේ සඳහා බයිට් 8 ක ඉඩ වැය වේ.

• **char** - වේරියබල් එකක තනි අක්ෂරයක් පමණක් ගබඩා කිරීමට අවශ්‍යවූ විට මෙම දත්ත වර්ගය යොදා ගත යුතුය. මේ සඳහා බයිට් 1 ක ඉඩ වැය වේ.

• **string** - අකුරු වැඩි ප්‍රමාණයක් ඇතුළු කළ හැක. මේ සඳහා වැයවෙන ඉඩ අකුරු ගණන අනුව වෙනස් වේ.



```
main.cpp x
C:\Users\Parigarak\Documents\C\EasyGuide\bin\Debug\EasyGuide.exe

1
4
2
4
4
8
1

Process returned 0 (0x0)   execution time : 0.031 s
Press any key to continue.
```

06

• **bool** - වේරියබල් එකක ගබඩා කිරීමට බලාපොරොත්තු වෙන්නේ ඔබ හෝ නැත හෝ යන පිළිතුරු දෙකෙන් එකක් පමණක් නම් මෙය භාවිතා කරන්නට පුළුවන. මේ සඳහා බයිට් 1 ක ඉඩ වැය වේ.

මෙම ප්‍රධාන වේදිකා ටයිප්වලට අමතරව unsigned char, signed char, unsigned int, signed int, signed short int, signed long int, long double, wchar\_t යන දත්ත වර්ගද C++ තුළදී භාවිතා වේ.

#### වේරියබල් වෙත දත්ත ඇතුළු කිරීම

ඉහත විස්තර කළ ආකාරයට අවශ්‍ය දත්ත වර්ගය යොදා වේරියබල් සකස් කර ගැනීමෙන් පසුව ඊට දත්ත ඇතුළු කළ හැක. සංඛ්‍යා භාවිතා වෙත වේරියබල් එකක් සඳහා දත්ත ඇතුළු කරන්නේ පහත දක්වා ඇති ආකාරයටයි.

```
int a;
a = 10;
```

මෙහි දෙවැනි පේළියේදී a නම් වේරියබල් එකට දත්ත ඇතුළු කර ඇති ආකාරයට ඕනෑම සංඛ්‍යාත්මක දත්තයක් වේරියබල් එකකට ඇතුළු කළ හැකිය. මීට අමතරව වේරියබල් එක සකස් කරන විටදීම ඊට දත්ත ඇතුළු කිරීමද කළ හැකිය. ඒ සඳහා ලෙස int b (100); යන කෝඩ් එක ලිවිය යුතුය. වේරියබල් එක තුළට ඇතුළු කළ යුතු සංඛ්‍යාව මෙහි වරහන් තුළ ලියා තිබේ. එහෙත් වැඩසටහන් සකස් කරන විට විටින් විට වේරියබල් සඳහා දත්ත ඇතුළු කිරීමට අවශ්‍ය වෙත බැවින් මෙම ක්‍රමය සෑම විටදීම යොදා ගන්නට බැරිය.

යම් වේරියබල් එකක් තුළ එක් අකුරක් හෝ සංකේතයක් ගබඩා කිරීම සඳහා char වේදිකා ටයිප් එක භාවිතා කරන බව අප දනිමු. එ වැනි වේරියබල් එකකට දත්ත ඇතුළු කරන්නේ පහත දක්වා ඇති සේය.

```
char b;
b = 'c';
```

මෙහිදී පළමු පේළියේ සකස් කර ඇති b නම් වේරියබල් එක සඳහා දෙවැනි පේළියේදී c අකුර ඇතුළු කර තිබේ. මේ ආකාරයෙන් එක් අකුරක් හෝ සංකේතයක් හෝ වේරියබල් එකක් තුළට ඇතුළු කරන විට තනි උදාහරණය ලකුණු දෙකක් තුළ එම අකුර හෝ සංකේතය හෝ ලිවිය යුතුය. මෙහිදීද වේරියබල් එක සැකසීම සහ දත්ත ඇතුළු කිරීම තනි පෙළකදී වුවද කරන්නට පුළුවන. එහිදී ඉහත විස්තර කළ ආකාරයට අදාළ දත්තය වරහන් තුළ ලිවිය යුතුය. එ-හෙත් string වේදිකා ටයිප් එක ඇති වේරියබල් එකකට දත්ත ඇතුළු කරන ආකාරය මීට වෙනස්ය. එහිදී අකුරු දක්විය යුත්තේ උදාහරණය ලකුණ තුළය. එනම් පහත දක්වා ඇති සේය.

```
string a;
a = "abc";
```

එසේම ඉදහත දත්ත වර්ගය ඇති වේරියබල් එකක් සඳහා දත්ත ඇතුළු කරන්නේ මේ සියල්ලටම වඩා වෙනස් විදියකටය. මෙහිදී

```
*main.cpp x
1 #include <iostream>
2 using namespace std;
3 int main ()
4 {
5     int a, b;
6     int c;
7     a = 10;
8     b = 20;
9     c = a + b;
10    cout << c;
11    return 0;
12 }
```

07

වේරියබල් එක තුළට 'සත්‍යය' යන විධානය ඇතුළු කිරීමට true හෝ 1 යන්න ලිවිය යුතුය. 'අසත්‍ය' යන විධානය ඇතුළු කිරීමට false හෝ 0 යන්න ලිවිය යුතුය. මෙසේ bool දත්ත වර්ගය ඇති වේරියබල් එකකට සත්‍යය යන විධානය ඇතුළු කර ඇති ආකාරය පහතින් දක්වමු.

```
bool a;
a = true;
```

#### වැඩසටහනක් සකසා ක්‍රියාත්මක කරවීම

එක් එක් වේදිකා ටයිප් එකෙහි ප්‍රමාණය අවබෝධ කර ගනු වස් පහත දක්වා ඇති වැඩසටහන අප මෙහිදී භාවිතා කරන කෝඩ් බ්ලොක්ස් අයිඩීඑ එක තුළ ටයිප් කර ගන්න. ඉන්පසුව එහි කම්පයිල් ටූල් බාර් එකෙහි ඇති Build and Run යන බට්න් එක ක්ලික් කළ විට මෙම වැඩසටහන ක්‍රියාත්මක වෙයි. (05 රූපය)

```
#include <iostream>
using namespace std;
int main(){
    cout << sizeof(char) << endl;
    cout << sizeof(int) << endl;
    cout << sizeof(short int) << endl;
    cout << sizeof(long int) << endl;
    cout << sizeof(float) << endl;
    cout << sizeof(double) << endl;
    cout << sizeof(bool) << endl;
    return(0);
}
```

මෙම වැඩසටහන ක්‍රියාත්මක වන විට sizeof යන කීවර්ඩ් එකට ඉදිරියෙන් වරහන් තුළ ඇති එක් එක් දත්ත වර්ගවල ප්‍රමාණය දැක ගැනීමට පුළුවන. (06 රූපය)

#### ලෝකල් වේරියබල්

ලෝකල් වේරියබල් එකක් ක්‍රියාත්මක වෙන්නේ ෆන්ක්ෂන් (function) එකක් හෝ බ්ලොක් (block) එකක් හෝ තුළදී පමණි. ඒ අනුව ෆන්ක්ෂන් එකෙහි හෝ බ්ලොක් එකෙහි පිටත සිට හෝ වේරියබල් එක වෙත දත්ත ඇතුළු කිරීමටත්, එහි ඇති දත්ත භාවිතා කිරීමටත් නොහැකිය. මෙය තේරුම් ගැනීම සඳහා පහත දක්වා කෝඩ් අධ්‍යයනය කරන්න. (07 රූපය)

```
#include <iostream>
using namespace std;
int main ()
{
    int a, b;
```

```

main.cpp X
C:\Users\Parigana\Documents\C++\EasyGuide\bin\Debug\EasyGuide.exe
30
Process returned 0 (0x0)   execution time : 0.051 s
Press any key to continue.
08

```

```

int c;
a = 10;
b = 20;
c = a + b;
cout << c;
return 0;
}

```

මෙහි 5 සහ 6 පේළිවලදී a, b සහ c යන වේරියබල් තුන සකස් කර තිබේ. ඒවා සකස් කර තිබෙන්නේ 3 වැනි පේළියේදී ආරම්භ වෙන main බලාක් එක තුළය. එම නිසා එම වේරියබල් තුන ක්‍රියාත්මක වෙන්නේ එම බලාක් එක තුළදී පමණි. ලෝකල් වේරියබල් ලෙස හැඳින්වෙන්නේ මේවාය.

මෙහි 7 සහ 8 යන පේළිවලදී එම a, b යන වේරියබල් සඳහා පිළිවෙළින් 10 සහ 20 යන අගයන් ඇතුළු කර තිබේ. 9 වැනි පේළිය ලියා ඇත්තේ එම වේරියබල් දෙකෙහි ඇති අගයන් එකතු කළ විට ලැබෙන පිළිතුර c නම් වේරියබල් එක තුළ ගබඩා කිරීමටයි. ඉන්පසුව 10 වැනි පේළිය මගින් C වේරියබල් එකෙහි අගය අවුට්පුට් කරනු ලැබේ.

මෙහිදී සකස් කළ ලෝකල් වේරියබල් තුළට අගයන් ඇතුළු කිරීම සහ එම අගයන් අවුට්පුට් කිරීමට අවශ්‍ය කේත සියල්ල main බලාක් එක තුළම පමණක් ලියා ඇති නිසා වැඩසටහනේ ක්‍රියාකාරීත්වයට එය බලපාන්නේ නැත. මෙම කේත ක්‍රියාත්මක වීමෙන් පසුව ලැබෙන අවුට්පුට් 08 රූපයේ දක්වා ඇති සේය.

#### ග්ලෝබල් වේරියබල්

ග්ලෝබල් වේරියබල් එකක් සම්පූර්ණ කේත සටහන තුළදීම ක්‍රියාත්මක වේ. මේ නිසා කේත සටහන තුළ බලාක් සහ ෆන්ක්ෂන් කොපමණ තිබුණද සකස් කර ඇති ග්ලෝබල් වේරියබල් එකක් ඒ සියල්ල තුළදීම භාවිතා කරන්නට පුළුවන. ෆන්ක්ෂන් එකක් හෝ බලාක් එකක් තුළදී ගබඩා කරනු ලබන දත්ත හෝ තවත් ෆන්ක්ෂන් එකක් තුළදී මෙන්ම බලාක් එකක් තුළදී භාවිතා කිරීමට හැකිවීම ග්ලෝබල් වේරියබල්වල ඇති ප්‍රධාන වාසියකි. එහෙත් වැඩසටහන ආරම්භ වෙන විටම මේවාට අවශ්‍ය මතක ධාරිතාව ලබා ගන්නා නිසා වැඩසටහනක් සකසන විට අවශ්‍යම තැන්වලට පමණක් ග්ලෝබල් වේරියබල් භාවිතා කර, අනෙක් තැන්වලට ලෝකල් වේරියබල් භාවිතා කළ යුතුය. මෙම ග්ලෝබල් වේරියබල් භාවිතා කරන ආකාරය තේරුම් ගැනීම සඳහා පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න. (09 රූපය)

```

#include <iostream>
using namespace std;
int a;
int main ()

```

```

main.cpp X
1 #include <iostream>
2 using namespace std;
3 int a;
4 int main ()
5 {
6     int b, c;
7     b = 10;
8     c = 20;
9     a = b + c;
10    cout << a;
11    return 0;
12 }
13
09

```

```

{
int b, c;
b = 10;
c = 20;
a = b + c;
cout << a;
return 0;
}

```

මෙම කෝඩ් සටහනේ 3 වැනි පේළියේදී a යන නමින් වේරියබල් එකක් සකස් කර තිබේ. එම වේරියබල් එක සකස් කර තිබෙන්නේ main නම් ප්‍රධාන බලාක් එකට ඉහළින්. එම නිසා එම වේරියබල් එක වැඩසටහන පුරාම ග්ලෝබල් වේරියබල් එකක් වශයෙන් ක්‍රියාත්මක වේ.

මෙහි 6 වැනි පේළියේදී කලින් ආකාරයටම b සහ c යන නම්වලින් වේරියබල් දෙකක් සකස් කර තිබේ. මේවා main බලාක් එක තුළ ලියා ඇති නිසා ක්‍රියාත්මක වෙන්නේ ලෝකල් වේරියබල් ආකාරයටයි. මෙහි 7 සහ 8 යන පේළිවලදී එම වේරියබල්වලට පිළිවෙළින් 10 සහ 20 යන අගයන් ඇතුළු කර තිබේ.

මෙහි 9 වැනි පේළියේදී, b සහ c යන ලෝකල් වේරියබල් දෙක එකිනෙක හා එකතු කළ විට ලැබෙන අගය, a ග්ලෝබල් වේරියබල් එක තුළට යොමු කර තිබේ. ඒ අනුව main බලාක් එක තුළදී ඊට පිටතින් ඇති වේරියබල් එකක් භාවිතා කර ඇති ආකාරය පෙනෙයි. මෙහි 10 වැනි පේළියේදී අවුට්පුට් කරන්නේද ග්ලෝබල් වේරියබල් එකයි. මෙහි සකස් කර ඇති a නම් ග්ලෝබල් වේරියබල් එක වෙත, මෙම main බලාක් එක තුළදී ඇතුළු කළ අගය, වෙනත් බලාක් එකක් තුළදීද එක සේ භාවිතා කරන්නට පුළුවන.

#### කොන්ස්ටන්ට්ස් භාවිතා කිරීම

වේරියබල් එකක ඇති දත්ත වැඩසටහනේ විවිධ තැන්වල සිට වෙනස් කරන්නට පුළුවන් බව දන් දනිමු. එහෙත් කොන්ස්ටන්ට්ස් එකක දත්ත එසේ වෙනස් කළ නොහැක. එනම් එය නියතයකි. කොන්ස්ටන්ට්ස් එක සකස් කරන තැනදී ඊට දත්ත ඇතුළු කරනු ලබයි. ඉන්පසුව වැඩසටහනේ කිසිදු තැනකදී ඒවා වෙනස් කරන්නට හෝ අලුතින් දත්ත ඇතුළු කරන්නට හෝ නොහැකිය. එහෙත් ඒ තුළ ඇති දත්ත අවශ්‍ය ආකාරයට භාවිතා කරන්නට පුළුවන.

කොන්ස්ටන්ට්ස් භාවිතා කරන ආකාරය තේරුම් ගැනීම සඳහා පහත දක්වා කෝඩ් සටහන අධ්‍යයනය කරන්න. මෙම වැඩසටහන සකසා ඇත්තේ මුලින් ඇතුළු කරනු ලබන අගයන් දෙකක් ගුණ කළ විට ලැබෙන පිළිතුර අවුට්පුට් කිරීම සඳහාය. (12 වැනි පිටුවේ ඇති 10 රූපය)

```

#include <iostream>
using namespace std;

```



```

main.cpp x
1  #include <iostream>
2  using namespace std;
3  #define LENGTH 10
4  #define WIDTH 5
5  int main()
6  {
7      int area;
8      area = LENGTH * WIDTH;
9      cout << area;
10     return 0;
11 }
12

```

```

#define LENGTH 10
#define WIDTH 5
int main()
{
    int area;
    area = LENGTH * WIDTH;
    cout << area;
    return 0;
}

```

මෙහි 3 වැනි පේළියේදී Length යන නමින් කොන්ස්ටන්ට් එකක් ලබා ගෙන තිබේ. මෙහි Length යන නම වෙනුවට වෙනත් නමක් වුවද භාවිතා කරන්නට පුළුවන. එම කොන්ස්ටන්ට් එක සකසන විටදීම ඊට 10 යන අගය ඇතුළු කර තිබේ. එසේ ඕනෑම කොන්ස්ටන්ට් එකක් සකසන විට ඊට අදාළ අගයන් ඇතුළු කළ යුතුය.

```

main.cpp x
C:\Users\Parigamika\Documents\EasyGuide\bin\Debug\EasyGuide.exe
50
Process returned 0 (0x0)   execution time : 0.026 s
Press any key to continue.

```

මෙහි 4 වැනි පේළියේදී Width යන නමින් නවත් කොන්ස්ටන්ට් එකක් සකස් කර තිබේ. ඊට ඇතුළු කර තිබෙන්නේ 5 යන අගයයි. මේ ආකාරයෙන් කොන්ස්ටන්ට්වලට අගයන් ඇතුළු කිරීමෙන් පසුව එම අගයන් ඕනෑම තැනකදී භාවිතා කරන්නට පුළුවන. මෙම කොන්ස්ටන්ට් වී දෙක සකස් කර තිබෙන්නේ කිසිදු බලාත් එකකට අදාළ වන සේ නොවේ. එනිසා මෙම කොන්ස්ටන්ට් දෙක සම්පූර්ණ කේත සටහන තුළදී ක්‍රියාත්මක වෙන්නේ ග්ලෝබල් වේරියබල් එකක් ආකාරයටයි.

මෙහි 7 වැනි පේළියේදී area යන නමින් ලෝකල් වේරියබල් එකක් සකස් කර තිබේ. 8 වැනි පේළියේදී එම area වේරියබල් එක තුළට Length හා Width කොන්ස්ටන්ට් දෙක ගුණ කර ඊට අදාළ අගය ඇතුළු කර තිබේ. ඒ අනුව මෙහිදී කොන්ස්ටන්ට් දෙක තුළ ඇති දත්තද සාමාන්‍ය වේරියබල් තුළ ඇති දත්ත භාවිතා කරන්නාක් සේම භාවිතා කර තිබෙන බව පෙනේ. මෙම වැඩසටහන ක්‍රියාත්මක වෙන විට Length සහ Width කොන්ස්ටන්ට් දෙක තුළ ඇති අගය එකිනෙක හා ගුණ කර පිළිතුර අවුට්පුට් කරයි. (|| රූපය)

## තුන් වැනි පරිච්ඡේදය

### ඔපරේටර්ස් භාවිතා කරන ආකාරය

**සිසු** ම පරිගණක භාෂාවකදීම කාරක හෙවත් ඔපරේටර්ස් (operators) යොදා ගැනෙයි. මෙම ඔපරේටර්ස් යොදා ගැනෙන්නේ සංඛ්‍යා ගණනය කිරීම, සංසන්දනය කර බැලීම, තාර්කිකව කොන්දේසි පරීක්ෂා කර බැලීම වැනි කටයුතු සඳහාය. සී ජ්ලස් ජ්ලස් තුළදී භාවිතා වෙන මෙවැනි ඔපරේටර්ස් ගණනාවකි.

- ඇරිත්මටික් ඔපරේටර්ස් (arithmetic operators) - සාමාන්‍ය අංක ගණිතයේදී අවශ්‍ය වෙන විවිධ ගණනය කිරීම් සඳහා අවශ්‍ය සංකේත මේ යටතේ වෙයි. එම සංකේත වේරියබල්ස් ඇතිව හෝ වේරියබල්ස් නැතිව හෝ සංඛ්‍යා සමඟ භාවිතා කර ගණිත ගැටලු නිරාකරණය කර ගන්නට පුළුවන.

- රිලේෂනල් ඔපරේටර්ස් (relational operators) - මෙහිදී වේරියබල්ස් දෙකක් තුළ ඇති සංඛ්‍යා හෝ කෙළින්ම වෙනත් සංඛ්‍යා හෝ

සසඳා බලන්නට පුළුවන. එසේ සසඳා බලා ඒවා අතර යම් ගැළපීමක් වේද නොවේද යන්න අනුව පිළිතුරු ලබා දෙයි. සංඛ්‍යාවලට අමතරව අකුරු හෝ සංකේත හෝ වුව එකිනෙක හා සමානවේදියි පරීක්ෂා කර බැලීම සඳහා මෙම ඔපරේටර්ස් භාවිතා කළ හැක.

- එසයිත්මන්ට් ඔපරේටර්ස් (assignment operators) - වේරියබල් එකක් තුළට අගය පැවරීම සඳහා මෙම ඔපරේටර්ස් යොදා ගනී. මේ යටතේ ඇති සමාන ලකුණු සහිත (=) ඔපරේටර්ස් නිතරම පාහේ භාවිතා වෙයි. වේරියබල් එකක් තුළට යම් අගයන් යොමු කළ යුත්තේ නම් සමාන ලකුණු a = 10 යන අනුකරණයෙන් භාවිතා කරයි. වේරියබල් එකක් තුළ දැනට ඇති අගයට නවත් සංඛ්‍යා එකතු කිරීම, වේරියබල් එකෙහි ඇති අගයෙන් යම් සංඛ්‍යාවක් අඩු කිරීම යනාදී කටයුතු වෙනුවෙන්ද එසයිත්මන්ට් ඔපරේටර්ස් යොදා ගනී.

```
main.cpp X
C:\Users\Parigana\Documents\C\LazyGuide\bin\Debug\LazyGuide.exe

Line 1 - Value of c is :31
Line 2 - Value of c is :11
Line 3 - Value of c is :210
Line 4 - Value of c is :2
Line 5 - Value of c is :1
Line 6 - Value of a is :22
Line 7 - Value of b is :9

Process returned 0 (0x0)   execution time : 0.020 s
Press any key to continue.
```

01

● **ලොජිකල් ඔපරේටර්ස් (logical operators)** - මෙම ඔපරේටර්ස් භාවිතා කරන්නේ තාර්කික පිළිතුර ලබාදීම සඳහාය. එමෙන්ම මෙහිදී සංඛ්‍යා දෙකක් හෝ වේරියබල්ස් දෙකක හෝ සමාන අසමානතා පරීක්ෂා කරන්නේද තාර්කික ද්වාරයකට අදාළ ඔපරේටර් එකකිනි.

● **බිට්වයිස් ඔපරේටර්ස් (bitwise operators)** - ලබා දෙන සංඛ්‍යා ද්විමය සංඛ්‍යා බිට් හරවා ගණනය කිරීම් කරයි. එම ගණනය කිරීමෙන් පසුව ප්‍රතිඵලය ලබා දෙන්නේ ද්විමය සංඛ්‍යා නැවතත් දශම (decimal) සංඛ්‍යා බිට් හැරවීමෙන් පසුවය.

#### වැඩසටහන් තුළදී ඇරිත්මැටික් ඔපරේටර්ස් භාවිතා කිරීම

මෙම සෑම ඔපරේටර් එකක්ම ක්‍රියාත්මක වෙන ආකාරය එකිනෙකට වෙනස්ය. එනිසා යම් වැඩසටහනක් තුළදී මේවා භාවිතා කරනු පිණිස එක් එක් ඔපරේටර්ස්වල ක්‍රියාකාරීත්වය කෙසේද යන්න දැන සිටිය යුතුය. එසේ වැඩසටහන් තුළදී ඇරිත්මැටික් ඔපරේටර්ස් භාවිතා කරන ආකාරය මෙහි විස්තර කර ඇත.

ඇරිත්මැටික් ඔපරේටර්ස් යොදා ගන්නේ එකතු කිරීම, අඩු කිරීම, බෙදීම වැනි මූලික ගණිත කර්ම කර ගැනීම සඳහාය. මෙහිදී භාවිතා කළ හැකි එක් එක් ගණිත කර්ම සහ ඒවා කර ගැනීමට විධාන වශයෙන් භාවිතා කරන ලකුණු මෙසේය.

● **(+)** - එකතු කිරීමට (addition) භාවිතා කරයි. මෙහිදී සංඛ්‍යා එකතු කර ගැනීමට පුළුවන. එසේම වේරියබල්ස් තුළ ඇති අගයන් වුව වේරියබල්ස්වල නම් භාවිතා කර එකතු කර ගන්නට පුළුවන.

● **(-)** - අඩු කිරීමට (subtraction) භාවිතා කරයි.

● **(\*)** - ගුණ කිරීමට (multiplication) භාවිතා කරයි.

● **(/)** - බෙදීමට (division) භාවිතා කරයි.

● **(%)** - එක් සංඛ්‍යාවක් තවත් සංඛ්‍යාවකින් බෙදූ පසු ශේෂය හෙවත් මාපාංකය (modulus) පෙන්වයි.

● **(++)** - වේරියබල් එකක ඇති සංඛ්‍යාවකට එකක් එකතු කිරීමට යොදා ගනී. යම් කිසි සංඛ්‍යාවක බලය වශයෙන් ඇති සංඛ්‍යාව ලිවීමට මෙම සංකේතය (increment operator) භාවිතා කරයි.

● **(//)** - බෙදීම සඳහා පසු ඇල ඉරි හෙවත් බැක් ස්ලැෂ් ලකුණු දෙකක් භාවිතා කළහොත්, බෙදීමෙන් අනතුරුව ලැබෙන පිළිතුරෙහි දශම සංඛ්‍යා වෙනොත් එය නොසලකා හැර පිළිතුර පෙන්වයි. එනම් මෙ හිදී නිඛිල බෙදීම (floor division) කරයි.

මෙවැනි ඇරිත්මැටික් ඔපරේටර්ස් භාවිතා කරන ආකාරය තේරුම් ගැනීම සඳහා පහත දැක්වෙන කේත සටහන විමසා බලන්න. මෙම ආදර්ශ කේත සටහන සකස් කොට ඇත්තේ ඇරිත්මැටික් ඔපරේටර්ස් සියල්ල යොදා ගනිමිනි.

```
int a = 21;
int b = 10;
```

```
main.cpp X
C:\Users\Parigana\Documents\C\LazyGuide\bin\Debug\LazyGuide.exe

Line 1 - Value of c is :0
Line 2 - Value of c is :1
Line 3 - Value of c is :0
Line 4 - Value of c is :1
Line 5 - Value of c is :0
Line 6 - Value of c is :1

Process returned 0 (0x0)   execution time : 0.067 s
Press any key to continue.
```

02

```
int c = 0;
c = a + b;
cout << "Line 1 - Value of c is :" << c << endl;
c = a - b;
cout << "Line 2 - Value of c is :" << c << endl;
c = a * b;
cout << "Line 3 - Value of c is :" << c << endl;
c = a / b;
cout << "Line 4 - Value of c is :" << c << endl;
c = a % b;
cout << "Line 5 - Value of c is :" << c << endl;
a++;
cout << "Line 6 - Value of a is :" << a << endl;
b--;
cout << "Line 7 - Value of b is :" << b << endl;
return 0;
```

● මෙහි පළමු පේළි තුනේදී a, b සහ c යන නම්වලින් වේරියබල්ස් තුනක් සකස් කොට ඇත. එම වේරියබල්ස් තුනට පිළිවෙළින් 21, 10 සහ 0 යන අගය ඇතුළු කොට තිබේ. ඉන්පසුව 4 වැනි පේළියේ සිට එම වේරියබල්ස් තුළ ඇති සංඛ්‍යා විවිධ ගණනය කිරීම්වලට ලක් කර, ඒවායේ අවුට්පුට් ලබා ගෙන තිබේ. ඒ අනුව 4 වැනි පේළිය ලියා තිබෙන්නේ එම වේරියබල්ස් අතරින් a සහ b වේරියබල්ස් දෙක තුළ ඇති අගයන් එකතු කිරීමටයි. එහිදී එම අගය C වේරියබල්ස් තුළට යොමු වෙයි.

● එසේ එම C වේරියබල්ස් තුළට යොමු කළ අගය මෙහි 5 වැනි පේළිය මඟින් අවුට්පුට් කරනු ලබයි. එහි ඇති Line 1 - Value of c is යන්න දක්වා තිබෙන්නේ ඒ ආකාරයෙන්ම තිරය මත අවුට්පුට් කිරීමටයි. ඒ අනුව, මෙම පේළිය ක්‍රියාත්මක වෙන විට තිරයේ Line 1 - Value of c is යන්න ප්‍රින්ට් වී ඉදිරියෙන් C වේරියබල් එකෙහි දූතට ඇති අගය ලෙස 31 යන්න ප්‍රින්ට් වෙයි.

● මෙම කේත සටහනේ 6 වැනි පේළිය තුළදී a වේරියබල් එකෙහි දූතට ඇති අගයෙන් b වේරියබල් එකෙහි ඇති අගය අඩු කරනු ලබයි. එය අවුට්පුට් කර ගන්නේ 7 වැනි පේළියෙනි. මේ ආකාරයෙන් 8, 10 සහ 12 යන පේළිද ක්‍රියාත්මක වෙයි. එහිදී පිළිවෙළින් a සහ b වේරියබල්ස් වැඩි කිරීම, a වේරියබල්ස් ඇති අගය b වේරියබල් එකෙහි ඇති අගයෙන් බෙදීම සහ a වේරියබල් එකෙහි ඇති අගයෙන් b වේරියබල්ස් බෙදා ඉතිරිය පෙන්වීමද කරයි.

● මෙහි 14 වැනි පේළියේ ඇති කේතය මඟින් a වේරියබල් එකෙහි දූතට ඇති අගයට 1 ක් එකතු කරනු ලබයි. ඉන්පසුව 15 වැනි පේළිය මඟින් එම a වේරියබල්ස් අවුට්පුට් කරයි. එවිට අවුට්පුට් සේ 22 යන්න ලැබේ. මේ ආකාරයටම 16 වැනි පේළිය තුළදී b වේරියබල්



එකෙන් 2 ක් අඩු කර 17 වැනි පේළියේදී එම b වේරියබල්ස් අඩුවුවා කරයි. (01 රූපය)

### ප්‍රමුඛතා අනුපිළිවෙළ

| ප්‍රමුඛතා අනුපිළිවෙළ | ඔපරේටර් වර්ගය |
|----------------------|---------------|
| 1                    | ( )           |
| 2                    | ++, --        |
| 3                    | *, /, %       |
| 4                    | +, -          |

කර්ම එකම සමීකරණයක් තුළ යොදන්නේ නම් එවිට එම ගණිත කර්ම කිරීමට විධාන දිය යුත්තේ ප්‍රමුඛතා අනුපිළිවෙළ හෙවත් ඔපරේටර් ප්‍රසිඩන්ස් (operator precedence) අනුව යමිනි. ගණිත කර්මවල ප්‍රමුඛතා අනුපිළිවෙළ පහත දක්වා ඇති වගුවෙන් පෙන්වන පරිදි වෙයි.

මේ අනුව වැඩිම බලයක් ලැබෙන්නේ වරහන්වලටය. මුලින්ම එකතු කිරීම හෝ අඩු කිරීම ක්‍රියාත්මක විය යුත්තේ නම් එම කොටස වරහන් තුළ යෙදිය යුතුවීම නිදසුනකි. ආදර්ශනයක් ලෙස cout << (2 + 4) \* 3 / 5 - 10; යන ගණිත ගැටලුව සලකන්න. එහි වරහන් තුළ ඇති කොටස මුලින් ක්‍රියාත්මක කළ යුතු බැවින් මුලින්ම කරන්නේ 2 සහ 4 එකතු කිරීමයි. ඉන්පසුව එය 3 න් වැඩි කර 5 න් බෙදයි. සියල්ල අවසානයේදී ලැබෙන පිළිතුරෙන් 10 ක් අඩු කරනු ලැබේ. (02 රූපය) අවසාන පිළිතුර ලැබෙන්නේ එවිටය.

**වැඩසටහනකදී විලේනාල් ඔපරේටර්ස් භාවිතා කරන අයුරු**  
වේරියබල්ස් දෙකක් තුළ ඇති සංඛ්‍යා හෝ ඊට පිටින් ඇති සාමාන්‍ය සංඛ්‍යා හෝ සසඳා බලනු පිණිස විලේනාල් ඔපරේටර්ස් යොදා ගැනේ. මෙසේ සසඳා බලා ලබා දෙන විධානයට ගැළපෙන්නේද නැද්ද යන්න අනුව පිළිතුරු ලබාදීමට මෙම ඔපරේටර්ස් භාවිතා කරයි. මෙහිදී භාවිතා කරන සංකේත මෙසේය.

● **සමාන ලකුණු දෙක (==)** - සමාන ලකුණු දෙකක් එක ලග යොදන්නේ එක් අගයක් තවත් අගයකට සමානදැයි පරීක්ෂා කිරීම සඳහාය. මෙහිදී සමාන ලකුණු දෙකට කලින් ඇති අගය සමාන ලකුණු දෙකට පසුව ඇති අගයට සමාන නම් පිළිතුර සත්‍ය ලෙසද, නැත්නම් පිළිතුර අසත්‍ය ලෙසද ප්‍රකාශ කරනු ලබයි.

● **අසමාන බව ප්‍රකාශ කිරීම (!=)** - මුලින් ඇති අගය හා පසුව ඇති අගය සසඳන විට එම අගය දෙක අසමාන බව ප්‍රකාශ කිරීම සඳහා මෙම සංකේතය භාවිතා කරයි. ඒ අනුව මෙහිදී අගය දෙක අසමාන නම් පිළිතුර සත්‍ය ලෙසද, අගය දෙක සමාන නම් පිළිතුර අසත්‍ය ලෙසද පිළිතුර ලබා දෙයි.

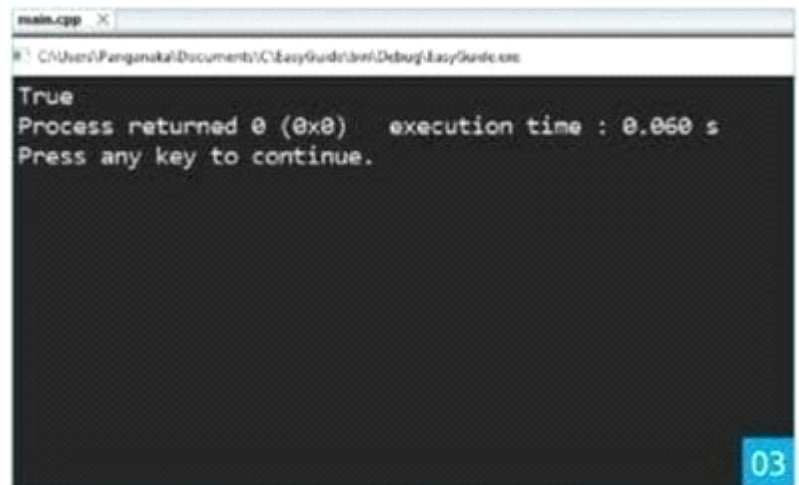
● **විශාල බව ප්‍රකාශ කිරීම (>)** - මුලින් ඇති අගයට වඩා පසුව ඇති අගය විශාල බව ප්‍රකාශ කිරීම සඳහා මෙම සංකේතය භාවිතා කරයි. ඒ අනුව මෙහිදී 10 > 10 ලෙස දැක්වුවද, පිළිතුර ලැබෙන්නේ අසත්‍ය ලෙසය. වම් පස ඇති සංඛ්‍යාවට වඩා දකුණු පස ඇති සංඛ්‍යාව විශාල නොවීම ඊට හේතුවයි.

● **විශාල හෝ සමාන හෝ බව ප්‍රකාශ කිරීම (>=)** - මුලින් ඇති අගයට වඩා පසුව ඇති අගය විශාල හෝ සමාන හෝ බව ප්‍රකාශ කිරීම සඳහා මෙම සංකේතය භාවිතා කරයි. ඒ අනුව මෙහිදී 10 >= 10 යන්න ලබා දුන් විට පිළිතුර ලැබෙන්නේ සත්‍ය ලෙසය.

● **කුඩා හෝ සමාන හෝ බව ප්‍රකාශ කිරීම (<=)** - මුලින් ඇති අගයට වඩා පසුව ඇති අගය කුඩා හෝ සමාන හෝ බව ප්‍රකාශ කිරීම සඳහා මෙම සංකේතය භාවිතා කරයි.

මෙම විලේනාල් ඔපරේටර්ස් භාවිතා කරන ආකාරය අවබෝධ කර ගැනීම පිණිස උපකාරී වෙන කෝඩ් පහත දැක්වෙයි.

```
int a = 21;
int b = 10;
int c = 0;
c = a == b;
cout << "Line 1 - Value of c is : " << c << endl ;
```



```
c = a > b;
cout << "Line 2 - Value of c is : " << c << endl ;
c = a < b;
cout << "Line 3 - Value of c is : " << c << endl ;
c = a >= b;
cout << "Line 4 - Value of c is : " << c << endl ;
c = a <= b;
cout << "Line 5 - Value of c is : " << c << endl ;
c = a != b;
cout << "Line 6 - Value of c is : " << c << endl ;
```

● මෙහි පළමු පේළි තුනේදී a, b සහ c යන නම්වලින් int වර්ගයේ වේරියබල් තුනක් සකස් කොට තිබේ. එම වේරියබල් තුනට පිළිවෙලින් 21, 10 සහ 0 යන අගය ඇතුළු කොට තිබේ. ඉන්පසුව 4 වැනි පේළියේ සිට එම වේරියබල් තුළ ඇති සංඛ්‍යා විවිධ සැසඳීම් වලට ලක් කර, ඒවායේ අඩුවුවා ලබා ගෙන තිබේ. ඒ අනුව 4 වැනි පේළිය ලියා තිබෙන්නේ a සහ b වේරියබල්වල ඇති අගය සමාන දැයි පරීක්ෂා කර බැලීම සඳහාය. එහිදී ඒවායේ ඇති සංඛ්‍යා අසමාන බැවින් c නම් වේරියබල් එක තුළ ගබඩා වෙන්නේ 0 යන්නයි. එය 5 වැනි පේළිය මගින් අඩුවුවා කරයි.

● එම 5 වැනි පේළියේදී Line 1 - Value of c is , යන්න යොදා ඇත්තේ පිළිතුරට කලින් තිරයේ පෙන්වීම සඳහාය. මෙහි පිළිතුර පෙන්වන්නේ << ලකුණෙන් පසුව ඇති c යන්න මගින්ය. මෙහි 6 වැනි පේළියේදී a > b ලෙස ප්‍රකාශ කොට තිබේ. එයින් කියැවෙන්නේ b වේරියබල් එකෙහි ඇති අගයට වඩා a වේරියබල් එකෙහි ඇති අගය විශාල බවය. මෙහි ලබාදී ඇති කොන්දේසිය සත්‍ය බැවින් 7 වැනි පේළියේදී පිළිතුර ලෙස 1 යන්න අඩුවුවා කරයි.

● මෙහි 8 වැනි පේළියේදී ප්‍රකාශය ලබාදී තිබෙන්නේ a < b ලෙසයි. එයින් කියැවෙන්නේ b වේරියබල් එකෙහි ඇති අගය a වේරියබල් එකෙහි ඇති අගයට වඩා කුඩාය යන්නයි. මෙහිදී a වේරියබල් එකෙහි ඇති අගය විශාල බැවින් 9 වැනි පේළියේදී පිළිතුර ලෙස ලැබෙන්නේ 0 යන්නයි.

● මේ ආකාරයෙන්ම 10 වැනි පේළියේදී a >= b යන්න මගින් b වේරියබල් එකෙහි ඇති අගයට වඩා a වේරියබල් එකෙහි ඇති අගය විශාල හෝ සමාන හෝ දැයි ප්‍රකාශ කිරීමද, 12 වැනි පේළියේදී a <= b යන්න මගින් b වේරියබල් එකෙහි ඇති අගයට වඩා a වේරියබල් එකෙහි ඇති අගය කුඩා හෝ සමාන හෝ දැයි ප්‍රකාශ කිරීමද කරනු ලබයි.

● මෙහි ඇති 14 වැනි පේළියේදී සලකා බැලෙන්නේ a වේරියබල් එකෙහි ඇති අගයට b වේරියබල් එකෙහි ඇති අගය අසමානද යන්නයි. ඒ අනුව එය සත්‍ය බැවින් 1 යන්න අඩුවුවා කරයි. (02 රූපය)

● මෙහිදී ඔපරේටර් එකෙහි ක්‍රියාකාරිත්වය අනුව පිළිතුර සත්‍ය වේ නම් ඒ බව දැක්වීමට 1 ද, පිළිතුර අසත්‍ය නම් ඒ බව දැක්වීමට 0 ද



```

main.cpp X
C:\Users\Pangana\Documents\EasyGuide\src\Debug\EasyGuide.exe

Line 1 - Value of c is 31
Line 2 - Value of c is 52
Line 3 - Value of c is 1092
Line 4 - Value of c is 52
Line 5 - Value of c is 31
Line 6 - Value of c is 1

Process returned 0 (0x0)   execution time : 0.044 s
Press any key to continue.

```

04

අවුට්පුට් සේ ලබා දෙයි. මෙසේ 1 සහ 0 වෙනුවට true සහ false ලෙස අවුට්පුට් ලබා දිය යුත්තේ නම් ඒ සඳහා පහත දක්වා ඇති ආකාරයට කෝඩ් ලිවිය යුතුය.

```

int a = 21;
int b = 10;
a > b ? cout << "True" : cout << "False";

```

● මෙහි යොදා ඇති ප්‍රශ්නාර්ථ ලකුණින් (?) එසේ වුවහොත් යන්නද, දෙතින් ලකුණින් (:) එසේ නොවුවහොත් යන්නද දක්වා තිබේ. ඒ අනුව  $a > b$  යන ප්‍රකාශය සත්‍ය නම් ප්‍රශ්නාර්ථ ලකුණින් පසුව ඇති කේත කොටස ක්‍රියාත්මකවී True යන්න අවුට්පුට් සේ ලබා දෙයි. ලබාදී ඇති ප්‍රකාශය අසත්‍ය නම් දෙතින් ලකුණින් පසුව ඇති කේත කොටස ක්‍රියාත්මකවී False යන්න අවුට්පුට් සේ ලබා දෙයි. මේ අනුව මෙහිදී a වේරියබල් එකෙහි ඇති අගය b වේරියබල් එකෙහි ඇති අගයට වඩා වැඩි බැවින්, ලබාදී ඇති ප්‍රකාශනය සත්‍ය නිසා, True යන්න අවුට්පුට් සේ ලබා දෙයි. (03 රූපය)

**වැඩසටහනකදී එසයින්මන්ට් ඔපරේටර්ස් භාවිතා කරන අයුරු**

වේරියබල් එකකට අගය පැවරීම සඳහා යොදා ගන්නේ මෙම එසයින්මන්ට් ඔපරේටර්ස්ය. මේ යටතේ ඇති සමාන ලකුණු සහිත "=" කාරකය හෙවත් ඔපරේටර් එක නිතරම පාහේ භාවිතා වෙන්නකි.

ඒ අනුව වේරියබල් එකක් තුළට යම් අගයක් යොමු කරන්නේ නම් සමාන ලකුණ  $a = 10$  ලෙස භාවිතා කරන බව අප දනිමු. ඊට අමතරව වේරියබල් එකක් තුළ දැනට ඇති අගයට තවත් සංඛ්‍යා එකතු කිරීම, වේරියබල් එකෙහි ඇති අගයෙන් යම් සංඛ්‍යාවක් අඩු කිරීම යනාදී විවිධ ගණිත කර්ම කරන අවස්ථාවලදී එසයින්මන්ට් ඔපරේටර්ස් භාවිතා කරයි. ඒ මෙසේය.

● **සමාන ලකුණ භාවිතා කිරීම (=)** - වේරියබල් එකක් තුළට යම් දත්තයක් යොමු කිරීමට භාවිතා කරයි. එවිට එම වේරියබල්ස් තුළ දැනට යම් දත්තයක් තිබේ නම් එය ඉවත්වී අලුත් දත්තය වේරියබල්ස් තුළ තැන්පත් වේ.

● **එකතු කිරීම සමග සමාන ලකුණ භාවිතා කිරීම (+=)** - වේරියබල් එකක දැනට ඇති අගයට තවත් අගයක් එකතු වෙයි. a නම් වේරියබල් එකක දැනට පවතින අගයට 10ක් එකතු කරන්නට අවශ්‍ය වුවේය කියා සිතන්න. එවිට සාමාන්‍ය ආකාරයට කේත ලියන්නේ  $a = a + 10$  ලෙසය. එහෙත් මෙම එසයින්මන්ට් ඔපරේටර් එක භාවිතා කර  $a += 10$  ලෙස කේත ලියා එය කළ හැක. අඩුවෙන් කේත ලියා විධාන දිය හැකිවීම මෙහි ඇති වාසියයි.

● **සාණ ලකුණ සමග සමාන ලකුණ භාවිතා කිරීම (-=)** - වේරියබල් එකක දැනට ඇති අගයකින් යම් අගයක් අඩු කරන්නට පුළුවන. මෙම ක්‍රියාකාරකම කර ගන්නට අවශ්‍ය නම් සාමාන්‍යයෙන්  $a = a - 2$  ලෙස කේත ලිවිය යුතුය. එහෙත් එහෙත් මෙම එසයින්මන්ට් ඔපරේටර් එක භාවිතා කර  $a -= 2$  ලෙස කෙටි කේතයක් ලියා එම ක්‍රියාකාරකම කර ගත හැක.

● **ගුණ කිරීමේ ලකුණ සමග සමාන ලකුණ භාවිතා කිරීම (\*=)** - වේරියබල් එකක දැනට ඇති අගය යම් අගයකින් ගුණ කර ගන්නට පුළුවන.

● **බෙදීමේ ලකුණ සමග සමාන ලකුණ භාවිතා කිරීම (/=)** - වේරියබල් එකක දැනට ඇති අගය යම් අගයකින් බෙදා ගන්නට පුළුවන.

● **ප්‍රතිගත ලකුණ සමග සමාන ලකුණ භාවිතා කිරීම (%=)** - වේරියබල් එකක දැනට ඇති අගය යම් අගයකින් බෙදා ශේෂය පමණක් පෙන්වයි.

මෙම එසයින්මන්ට් ඔපරේටර් සියල්ල භාවිතා කරන ආකාරය හුරුවීම සඳහා සකසා ඇති පහත දැක්වෙන කෝඩ් අධ්‍යයනය කොට ඒවා ඉගෙන ගැනීමට උත්සාහ ගන්න.

```

int a = 21;
int b = 10;
int c = 0;
c = a + b;
cout << "Line 1 - Value of c is " << c << endl ;
c += a;
cout << "Line 2 - Value of c is " << c << endl ;
c *= a;
cout << "Line 3 - Value of c is " << c << endl ;
c /= a;
cout << "Line 4 - Value of c is " << c << endl ;
c -= a;
cout << "Line 5 - Value of c is " << c << endl ;
c = c%3;
cout << "Line 6 - Value of c is " << c << endl ;

```

● මෙම කේත සටහනේ පළමු පේළි තුනේදී a, b සහ c යන නම් වලින් වේරියබල්ස් තුනක් සකස් කොට ඇත. එම වේරියබල්ස් තුනට පිළිවෙලින් 21, 10 සහ 0 යන අගය ඇතුළු කොට තිබේ. ඉන් පසුව 4 වැනි පේළියේ දී a සහ b යන වේරියබල්ස් දෙක එකිනෙක හා එකතු කළ විට ලැබෙන පිළිතුර c වේරියබල්ස් ට යොමු කරනු ලබයි. මෙහිදී 21 සහ 10 යන අගයන් එකතුවී 31 යන සංඛ්‍යාව c වේරියබල්ස් වෙත යොමු වේ. මෙහිදී එසයින්මන්ට් ඔපරේටර් එකක් සේ භාවිතා කර ඇත්තේ සමාන ලකුණ පමණි.

● ඉන්පසුව 5 වැනි පේළියේදී ජ වේරියබල් එකෙහි අගය අවුට්පුට් කරනු ලැබේ. මෙහි ඇති cout යනු සාමාන්‍යයෙන් C++ භාෂාවේ අවුට්පුට් සඳහා යොදා ගන්නා කීවර්ඩ් (keyword) එකයි. එහි ඉදිරියේ ඇති Line 1 - Value of c is යන්න ඒ ආකාරයෙන්ම අවුට්පුට් කෙරේ. එහි << යන්නෙන් පසුව ඇති c අකුර යනු ඉහත සංඛ්‍යාවල එකතුව ඇතුළු කළ වේරියබල් එකයි. මේ අනුව මෙහිදී එම එකතුව අවුට්පුට් කරනු ලැබේ. මේ පේළියේම ඇති endl යන්න මගින් අවුට්පුට් එකෙහි ඇති එම පේළිය අවසන් කරයි.

● මෙහි ඇති 6 වැනි පේළියේදී + ලකුණ සමග = ලකුණ භාවිතා කොට තිබේ. මේ නිසා මෙහිදී දැනට c වේරියබල් එකෙහි ඇති අගයට a වේරියබල් එකෙහි ඇති අගය එකතු කරයි. ඒ අනුව දැන් c වේරියබල් එකෙහි තිබූ 31 යන අගයට a වේරියබල් එකෙහි ඇති 21 යන අගය එකතුවී පිළිතුර ලෙස 52 යන්න ලැබේ.

● ඉන්පසුව එය 7 වැනි පේළිය මගින් ඉහත විස්තර කළ ආකාරයටම අවුට්පුට් කරනු ලැබේ. මේ ආකාරයටම 8 වැනි පේළියේදී ගුණ කිරීමේ ලකුණ සමග සමාන ලකුණ භාවිතා කිරීමේ එසයින්මන්ට් ඔපරේටර් එක භාවිතා කොට තිබේ. ඒ අනුව c වේරියබල් එකෙහි දැනට ඇති අගය වෙත 52 යන සංඛ්‍යාව a වේරියබල් එකෙහි ඇති අගය වෙත 21 යන්නෙන් ගුණ වේ. මෙවිට පිළිතුර ලෙස 1092 යන්න ලැබෙයි. මේ ආකාරයෙන් අනෙක් පේළිවල ගණනය කිරීම්ද කෙරෙයි. ඒවායේ ප්‍රතිඵල විමසා බලා සම්පූර්ණවී ඇති ගණනය කිරීම්



මොනවාදැයි යන්න අවබෝධ කර ගන්නට උත්සාහ කරන්න. (04 රූපය)

**වැඩසටහනකදී ලොජිකල් ඔපරේටර්ස් භාවිතා කරන අයුරු**

මේවා භාවිතා වෙන්නේ තාර්කික පිළිතුරු ලබාදීම සඳහාය. එමෙන්ම මෙහිදී සංඛ්‍යා දෙකක හෝ වේරියබල් දෙකක හෝ සමාන අසමානතාව පරීක්ෂා කරන්නේද ලොජිකල් ඔපරේටර්ස් මගිනි. එවැනි අවස්ථා කීපයක් මෙසේය.

• **&&** - දෙපස ඇති ප්‍රකාශ දෙකක් and යන විධානයෙන් (එනම්, සහ යන අර්ථයෙන්) පරීක්ෂා කර බලන්නට මෙය යොදා ගැනෙයි. එහිදී මෙහි පිළිතුර True ලෙස අවුට්පුට් වෙන්නේ දෙපස ඇති ප්‍රකාශ දෙකම සත්‍ය වීම්දී පමණි.

• **||** - දෙපස ඇති ප්‍රකාශ දෙකක් දර යන විධානයෙන් (එනම්, හෝ යන අර්ථයෙන්) පරීක්ෂා කර බලන්නට මෙය යොදා ගැනෙයි. එහිදී දෙපස ඇති ප්‍රකාශ දෙකෙන් එකක් හෝ සත්‍ය නම් මෙහි පිළිතුර ලෙස True යන්න ලැබේ.

• **!** - නොවේ (not) යන්න ප්‍රකාශ කිරීමට මෙය යොදා ගැනෙයි. ඒ අනුව ඉදිරියෙන් ඇති එක් ප්‍රකාශයක් සත්‍ය නම්, පිළිතුර ලෙස False යන්නත්, ඉදිරියෙන් ඇති ප්‍රකාශය අසත්‍ය නම් True යන්නත් අවුට්පුට් සේ ලබා දීමට මෙය යොදා ගැනෙයි.

මෙම ලොජිකල් ඔපරේටර්ස් භාවිතා කරන ආකාරය අවබෝධ කර ගැනීම පිණිස පහත දක්වා ඇති කෝඩ් විමසා බලන්න. (01 රූපය)

```
cout << (true && true) << endl ;
cout << (false && true) << endl ;
cout << (1 == 1 && 2 == 1) << endl ;
cout << (1 == 1 || 2 != 1) << endl ;
cout << (true && 1 == 1) << endl ;
cout << (false && 0 != 0) << endl ;
cout << (true || 1 == 1) << endl ;
cout << (1 != 0 && 2 == 1) << endl ;
cout << (! true) << endl ;
cout << (! false) << endl ;
cout << (! (true && false)) << endl ;
cout << (! (1 == 1 && 0 != 1)) << endl ;
cout << (! (10 == 1 || 1000 == 1000)) << endl ;
cout << (! (1 != 10 || 3 == 4)) << endl ;
cout << (1 == 1 && (! (0 == 1 || 1 == 1))) << endl ;
```

• මෙහි පළමු පේළිය තුළදී && ඔපරේටර් එක භාවිතා කොට තිබේ. එහි දෙපසින්ම ඇති ප්‍රකාශ දක්වා ඇත්තේ True ලෙසය. ඒ අනුව එහිදී ලබාදී ඇති ප්‍රකාශ දෙකම සත්‍ය බැවින් මෙය ක්‍රියාත්මක වෙත විට ප්‍රතිඵලය ලෙස ලැබෙන්නේද 1 යන්නයි.

• මෙහි දෙවැනි පේළියේදී දී ඇති ප්‍රකාශ දෙකෙන් සත්‍ය වෙන්නේ එකක් පමණි. මේ නිසා && ඔපරේටර් එක හරහා ලබා දෙන අවුට්පුට් එක වෙන්නේ 0 යන්නයි.

• මෙහි 3 වැනි පේළිය තුළ තරමක් වෙනස්ය. එහිදී && ඔපරේටර් එකට වම් පසින් 1 යන්න, 1 ට සමාන බව දක්වා ඇත. ඒ අනුව එය සත්‍ය ප්‍රකාශනයකි. එහෙත් && ඔපරේටර් එකට දකුණු පසින් ඇති ප්‍රකාශනයේ තිබෙන්නේ 2 යන්නට 1 යන්න සමාන බවයි. එය අසත්‍ය ප්‍රකාශයක් බැවින් && ඔපරේටර් එක ක්‍රියාත්මක වී අවුට්පුට් එක ලෙස 0 යන්න ලබා දෙයි.

• මෙහි 4 වැනි පේළියේදී && ඔපරේටර් එක වෙනුවට භාවිතා කර ඇත්තේ || ඔපරේටර් එකයි. එහිදී වම් පස සහ දකුණු පස ඇති ප්‍රකාශ දෙකම සත්‍ය වේ. එම නිසා පිළිතුර ලෙස 1 යන්න ලැබේ. එමෙන්ම මෙහිදී එක් ප්‍රකාශයක් පමණක් සත්‍ය වුවද || ඔපරේටර් එක භාවිතා

කොට ඇති නිසා පිළිතුර ලෙස 1 යන්න ලැබෙයි.

• මෙහි ඇති 5 වැනි පේළියේදී && ඔපරේටර් එකට දෙපසින් ඇති ප්‍රකාශ දෙක තරමක් වෙනස් ආකාරයකින් යොදා ඇති ආකාරය පෙනේ. එහිදී වම් පස ප්‍රකාශයට True ලෙස යොදා ඇති නිසා එම අවස්ථාව සත්‍ය අවස්ථාවක් ලෙස සැලකේ. දකුණු පස ඇති ප්‍රකාශයේ දක්වා ඇත්තේ 1 යන්න 1 ට සමාන බවය. එයද සත්‍ය බැවින් පිළිතුර ලෙස 1 යන්න ලබා දේ. මීට සමාන ආකාරයකින් 6, 7 සහ 8 යන පේළිවල කේතද ලියා තිබේ.

• මෙහි 9 වැනි පේළියේදී ! ඔපරේටර් එක භාවිතා කොට ඇත. එහිදී ! True යන්න දක්වා ඇති නිසා පිළිතුර ලෙස 0 යන්න ලැබේ. 10 වැනි පේළියේ කේත ලියා ඇත්තේ මීට විරුද්ධ ආකාරයටය. එහිදී ! False ලෙස කේතය ලියා ඇති නිසා අවුට්පුට් එක ලෙස 1 යන්න ලැබෙයි. මෙම අවස්ථා දෙකේදීම ! ඔපරේටර් එක භාවිතා කරන ලද්දේ තනි ප්‍රකාශයක් සඳහාය. එහෙත් 11 වැනි පේළියේදී තවත් ප්‍රකාශයක් සඳහා ! ඔපරේටර් එක භාවිතා කර ඇත.

• සාමාන්‍යයෙන් ඉදිරියෙන් ඇති එක් ප්‍රකාශයක් සඳහා පමණක් ක්‍රියාත්මක වීම ! ඔපරේටර් එකෙහි ස්වභාවයයි. එහෙත් ! ඔපරේටර් එක සමඟ && හෝ || ඔපරේටර් එක භාවිතා කොට ප්‍රකාශ කීපයක්ම වුවද ලබා දෙන්නට පුළුවන. ඒ අනුව මෙහි වරහන් තුළ ඇති True && False යන්න මගින් && ඔපරේටර් එක ක්‍රියාත්මක වී දෙපස ඇති ප්‍රකාශවලට අදාළ පිළිතුර ලෙස 0 යන්න ලබාදේ. එහෙත් මුලට ! ඔපරේටර් එක යොදා ඇති නිසා මෙහිදී පිළිතුර ලෙස 1 යන්න ලැබෙයි.

• මෙහි ඇති 12 වැනි පේළියේ කේත ක්‍රියාත්මක වෙන්නේද මීට තරමක් සමාන ආකාරයටය. මෙහි 13 වැනි පේළියේදී එලෙසම ක්‍රියාත්මක වුවද එහිදී ! ඔපරේටර් එක සමඟ භාවිතා කොට තිබෙන්නේ || ඔපරේටර් එකයි. ඒ ආකාරයෙන්ම || ඔපරේටර් එක භාවිතා කොට 14 වැනි පේළියේ කේත ලියා තිබේ.

• ඉහත කේත සටහනේ අවසන් පේළිය තරමක් සංකීර්ණය. එහිදී &&, || සහ ! යන ඔපරේටර් තුනම භාවිතා කර තිබේ. එහිදී මුලින් ඇති && ඔපරේටර් එකට වම් පසින් 1 == 1 යන ප්‍රකාශය දක්වා ඇත. එම && ඔපරේටර් එකට දකුණු පසින් ඇති කොටස තිබෙන්නේ වරහන් තුළය. එය !(1 == 1 or 1 == 0) යන ආකාරයෙන් තිබේ. ඒ අනුව මෙහිදී ! ඔපරේටර් එක සමඟ || ඔපරේටර් එක භාවිතා කොට තිබේ.

• මෙහි || ඔපරේටර් එක භාවිතා කර ඇති කොටස වෙත 0 == 1 || 1 == 0 යන්නෙහි ලබාදී ඇති එක් ප්‍රකාශයක් සත්‍ය බැවින්, දර ඔපරේටර් එක ක්‍රියාත්මක වී පිළිතුර ලෙස 1 යන්න ලබාදිය යුතු වුවත්, මෙය not ඔපරේටර් එකට අදාළව ක්‍රියාත්මක වෙත නිසා පිළිතුර ලෙස 0 යන්න අවුට්පුට් කරයි. මේ අනුව මෙම කේත පේළියට අදාළ ව පිළිතුර ලැබෙන්නේ 0 ලෙසය. (05 රූපය)

**වැඩසටහනකදී බිට්වයිස් ඔපරේටර්ස් භාවිතා කරන අයුරු**

බිට් මත ක්‍රියාත්මක වෙත කාරක හෙවත් බිට්වයිස් ඔපරේටර්ස් (bit-



```

main.cpp X
Select C:\Users\Panganaka\Documents\C++\EasyGuide\bin\Debug\EasyGuide.exe

Line 1 - Value of c is : 12
Line 2 - Value of c is : 61
Line 3 - Value of c is : 49
Line 4 - Value of c is : -61
Line 5 - Value of c is : 120
Line 6 - Value of c is : 30

Process returned 0 (0x0)   execution time : 0.038 s
Press any key to continue.
06

```

wise operators) ක්‍රියාත්මක වෙන විට, අප විසින් දෙනු ලබන සංඛ්‍යා මූලිකම ද්වීමය හෙවත් බයිනරි සංඛ්‍යා බවට හරවා ගනු ලබයි. ගණනයවීම වෙන්වනේ ඉන් පසුවය. මෙම ගණනය කිරීම බයිනරි ආකාරයට කෙරුණත් ප්‍රතිඵලය ලබා දෙන්නේ පිළිතුර නැවතත් දශම හෙවත් ඩෙසිමල් (decimal) සංඛ්‍යා බවට හැරවීමෙන් පසුවය. බිට් වයිස් ඔපරේටර්ස් ක්‍රියාත්මක කරවීම පිණිස සි ජලස් ජලස් තුළදී විශේෂ සංකේත භාවිතා වෙයි. ඒවා මෙසේය.

● **බයිනරි ඇන්ඩ් ඔපරේටර් (binary AND operator) ("&")** - දී ඇති සංඛ්‍යා දෙකක් බයිනරි සංඛ්‍යා බවට හරවා, බුලියන් And ද්වාරය හෙවත් ගේට් එක හරහා ප්‍රතිඵල ලැබෙන ආකාරයට, සංඛ්‍යා දෙක ගණනය කරනු ලබයි. ඉන්පසුව ලැබෙන පිළිතුර ඩෙසිමල් සංඛ්‍යා වක් බවට හරවා අවුට්පුට් හෙවත් අවුට්පුට් කරයි. බුලියන් And ගේට් එක ක්‍රියාත්මක වෙන්නේ 'සහ' යන ක්‍රියාවට අනුවය. මෙහිදී බයිනරි සංඛ්‍යා ලෙස 1 සහ 1 යන අවස්ථාවේදී පමණක් පිළිතුර ලෙස 1 ලැබෙයි. මෙහිදී 0 සහ 1, 1 සහ 0 මෙන්ම 0 සහ 0 යන අවස්ථාවලදී පිළිතුර ලැබෙන්නේ 0 ලෙසය.

● **බයිනරි ඔර් ඔපරේටර් (binary OR operator) ("|")** - දී ඇති සංඛ්‍යා දෙකක් බයිනරි සංඛ්‍යා බවට හරවා, බුලියන් Or ගේට් එක හරහා ප්‍රතිඵල ලැබෙන ආකාරයට සංඛ්‍යා දෙක ගණනය කරනු ලබයි. පසුව ලැබෙන පිළිතුර ඩෙසිමල් සංඛ්‍යාවක් බවට හරවා අවුට්පුට් කරයි. බුලියන් Or ගේට් එක ක්‍රියාත්මක වෙන්නේ 'හෝ' යන ක්‍රියාවට අනුවය. මෙහිදී බයිනරි සංඛ්‍යා ලෙස 0 හෝ 0 යන අවස්ථාවේදී පමණක් පිළිතුර ලෙස 0 ලැබෙයි. 0 හෝ 1, 1 හෝ 0 මෙන්ම 1 හෝ 1 යන අවස්ථාවලදී පිළිතුර ලැබෙන්නේ 1 ලෙසය.

● **බයිනරි එක්ස් ඔපරේටර් (binary XOR operator) ("^")** - දී ඇති සංඛ්‍යා දෙකක් බයිනරි සංඛ්‍යා බවට හරවා, බුලියන් Xor ගේට් එක හරහා ප්‍රතිඵල ලැබෙන ආකාරයට සංඛ්‍යා දෙක ගණනය කරනු ලබයි. පසුව ලැබෙන පිළිතුර ඩෙසිමල් සංඛ්‍යාවක් බවට හරවා අවුට්පුට් කරයි. බුලියන් Xor ද්වාරයේ ක්‍රියාකාරිත්වය බොහෝ දුරට Or ද්වාරයේ ක්‍රියාකාරිත්වයට සමානය. මෙහි ඇති එකම වෙනස නම් 1 හෝ 1 යන අවස්ථාවේදී පිළිතුර 0 ලෙස ලැබීමය.

● **බයිනරි නම්පලිමන්ට් ඔපරේටර් (binary complement operator) ("~")** - දී ඇති සංඛ්‍යාවකට විරුද්ධ සංඛ්‍යාව මෙහිදී පිළිතුර ලෙස අවුට්පුට් කෙරේ. ඒ අනුව 1 ලබාදී ඇත්නම් අවුට්පුට් වෙන්නේ 0 යන්නයි. 0 අවුට්පුට් කර ඇත්නම් 1 යන්න අවුට්පුට් වේ.

● **බයිනරි ලෙෆ්ට් ෂිෆ්ට් ඔපරේටර් (binary left shift operator) ("<<")** - දී ඇති සංඛ්‍යාවක් බයිනරි සංඛ්‍යාවක් බවට හරවා සම්පූර්ණ සංඛ්‍යාව එක් බිට්වකින් වම් පසට ගෙන යනු ලබයි.

● **බයිනරි රයිට් ෂිෆ්ට් ඔපරේටර් (binary right shift operator) (">>")** - දී ඇති සංඛ්‍යාවක් බයිනරි සංඛ්‍යාවක් බවට හරවා සම්පූර්ණ සංඛ්‍යාව එක් බිට්වකින් දකුණු පසට ගෙන යනු ලබයි. මෙම බිට් වයිස් ඔපරේටර්ස් ක්‍රියාත්මක වෙන ආකාරය අවබෝධ කර ගැනීම සඳහා පහත දැක්වෙන කෝඩ් සටහන අධ්‍යයනය කරන්න.

```

int a = 60;
int b = 13;
int c = 0;
c = a & b;
cout << "Line 1 - Value of c is : " << c << endl ;
c = a | b;
cout << "Line 2 - Value of c is : " << c << endl ;
c = a ^ b;
cout << "Line 3 - Value of c is : " << c << endl ;
c = ~a;
cout << "Line 4 - Value of c is : " << c << endl ;
c = a << 1;
cout << "Line 5 - Value of c is : " << c << endl ;
c = a >> 1;
cout << "Line 6 - Value of c is : " << c << endl ;

```

● මෙහි පළමු පේළි තුන තුළදී a, b සහ c ලෙස වේරියබල් හෙවත් වේරියබල්ස් තුනක් සකස් කර ඒවා සඳහා 60, 13 සහ 0 යන අගය ඇතුළු කර තිබේ. මෙහි හතර වැනි පේළිය තුළදී බයිනරි ඇන්ඩ් ඔපරේටර් එක භාවිතා කොට තිබේ. ඒ අනුව එහිදී a වේරියබල් එකෙහි ඇති අගය මූලිකම බයිනරි සංඛ්‍යාවක් බවට හරවයි. එවිට එය ලැබෙන්නේ 111100 ලෙසය. දෙවනුව b වේරියබල් එකෙහි අගයද බයිනරි සංඛ්‍යාවක් බවට හරවයි. එවිට අගය ලැබෙන්නේ 1101 ලෙසය. දැන් මෙය මූලික ලැබුණු සංඛ්‍යාව සමඟ ඇන්ඩ් ඔපරේටර් එකෙන් සලකා බැලෙයි. ඒ පහත දැක්වෙන පිටපතය.

60 = 111100

13 = 001101

12 = 001100

● මෙහි පිළිතුර වෙන 001100 යන්න ඩෙසිමල් සංඛ්‍යාවලින් නම් 12ය. ඒ අනුව පිළිතුර ලෙස 12 යන්න 5 වැනි පේළිය මගින් අවුට්පුට් කරයි. මේ ආකාරයෙන්ම 6 වැනි සහ 8 වැනි පේළි දෙකද ක්‍රියාත්මක වේ. ඒවායේදී බයිනරි ඔර් ඔපරේටර් එක සහ බයිනරි එක්ස් ඔපරේටර් එක භාවිතා කොට ඇත.

● මෙහි 10 වැනි පේළියේදී භාවිතා කොට තිබෙන්නේ බයිනරි නම්පලිමන්ට් ඔපරේටර් එකයි. එහිදී a වේරියබල් එකෙහි ඇති අගයේ අනුපූරකය හෙවත් කම්පලිමන්ට් එක සලකා බැලෙයි. මෙහි ඇති 12 වැනි පේළියේදී බයිනරි ලෙෆ්ට් ෂිෆ්ට් ඔපරේටර් එක භාවිතා කොට තිබේ. එහිදී a වේරියබල් එකෙහි ඇති අගය බයිනරි සංඛ්‍යාවක් බවට හරවා එක් බිට් එකක් වම් පසට ලබා ගන්නේ පහත දැක්වා ඇති සේය.

60 = 111100

බිට් එකක් වම් පසට යොමු කිරීම = 1111000

● මෙවිට 120 යන්න පිළිතුර ලෙස ලැබෙයි. මේ ආකාරයෙන්ම 14 වැනි පේළියද ක්‍රියාත්මක වේ. එහිදී බිට් එක යොමු වෙන්නේ දකුණු පසටයි. එය පහත ආකාරයෙන් දැක්විය හැකිය.

60 = 111100

බිට් එකක් දකුණු පසට යොමු කිරීම = 011110

● මෙවිට 30 යන්න පිළිතුර ලෙස ලැබෙයි. මෙම අවස්ථා දෙකේදීම බිට් 1ක් බැගින් යොමු වෙන්නේ a << 1 සහ a >> 1 ලෙස දක්වා තිබීම නිසාය. මෙවායේ 1 වෙනුවට වෙනත් අංකයක් යොදා තිබුණේ නම් එම අංකයෙන් දක්වා ඇති බිට් ගණනින් පිහිටීම වෙනස්වී පිළිතුර ලැබේ. (06 රූපය) මෙම වැඩසටහනෙහි ඇති අගයන් වෙනස් කරමින් එය ක්‍රියාත්මක කොට බලා, බිට් වයිස් ඔපරේටර්ස් ක්‍රියාත්මක වෙන ආකාරය අවබෝධ කර ගන්නට උත්සාහ කරන්න. ■



## හතර වැනි පරිච්ඡේදය

## වැඩසටහනකදී කන්ඩිෂනල් ස්ට්‍රක්චර්ස් භාවිතා කරන ආකාරය

**සිදු** ම පරිගණක භාෂාවකමිදීම පාහේ කන්ඩිෂනල් ලොජික් ක්‍රමය භාවිතා වේ. මේවායින් කෙරෙන්නේ, දෙනු ලබන කොන්දේසියක් හෙවත් කන්ඩිෂන් එකක් පරීක්ෂා කර බලා, ඊට උචිත පිළිතුරු ලබා දීමත්, එම පිළිතුරු අනුව යමින් වැඩ සටහන ඉදිරියට ක්‍රියාත්මක කිරීමත්ය. මෙම කන්ඩිෂනල් ලොජික් භාවිතා කරන ආකාරය ක්‍රමලේඛ භාෂාවෙන් භාෂාවට වෙනස් වෙයි. එසේ යම් භාෂාවක භාවිතා කරන කන්ඩිෂනල් ලොජික් විධාන කීපයකින් සෑදි සැලැස්මක් කන්ඩිෂනල් ස්ට්‍රක්චර් (conditional structure) එකක් ලෙස හඳුන්වයි. මෙසේ සි ජලස් ජලස් තුළදී කන්ඩිෂනල් ලොජික් භාවිතා කරන ක්‍රම කීපය කි. ඒ ගැන විස්තර මෙසේය.

## දෙනු ලබන කන්ඩිෂන් එකක් සත්‍ය නම් පමණක්

## යම් කාර්යක් ඉටුකිරීමට සැලැස්වීම

මේ සඳහා යොදා ගත යුතු කන්ඩිෂනල් ස්ට්‍රක්චර් එකෙහි පොදු ආකෘතිය මෙසේය.

```
if (කන්ඩිෂන් එක)
```

```
{
```

```
    කන්ඩිෂන් එක සත්‍ය නම් විය යුතු කාර්ය;
```

```
}
```

මෙහි ඇති if යන පදය සීමිත අකුරුවලින් පමණක් ලිවිය යුතුය. මෙහිදී වරහන් තුළ සටහන් කරන්නේ කන්ඩිෂන් එකයි. එම කන්ඩිෂන් එක සත්‍ය නම් විය යුතු කාර්යක් හෝ කාර්ය කීපයක් හෝ සඟල වරහන් තුළ සටහන් කළ යුතුය.

මෙම කෝඩ් ලිවීමේ පිළිවෙළ අවබෝධ කර ගැනීම පිණිස පහත දක්වන කෝඩ් සටහන අධ්‍යයනය කරන්න. (01 රූපය)

```
*main.cpp x
1  #include <iostream>
2  using namespace std;
3  int main () {
4      int a = 100;
5      if( a == 100 )
6      {
7          cout << "Yes" << endl;
8      }
9      return 0;
10 }
11
```

01

```
#include <iostream>
using namespace std;
int main () {
    int a = 100;
    if( a == 100 )
    {
        cout << "Yes" << endl;
    }
    return 0;
}
```

● මෙහි නිල් පැහැයෙන් දක්වා ඇති කෝඩ් කොටස් සෑම වැඩසටහනක් සඳහාම අත්කරගත විය යුතු ඒවාය. එමෙන්ම ඒවා කෝඩ් ලිවීමට යොදා ගන්නා කෝඩ්බලොක්ස් වැඩසටහන විසින් ලබා දෙන බැවින් අමුතුවෙන් ලිවිය යුතු නැත. මෙහි විස්තර කෙරෙන්නේ තමා විසින් ලිවිය යුතු කෝඩ් කොටස් පිළිබඳව පමණි. කළු පැහැයෙන් දක්වා ඇත්තේ ඒවායි.

● මෙසේ කළු පැහැයෙන් ඇති පළමු පේළියේදී a නම් වේරියබල් එකට 100 යන අගය ඇතුළු කර තිබේ. දෙවැනි පේළියේදී යොදා ඇති කන්ඩිෂනල් ලොජික් එක මගින් a වේරියබල් එකෙහි ඇති අගය 100 ට සමානදැයි පරීක්ෂා කර බලා, එය එසේ නම් Yes යන්න අවුට් පුට් කරයි. (02 රූපය)

මෙහිදී කන්ඩිෂන් එක පරීක්ෂා කරන විට ඒ සඳහා සමාන (==) යන කම්පැරිසන් ඔපරේටර් එක (comparison operators) යොදා ගෙන ඇත. මේ වෙනුවට විශාල හෝ සමාන (>=), කුඩා (<) වැනි වෙනත්

```
main.cpp x
Select C:\Users\Patganaka\Documents\EasyGuide\src\Debug\EasyGuide.exe
Yes

Process returned 0 (0x0)   execution time : 0.020 s
Press any key to continue.
```

02

```

main.cpp X
1  #include <iostream>
2  using namespace std;
3  int main () {
4      int a = 100;
5      if( a == 100 ) {
6          cout << "Yes" << endl;
7      } else {
8          cout << "No" << endl;
9      }
10     cout << "Good bye!" << endl;
11     return 0;
12 }
13

```

03

```

main.cpp X
1  #include <iostream>
2  using namespace std;
3  int main () {
4      char grade = 'B';
5      switch(grade)
6      {
7          case 'A' :
8              cout << "Excellent!" << endl;
9              break;
10         case 'B' :
11             cout << "Good!" << endl;
12             break;
13         case 'C' :
14             cout << "Well done" << endl;
15             break;
16         case 'D' :
17             cout << "You passed" << endl;

```

05

```

main.cpp X
C:\Users\Parigana\Documents\C\EasyGuide\bin\Debug\EasyGuide.exe
Yes
Good bye!

Process returned 0 (0x0)   execution time : 0.048 s
Press any key to continue.

```

04

```

main.cpp X
C:\Users\Parigana\Documents\C\EasyGuide\bin\Debug\EasyGuide.exe
Good!

Process returned 0 (0x0)   execution time : 0.020 s
Press any key to continue.

```

06

කම්පෑරිසන් ඔපරේටර් එකක් වුවද භාවිතා කරන්නට පුළුවන.

මෙම කේත සටහනම පහත දක්වා ඇති ආකාරයට ලියුවේ නම් කිසිදු අවුට්පුට් එකක් ලැබෙන්නේ නැත. එහිදී දී ඇති කන්ඩිෂන් එක අසත්‍ය වෙත බැවිනි.

```

int a = 100;
if( a > 100 )
{
    cout << "Yes" << endl;
}

```

දෙනු ලබන කන්ඩිෂන් එකක් සත්‍ය නම් එක් කාර්යක්ද, අසත්‍ය නම් තවත් කාර්යක්ද ඉටු වීමට සැලැස්වීම මේ සඳහා යොදා ගත යුතු කන්ඩිෂනල් ස්ට්‍රක්චර් එකෙහි පොදු ආකෘතිය මෙසේය.

```

if(කන්ඩිෂන් එක) {
    කන්ඩිෂන් එක සත්‍ය නම් විය යුතු කාර්ය;
}
else
{
    කන්ඩිෂන් එක අසත්‍ය නම් විය යුතු කාර්ය;
}
කන්ඩිෂන් එක අසත්‍ය නම් විය යුතු කාර්ය;

```

මෙම කෝඩ් ලිවීමේ පිළිවෙළ අවබෝධ කර ගැනීම පිණිස පහත දැක්වෙන කෝඩ් සටහන අධ්‍යයනය කරන්න. (03 රූපය)

```

int a = 100;
if( a == 100 ) {
    cout << "Yes" << endl;
} else {
    cout << "No" << endl;
}
cout << "Good bye!" << endl;

```

මෙහිදී පළමු පේළියේදී a නම් වේරියබල් එකට 100 යන අගය ඇතුළු කර තිබේ. දෙවැනි පේළියේදී a වේරියබල් එකෙහි ඇති අගය 100 දැයි පරීක්ෂා කර බලයි. එය එසේ නම් තුන් වැනි පේළිය ක්‍රියාත්මක වී Yes යන්න අවුට්පුට් කරයි. මෙහි 4 වැනි පේළියේ තිබෙන else කොටස ක්‍රියාත්මක වෙන්නේ දෙවැනි පේළියේ ඇති කන්ඩිෂන් එක අසත්‍ය නම් එවිටය. කන්ඩිෂන් එක සත්‍ය වුවද අසත්‍ය වුවද, මෙහි අවසානයේ ඇති කේත පේළිය අවුට්පුට් වේ. (04 රූපය)

මෙම ක්‍රමයේදී කන්ඩිෂන් එකක් සත්‍ය නම් හෝ අසත්‍ය නම් හෝ එක් කාර්යක් පමණක් නොව කාර්ය කීපයක්ම වුවද කර ගන්නට පුළුවන. ඒ සඳහා අවශ්‍ය කරන කෝඩ් පේළි if හෝ else හෝ යන්නට පසුව ලිවිය යුතුය.

**දී ඇති කන්ඩිෂන් එකක් අසත්‍ය නම් නවත් කන්ඩිෂන් එකක් පරීක්ෂා කිරීම**

ඉහත විස්තර කළ කෝඩ් සටහන් තුළදී, දෙනු ලැබ ඇති කන්ඩිෂන් එකක් අසත්‍ය නම් යම් කාර්යක් හෝ කාර්ය කීපයක් හෝ කරන ලදී. එහෙත් දී ඇති කන්ඩිෂන් එක අසත්‍ය වීමේ තවත් කන්ඩිෂන් එකක් පරීක්ෂා කර බලන්නට අවශ්‍ය නම් else යටතේ තවත් if විධානයක් යෙදිය යුතුය. එහෙත් ඒ වෙනුවට ස්විච් (switch) කන්ඩිෂනල් ස්ට්‍රක්චර් එක යොදා ගන්නට පුළුවන. මෙම කෝඩ් ලිවිය යුතු පොදු ආකෘතිය මෙසේය.

```

switch (ස්විච්මන්ට් එක) {
    case ස්විච්මන්ට් එකෙහි සත්‍යතාව:
        සත්‍ය නම් විය යුතු කාර්ය;
        break; අවශ්‍යතාව මත පමණක්
    case ස්විච්මන්ට් එකෙහි සත්‍යතාව:
        සත්‍ය නම් විය යුතු කාර්ය;
        break; අවශ්‍යතාව මත පමණක්
    default : අවශ්‍යතාව මත පමණක්

```



සියලු දේ සහය නම් විය යුතු කාර්යය;

මෙහිදී මූලික switch ඉදිරියෙන් සටහන් කරන්නේ ප්‍රකාශනයක හෙවත් ස්ටේට්මන්ට් එකකි. කන්ඩිෂන්වල සහයතාව පරීක්ෂා කරන්නේ එම ස්ටේට්මන්ට් එක තුළ භාවිතා වෙන වේරියබල් එකට අනුවය. දෙවැනි ඡේදයේ ඇති case යන්න ඉදිරියේ සටහන් කරන්නේ ස්ටේට්මන්ට් එකෙහි සහයතාව පරීක්ෂා කළ යුතු ආකාරයයි. ඉන්පසුව ඇති ඡේදයේ එම ස්ටේට්මන්ට් එක සහය නම් විය යුතු කාර්ය හෝ කාර්ය කීපයක් හෝ වුවද සටහන් කරන්නට පුළුවන.

මෙහි ඇති break; යන්න යොදා ගැනෙන්නේ එම ස්ටේට්මන්ට් එක සහය වූ පසුව ඉන් ඉදිරියට වැඩසටහන ක්‍රියාත්මකවීම නතර කිරීමට යි. මේ නිසා එය යොදන්නේ අවශ්‍යතාව මත පමණි. මේ ආකාරයෙන් අවශ්‍ය තරම් case ලෙස ඇති කෝඩ් කොටස් ලියන්නට පුළුවන.

සියල්ල අවසානයේදී එක් ස්ටේට්මන්ට් එකක් හෝ සහය නැත්නම් විය යුතු කාර්යක් තිබේ නම් default : යන්න සඳහන් කර ඊට පසු ඡේදයේ එය දක්වන්නට පුළුවන.

මෙම කෝඩ් ලිවීමේ පිළිවෙළ අවබෝධ කර ගැනීම පිණිස පහත දැක්වෙන කෝඩ් අධ්‍යයනය කරන්න. (05 රූපය)

```
char grade = 'B';
switch(grade)
{
case 'A' :
cout << "Excellent!" << endl;
break;
case 'B' :
cout << "Good!" << endl;
```

```
break;
case 'C' :
cout << "Well done" << endl;
break;
case 'S' :
cout << "You passed" << endl;
break;
case 'W' :
cout << "Better try again" << endl;
break;
default :
cout << "Invalid grade" << endl;
}
```

මෙහි පළමු ඡේදයේදී grade නම් වේරියබල් එක වෙතට B නම් අගය ඇතුළු කර තිබේ. දෙවැනි ඡේදයේදී switch යන්නට ඉදිරියෙන් grade ලෙස, ඉහත සකස් කළ වේරියබල් එකෙහි නම් ලබාදී ඇති නිසා switch යටතේ පරීක්ෂා කරන්නේ grade වේරියබල් එක සම්බන්ධවය. මේ අනුව 4 වැනි ඡේදයේදී case මගින් ඉහත වේරියබල් එකෙහි A අකුර තිබේදැයි පරීක්ෂා කරයි. එසේ නම් Excellent! යන්න අවුට්පුට් කර 6 වැනි ඡේදයෙන් වැඩසටහන අවසන් කරනු ලැබේ.

මෙහි 7 වැනි ඡේදය ක්‍රියාත්මක වෙන්නේ එම 4 වැනි ඡේදයේ තිබුණු ස්ටේට්මන්ට් එක අසහය නම් පමණි. ඒ අනුව එහිදී වේරියබල් එකෙහි තිබෙන්නේ B දැයි පරීක්ෂා කර බලනු ලැබේ. එසේ තිබේ නම් Good! යන්න අවුට්පුට් කෙරේ. මේ ආකාරයෙන් අනෙක් කෝඩ් කොටස්ද ක්‍රියාත්මක වේ.

මෙම වැඩසටහන ක්‍රියාත්මක වෙන විට මූලික grade වේරියබල් එකට B ඇතුළු කර ඇති බැවින් Good! යන්න අවුට්පුට් වෙයි. (06 රූපය)

## පස් වැනි පරිච්ඡේදය

### ලූප්ස් භාවිතා කරන ආකාරය

**සිදු** ම ක්‍රමලේඛ භාෂාවකදීම පාහේ පුනරුත්තර හෙවත් ලූප්ස් (loops) භාවිතා වේ. ලූප් එකක් මගින් කරන්නේ එකම කෝඩ් කොටසක් නැවත නැවත ක්‍රියාත්මක කරවීමයි. මෙහිදී ලූප් එකෙහි දී ඇති කන්ඩිෂන් අනුව ලූප් එකෙහි කෝඩ් සියල්ලම ක්‍රියාත්මකවී අවසන් වෙන තෙක් වෙනත් කෝඩ් කිසිවක් ක්‍රියාත්මක නොවේ. ක්‍රියාකාරීත්වය එකිනෙකට වෙනස්වීම මත ලූප් වර්ග ගණනාවක් වෙයි. ඒවා පහත දක්වා ඇති පරිදි වෙයි.

#### ෆෝ ලූප් එක භාවිතා කිරීම

ෆෝ (for) ලූප් එක භාවිතා කරන ආකාරය මෙහි විස්තර කර තිබේ.

#### ෆෝ ලූප් එකෙහි පොදු ආකෘතිය

ෆෝ (for) ලූප් යනු ක්‍රියාත්මක වෙන වාර ගණන මූලික ලබාදිය හැකි ලූප් ආකාරයකි. මේ නිසා යම් කෝඩ් එකක් නිශ්චිත වට ගණනාවක් තුළදී නැවත නැවත ක්‍රියාත්මක විය යුත්තේ නම් එම

කෝඩ් ලිවීමට ෆෝ ලූප්ස් භාවිතා කිරීම පහසුය. මෙහි පොදු කෝඩ් ආකෘතිය හෙවත් කෝඩ් ස්ට්‍රක්චර් එක මෙසේය.

```
for (int වේරියබල් එකක කන්ඩිෂන් එකක පියවර ගණන)
{
ලූප් එක තුළ ඇති කෝඩ් කොටස්;
}
ලූප් එකෙන් පිටත ඇති කෝඩ්
```

ෆෝ ලූප් එක ලියන විට for යන්න සීමිතව අකුරින් ලිවිය යුතුය. මෙසේ for යන්නට පසුව ඇති වේරියබල් එකෙහි අගය ලූප් එක ක්‍රියාත්මක වෙන සෑම වටයකදීම වෙනස් වෙයි. ලූප් එක වටයෙන් වටය ක්‍රියාත්මක වෙන බව හඳුනා ගන්නේ මෙම වේරියබල් එකෙනි. ලූප් එකෙහි ක්‍රියාකාරීත්වය අවසානයේදී එම අගය වෙනස්වීමද නවතී. මෙහි ඇති තිත් කොමා ලකුණු ඒ ආකාරයෙන් යෙදිය යුතුය.

```

1 #include <iostream>
2 using namespace std;
3 int main () {
4     for(int i = 1; i <= 10; i = i + 1)
5     {
6         cout << "value of i: " << i << endl;
7     }
8     return 0;
9 }
10

```

01

```

1 #include <iostream>
2 using namespace std;
3 int main () {
4     for(int i = 1; i <= 10; i = i + 2)
5     {
6         cout << "value of i: " << i << endl;
7     }
8     return 0;
9 }
10

```

03

```

value of i: 1
value of i: 2
value of i: 3
value of i: 4
value of i: 5
value of i: 6
value of i: 7
value of i: 8
value of i: 9
value of i: 10

Process returned 0 (0x0)   execution time : 0.053 s
Press any key to continue.

```

02

```

1 #include <iostream>
2 using namespace std;
3 int main () {
4     for(int i = 10; i >= 1; i = i - 1)
5     {
6         cout << "value of i: " << i << endl;
7     }
8     return 0;
9 }
10

```

04

මෙහි **කන්ඩිෂන් එක** යන නැත ලූප් එක ක්‍රියාත්මක කරවීම සඳහා බලපාන කන්ඩිෂන් එක ලබා දිය යුතුය. එවිට එම කන්ඩිෂන් එක සත්‍යව පවතින තාක් කල් එම ලූප් එක ක්‍රියාත්මක වෙයි. මෙහි ඇති **පියවර ගණන** යන නැත ලූප් එක ක්‍රියාත්මක විය යුත්තේ පියවරෙන් පියවරද, පියවර දෙකෙන් දෙකටද යන්න පිළිබඳ තොරතුරු ලබා දිය යුතුය.

- මෙහි දෙවැනි පේළියේ තිබෙන සඟල වරහනින් ලූප් එකෙහි ආරම්භය දැක්වෙයි. තුන් වැනි පේළියේදී කෝඩ් ලියනු ලබන්නේ ලූප් එක ක්‍රියාත්මක වෙන සෑම වටයකදීම ක්‍රියාත්මකවීම සඳහාය. මේ ආකාරයට කෝඩ් පේළි ගණනාවක් ලියන්නට පුළුවන.

- මෙහි හතර වැනි පේළියේ වසා ඇති සඟල වරහනින් ලූප් එකෙහි අවසානය දැක්වේ. මේ අනුව 5 වැනි පේළිය ලූප් එකට අයත් නැති බව C++ භාෂාව හඳුනා ගනු ලබයි. මෙසේ ඇති ෆෝ ලූප් එක භාවිතා කරන ආකාරය හේරුම් ගන්නට පහත දැක්වෙන කෝඩ් සටහන අධ්‍යයනය කරන්න. (01 රූපය)

```

#include <iostream>
using namespace std;
int main () {
    for(int i = 1; i <= 10; i = i + 1)
    {
        cout << "value of i: " << i << endl;
    }
    return 0;
}

```

මෙහි නිල් පැහැයෙන් ඇති කෝඩ් කොටස් සෑම වැඩසටහනක් සඳහාම අන්තර්ගත විය යුතු ඒවාය. එමෙන්ම ඒවා කෝඩ් ලිවීමට යොදා ගන්නා කෝඩ් බලාපොරොත්තු වැඩසටහන විසින් ලබා දෙන බැවින් අමුතුවෙන් ලිවිය යුතු නැත. මෙහි විස්තර කෙරෙන්නේ නම් විසින්

ලිවිය යුතු කෝඩ් කොටස් පිළිබඳව පමණි. මෙම සටහන තුළ කළ පැහැයෙන් දක්වා ඇත්තේ ඒවායි.

- මෙසේ කළ පැහැයෙන් දක්වා ඇති පළමු පේළියේදී මුලින්ම `int i = 1` ලෙස ලබාදී තිබෙන්නේ ලූප් එකෙහි ආරම්භක අගයයි. ඒ අනුව ආරම්භයේදී `i` නම් වේරියබල් එකෙහි අගය 1 ක් වේ. මෙහි ඇති පළමු තිත් කොමා ලකුණට පසුව තිබෙන්නේ කන්ඩිෂන් එකයි. එය `i <= 10` ලෙස ලබාදී ඇති බැවින් `i` වේරියබල් එකෙහි ඇති අගය 10 ට වඩා කුඩාව හෝ සමානව හෝ තිබෙන තාක් ලූප් එක ක්‍රියාත්මක වෙයි. මෙහිදී `i` වේරියබල් එකෙහි අගය 11 වූ විට තව දුරටත් ලූප් එක තුළ ඇති කෝඩ් ක්‍රියාත්මක වෙන්නේ නැත.

- මෙහි දෙවැනි තිත් කොමාවට පසුව තිබෙන්නේ ලූප් එක ක්‍රියාත්මක විය යුතු පියවර ගණනයි. එය `i = i + 1` ලෙස ලබාදී ඇති බැවින් ලූප් එක පියවරෙන් පියවර ක්‍රියාත්මක වෙයි. මේ අනුව පළමු වටයේදී `i` වේරියබල් එකෙහි ඇති අගය 1 වුවත් දෙවැනි වටයේදී එය 2 ලෙස වෙනස් කරන්නේද තුන් වැනි වටයේදී `i` වේරියබල් එකෙහි අගය 3 ලෙස වෙනස් කරන්නේද මෙම කෝඩ් කොටස මගිනි.

- මෙහි ඇති කළ පැහැයෙන් දක්වා තිබෙන 3 වැනි පේළියේ කෝඩ් ලියා තිබෙන්නේ `i` වේරියබල් එක අඩුවීමට කිරීමටය. මේ අනුව 1 සිට 10 දක්වා ඉලක්කම් වරකට එක බැගින් අඩුවීමට කෙරේ. (02 රූපය)

- මෙහිදී පියවර දෙකෙන් දෙකට ලූප් එක ක්‍රියාත්මක විය යුතු නම් මෙහි ඇති 1 නම් අගය වෙනුවට 2 යන්න ලබාදිය යුතුය. එවිට 1,3,5,7,9 යන ඔත්තේ සංඛ්‍යා අඩුවීමට වෙයි. (03 රූපය)

**වැඩි අගයක සිට අඩු අගයක් දක්වා ලූප් එක ක්‍රියාත්මක කරවීම**

ඉහත ආකාරයට ආරම්භක අගය, කන්ඩිෂන් එක මෙන්ම පියවර ගණන ලබාදුන් විට ලූප් එක ක්‍රියාත්මක වෙන්නේ අඩු අගයක සිට වැඩි අගයක් දක්වාය. එහෙත් පරාසය ලබාදීමට අවශ්‍ය වෙන්නේ වැඩි අගයක සිට අඩු අගයක් දක්වා ක්‍රියාත්මකවීමට නම් ආරම්භක අගය, කන්ඩිෂන් එක මෙන්ම පියවර ගණනද වෙනස් කළ යුතුය. එය



```
main.cpp X
C:\Users\Panganaka\Documents\EasyGuide\bin\Debug\EasyGuide.exe

value of i: 10
value of i: 9
value of i: 8
value of i: 7
value of i: 6
value of i: 5
value of i: 4
value of i: 3
value of i: 2
value of i: 1

Process returned 0 (0x0)   execution time : 0.024 s
Press any key to continue.
```

05

```
main.cpp X
1 #include <iostream>
2 using namespace std;
3 int main () {
4     int A1 = 10;
5     while (A1 >= 1) {
6         A1 = A1 - 1;
7     }
8     cout << "value of A1: " << A1 << endl;
9     return 0;
10 }
11
```

07

```
main.cpp X
1 #include <iostream>
2 using namespace std;
3 int main () {
4     int i=1;
5     for ( ; i <= 10; )
6     {
7         cout << "value of i: " << i << endl;
8         i = i+1;
9     }
10     return 0;
11 }
12

C:\Users\Panganaka\Documents\EasyGuide\bin\Debug\EasyGuide.exe

value of i: 1
value of i: 2
value of i: 3
```

06

```
main.cpp X
C:\Users\Panganaka\Documents\EasyGuide\bin\Debug\EasyGuide.exe

value of A1: 0

Process returned 0 (0x0)   execution time : 0.036 s
Press any key to continue.
```

08

අවබෝධ කර ගැනීම සඳහා පහත දැක්වෙන කෝඩ් අධ්‍යයනය කරන්න. (04 රූපය)

```
for(int i = 10; i >= 1; i = i - 1)
{
    cout << "value of i: " << i << endl;
}
```

● මෙහිදී ආරම්භක අගය ලෙස ලබාදී තිබෙන්නේ 10 යන්නයි. එහිදී ලබාදී තිබෙන කන්ඩිෂන් එකට අනුව එම වේරියබල් එකෙහි අගය එකට වඩා විශාලව හෝ සමානව හෝ තිබෙන තාක් ලූප් එක ක්‍රියාත්මක වෙයි. මෙහිදී පියවර ගණන ලබාදී තිබෙන්නේ i වේරියබල් එකෙහි ඇති අගය එකින් එක අඩු කිරීම සඳහාය. ඒ සඳහා i = i - 1 යන්න ලබාදී තිබේ.

● මේ අනුව ලූප් එක ක්‍රියාත්මක වෙන විටදී 10 සිට 1 දක්වා සංඛ්‍යා වරකට 1 බැගින් අඩුවීයාම කෙරේ. (05 රූපය) මෙහිදීද එක් පියවරෙන් පියවර අඩු කිරීම වෙනුවට පියවර දෙකෙන් දෙක අඩු කිරීමට අවශ්‍ය නම් i = i - 2 ලෙස මෙම කෝඩ් එක ලිවිය යුතුය.

#### ෆෝ ලූප් එක ලිවිය හැකි වෙනත් ආකාර

ෆෝ ලූප් එක ඉහත ආකෘතියට වඩා තරමක් වෙනස් ආකාරයෙන්ද ලියන්නට පුළුවන. මෙය තේරුම් ගැනීමට පහත දැක්වෙන කෝඩ් අධ්‍යයනය කරන්න.

```
int i=1;
for ( ; i <= 10; )
{
    cout << "value of i: " << i << endl;
    i = i+1;
}
```

● මෙම කෝඩ් සටහනේ for ලූප් එක ලියා ඇති පේළිය තුළ වේරියබල් එක සහ පියවර ගණන නොමැති බව පෙනෙනු ඇති. එම 'අඩු පාඩුව' මග හරවා තිබෙන්නේ කෝඩ් සටහන තුළ ඇති අනෙක්

කෝඩ්වලිනි. ඒ අනුව මෙහිදී පළමු පේළිය තුළ i නම් වේරියබල් එක සකස් කර එයට 1 ඇතුළු කර ඇත. මේ අනුව ලූප් එක තුළ වේරියබල් එකක් හෝ ආරම්භක අගයක් හෝ දක්වා නොතිබුණද, කන්ඩිෂන් එක ලෙස i වේරියබල් එකෙහි අගය 10 ට වඩා කුඩා හෝ සමාන හෝ වේදැයි විමසා බලන බැවින් මෙහිදී ලූප් එක ක්‍රියාත්මක කරවීම සඳහා, පළමු පේළියේ සකස් කර ඇති i වේරියබල් එක සහ එහි අගය භාවිතා කෙරේ.

● එමෙන්ම මෙහි දෙවැනි පේළිය තුළ ලූප් එක ක්‍රියාත්මක වෙන පේළි ගණන දක්වා නොතිබුණද 5 වැනි පේළිය තුළ i වේරියබල් එකට 1 බැගින් එකතු කරන නිසා එය ලූප් එක ක්‍රියාත්මක කරවීමේ පියවර ලෙස භාවිතා වේ. මේ අනුව මෙම කෝඩ් සටහන ක්‍රියාත්මක වෙන විටදී 1 සිට 10 දක්වා ඇති සංඛ්‍යා අඩුවීයාම කෙරේ. (06 රූපය)

#### වයිල් ලූප් එක භාවිතා කිරීම

වයිල් (While) ලූප් එක භාවිතා කරන ආකාරය මෙහි විස්තර කර තිබේ.

#### වයිල් ලූප් එකෙහි පොදු ආකෘතිය

කන්ඩිෂන් එකක් පරීක්ෂා කරමින් එම කන්ඩිෂන් එක සත්‍යව ඇති-තාක් පමණක් උපදෙස් මාලාවක්, නැවත නැවත ක්‍රියාත්මක කිරීමට වයිල් ලූප් එක භාවිතා කෙරේ. මෙහිදී කන්ඩිෂන් එක පරීක්ෂා කර බලනුයේ ලූප් එකට ඇතුළුවීමට කලින්. මේ නිසා කන්ඩිෂන් එක අසත්‍ය නම් ලූප් එකට ඇතුළු වෙන්නේ නැත. මෙහි කෝඩ් ස්ට්‍රක්චර් එක මෙසේය.

```
while (කන්ඩිෂන් එක)
```

```
{
```

```
    විය යුතු කාර්ය හෝ කාර්යන්;
```

```
}
```

මෙම කෝඩ් ආකෘතියට අනුව කෝඩ් ලියන ආකාරය අවබෝධ කර ගැනීම පිණිස පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න. (07 රූපය)

```

1 #include <iostream>
2 using namespace std;
3 int main () {
4     int A1 = 10;
5     while (A1 >= 1) {
6         A1 = A1 - 1;
7         cout << "value of A1: " << A1 << endl;
8     }
9     return 0;
10 }
11

```

09

```

1 #include <iostream>
2 using namespace std;
3 int main () {
4     int a = 5;
5     do {
6         a = a - 1;
7         cout << "value of a: " << a << endl;
8     } while (a >= 1);
9     return 0;
10 }
11

```

11

```

main.cpp X
C:\Users\Pariganaika\Documents\C++\EasyGuide\bin\Debug\EasyGuide.exe
value of b: 55
Process returned 0 (0x0)   execution time : 0.035 s
Press any key to continue.

```

10

```

main.cpp X
C:\Users\Pariganaika\Documents\C++\EasyGuide\bin\Debug\EasyGuide.exe
value of a: 4
value of a: 3
value of a: 2
value of a: 1
value of a: 0
Process returned 0 (0x0)   execution time : 0.045 s
Press any key to continue.

```

12

```

#include <iostream>
using namespace std;
int main () {
    int A1 = 10;
    while (A1 >= 1) {
        A1 = A1 - 1;
    }
    cout << "value of A1: " << A1 << endl;
    return 0;
}

```

මෙහි නිල් පැහැයෙන් ඇති කෝඩ් කොටස් සෑම වැඩසටහනක් සඳහාම අන්තර්ගත විය යුතු ඒවාය. එමෙන්ම ඒවා කෝඩ් ලිවීමට යොදා ගන්නා කෝඩ්ඩ්ලොක්ස් වැඩසටහන විසින් ලබා දෙන බැවින් අමුතුවෙන් ලිවිය යුතු නැත. මෙහි විස්තර කෙරෙන්නේ තමා විසින් ලිවිය යුතු කෝඩ් කොටස් පිළිබඳව පමණි. මෙම සටහන තුළ කළ පැහැයෙන් දක්වා ඇත්තේ ඒවායි.

- මෙහි කළ පැහැයෙන් ඇති පළමු පේළියේදී A1 යන නමින් වේරියබල් එකක් සකස් කර ඊට 10 යන අගය ඇතුළු කර තිබේ. දෙවැනි පේළියේදී ලූප් එක ආරම්භ කර තිබේ. එහි කන්ඩිෂන් එක ලෙස A1>=1 යන්න ලබාදී ඇත. ඒ අනුව A1 නම් වේරියබල් එකෙහි අගය 1ට වඩා විශාලව හෝ සමානව හෝ ඇති තාක් කල් මෙම ලූප් එක ක්‍රියාත්මක වෙයි. මෙම ලූප් එකෙන් ඉවත් වෙන්නේ A1 වේරියබල් එකෙහි අගය 0 හෝ ඊට වඩා අඩුවූ විටය.

- ඒ අනුව මෙහිදී ලූප් එක ක්‍රියාත්මක වෙන විට එම වේරියබල් එකෙහි දෑට ඇති අගයෙන් 1 බැගින් අඩු වෙයි. දෑට A1 වේරියබල් එකෙහි ඇති අගය 10 බැවින් ලූප් එක වට 10ක් ක්‍රියාත්මකවූ පසු A1 වේරියබල් එකෙහි ඇති අගය 0 බවට පත් වෙයි. ඉන්පසුව ලූප් එක එක් වටයක් හෝ ක්‍රියාත්මක නොවෙන නිසා, 0 යන අගය

පමණක් A1 වේරියබල් එකෙහි ඉතිරි වෙයි. ඒ අනුව මෙහි අඩුවූ එක ලූප් එකෙන් ඉවත්වූ පසුව ලබා දෙන නිසා අවසන් පිළිතුර ලෙස 0 පමණක් අඩුවූව කරනු ලැබේ. (08 රූපය)

මෙම වැඩසටහනම පහත දක්වා ඇති සේ ලියූ විට සෑම වටයකදීම අඩුවූව ලබා ගන්නට පුළුවන.

```

int A1 = 10;
while (A1 >= 1) {
    A1 = A1 - 1;
    cout << "value of A1: " << A1 << endl;
}

```

- මෙහි හතර වැනි පේළියේදී ලූප් එක තුළට යොමු කළ බැවින් ලූප් එක ක්‍රියාත්මක වෙන සෑම වටයකදීම එහි අඩුවූව එක ලැබේ. මෙහි පළමු වටයේදී A1 වේරියබල් එකෙන් 1ක් අඩු වෙන බැවින් හතර වැනි පේළිය මඟින් අඩුවූව කරනු ලබන්නේ 9 යන්නයි.

- ලූප් එක ක්‍රියාත්මක වෙන දෙවැනි වටයේදී, තුන් වැනි පේළියෙන් වේරියබල් එකෙහි ඇති අගයෙන් තවත් 1ක් අඩු කරන බැවින් හතර වැනි පේළිය මඟින් අඩුවූව කරන්නේ 8 යන්නයි. මේ ආකාරයෙන් ලූප් එක ක්‍රියාත්මක වෙන සෑම වටයකදීම A1 වේරියබල් එකෙන් 1 බැගින් අඩු කරමින් එම අගයන් අඩුවූව කරයි. (09 රූපය)

**සංඛ්‍යාවල වෙනස ලබා ගන්නා ආකාරය**

මෙම වැඩසටහන ක්‍රියාත්මක කර 1 සිට 10 දක්වා ඇති සංඛ්‍යා සියල්ලේ ඵලකාරය ලබා ගත යුත්තේය කියා සිතන්න. එවිට ලූප් එක, 1 සිට 10 දක්වා ක්‍රියාත්මක වෙන්නට ඉඩ හැර එහි දත්ත තවත් වේරියබල් එකක් වෙත එකතු කර ගත යුතුය. ලූප් එක ක්‍රියාත්මකවී අවසන් වෙනවාත් සමඟම මෙසේ එකතු කර ගන්නා වේරියබල් එකෙහි අගය අඩුවූව කළ යුතුය. මෙය හේරම් ගැනිස පිණිස පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න.

```

int a = 1;
int b = 0;

```



```

main.cpp X
1  #include <iostream>
2  using namespace std;
3  int main () {
4      int a = 0;
5      int b = 0;
6      cin >> a;
7      do{
8          b = b + a;
9          cin >> a;
10     } while (a != 0);
11     cout << b << endl;
12     return 0;
13 }
14

```

13

```

while (a <= 10){
    b = b + a;
    a = a + 1;
}
cout << "value of b: " << b << endl;

```

මෙහි පළමු පේළියේදී a නම් වේරියබල් එකට 0 යන අගයද, දෙවැනි පේළියේදී b නම් වේරියබල් එකට 0 යන අගයද ඇතුළු කර තිබේ. තුන්වැනි පේළියේදී ලූප් එක ක්‍රියාත්මක වෙන කන්ඩිෂන් එක ලෙස ලබාදී තිබෙන්නේ a වේරියබල් එකෙහි ඇති අගය 10 ට වඩා කුඩා හෝ සමාන හෝ වේය යන්නයි. ඒ අනුව එම කන්ඩිෂන් එක සත්‍යව ඇති තාක් ලූප් එක නැවත නැවත ක්‍රියාත්මක වෙයි.

ලූප් එක ක්‍රියාත්මක වෙන විට හතර වැනි පේළිය තුළ ලියා ඇති කෝඩ් මගින් b වේරියබල් එකෙහි දැනට ඇති අගයට a වේරියබල් එක එකතු කරයි. ඒ අනුව පළමු වටයේදී b වේරියබල් එකෙහි ඇති 0 යන අගයට a වේරියබල් එකෙහි ඇති 1 එකතු වී, b වේරියබල් එකෙහි ඇති අගය 1 ක් බවට පත් වේ.

මෙහි 5 වැනි පේළියෙන් a වේරියබල් එකෙහි ඇති අගයට 1ක් එකතු වෙන නිසා ඊළඟ වටය ක්‍රියාත්මක වෙන විට a වේරියබල් එකෙහි අගය තිබෙන්නේ 2ක් ලෙසය. එය හතර වැනි පේළිය මගින් නැවතත් b වේරියබල් එකට එකතු කරනු ලබන බැවින් දෙවැනි වටය ක්‍රියාත්මක වෙන විට b වේරියබල් එකෙහි අගය 3ක් ලෙස සකස් වෙයි. එතෙක් b වේරියබල් එකෙහි තිබුණු 1 යන අගයට දැන් a වේරියබල් එකෙහි ඇති 2 යන අගය එකතු වෙන බැවිනි.

මේ ආකාරයට a වේරියබල් එකෙහි ඇති අගය 10 වෙන තෙක් ලූප් එක ක්‍රියාත්මක වෙන නිසා 1 සිට 10 දක්වා ඇති සංඛ්‍යා සියල්ලේ එකතුව b නම් වේරියබල් එකට ලැබෙයි. මේ සියල්ල අවසන්වී ලූප් එකෙන් ඉවත්වූ විට b වේරියබල් එක අඩුවිටුවී කරන නිසා, එහි ඇති අවසන් අගය පමණක් පිළිතුර සේ ලැබෙයි. (10 රූපය)

#### ඩු වයිල් ලූප් එක භාවිතා කිරීම

යම් කන්ඩිෂන් එකක් පරීක්ෂා කරමින් එම කන්ඩිෂන් එක සත්‍යව ඇතිතාක් පමණක් උපදෙස් මාලාවක්, නැවත නැවත ක්‍රියාත්මක කිරීමට ඩු වයිල් ලූප් එක භාවිතා කරන්නේ, වයිල් ලූප් එකෙහිදී මෙන් නොව මෙහිදී කන්ඩිෂන් එක පරීක්ෂා කර බලනුයේ ලූප් එක අවසාන යේදීය. මෙහිසා කන්ඩිෂන් එක පරීක්ෂා කරන්නට කලින් එක් වරක් ලූප් එකෙන් විය යුතු කාර්ය කරනු ලැබේ. මෙම ඩු වයිල් ලූප් එකෙහි මෙහි කෝඩ් ස්ට්‍රක්චර් එක මෙසේය.

#### ඩු වයිල් ලූප් එකෙහි පොදු ආකෘතිය

```

do
{
    විය යුතු කාර්ය හෝ කාර්යන්;
}while(කන්ඩිෂන් එක);

```

මෙම කෝඩ් ආකෘතියට අනුව කෝඩ් ලියන ආකාරය අවබෝධ

```

main.cpp X
C:\Users\Parigana\Documents\EasyGuide\bin\Debug\EasyGuide.exe
8
10
12
0
30

Process returned 0 (0x0)   execution time : 8.222 s
Press any key to continue.

```

14

කර ගැනීම පිණිස පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න. (11 රූපය)

```

#include <iostream>
using namespace std;
int main () {
    int a = 5;
    do {
        a = a - 1;
        cout << "value of a: " << a << endl;
    } while (a >= 1);
    return 0;
}

```

මෙහි නිල් පැහැයෙන් ඇති කෝඩ් කොටස් සෑම වැඩසටහනක් සඳහාම අන්තර්ගත විය යුතු ඒවාය. එමෙන්ම ඒවා කෝඩ් ලිවීමට යොදා ගන්නා කෝඩ් බ්ලොක්ස් වැඩසටහන විසින් ලබා දෙන බැවින් අමුතුවෙන් ලිවිය යුතු නැත. මෙහි විස්තර කෙරෙන්නේ තමා විසින් ලිවිය යුතු කෝඩ් කොටස් පිළිබඳව පමණි. මෙම සටහන තුළ කළ කළ පැහැයෙන් දක්වා ඇත්තේ ඒවායි.

මෙහි කළ පැහැයෙන් ඇති පළමු පේළියේදී a යන නමින් වේරියබල් එකක් සකස් කර ඊට 5 යන අගය ඇතුළු කර තිබේ. දෙවැනි පේළියේදී do යන්න යොදා ලූප් එක ආරම්භ කර තිබේ. ඒ අනුව මෙහිදී ලූප් එක ආරම්භ කරන විට කිසිදු කන්ඩිෂන් එකක් දක්වන්නේ නැත.

මෙම ලූප් එක ක්‍රියාත්මක වෙන විට තුන් වැනි පේළිය මගින් a වේරියබල් එකෙහි දැනට ඇති අගයෙන් 1 බැගින් අඩු කරනු ලැබේ. ඉන් පසුව 4 වැනි පේළියෙන් එම a වේරියබල් එකෙහි ඇති අගය අඩුවිටුවී වේ.

මෙහි 5 වැනි පේළිය ලියා තිබෙන්නේ කන්ඩිෂන් එක පරීක්ෂා කිරීමටයි. එහිදී කන්ඩිෂන් එක ලෙස ලබාදී ඇත්තේ a >= 1 යන්නයි. ඒ අනුව a නම් වේරියබල් එකෙහි අගය 1 ට වඩා විශාලව හෝ සමානව ඇති තාක් කල් මෙම ලූප් එක ක්‍රියාත්මක වෙයි. දැනට a 1 වේරියබල් එකෙහි ඇති අගය 5 වෙන බැවින්, මෙහිදී ලූප් එක ක්‍රියාත්මක වෙයි. ඒ අනුව a වේරියබල් එකෙහි අගය 1 ට වඩා අඩුවූ විට ලූප් එකෙන් ඉවත් වෙයි.

මෙම ලූප් එක ක්‍රියාත්මක වෙන විට a වේරියබල් එකෙහි දැනට ඇති අගයෙන් 1 බැගින් අඩු කරමින් a වේරියබල් එකෙහි අගය අඩුවිටුවී කරන බැවින් මෙහි අඩුවිටුවී ලෙස 4 සිට 0 දක්වා ඇති අගයන් පෙන්වයි. (12 රූපය)

#### ඇතුළු කරනු ලබන යම් සංඛ්‍යා කීපයක එකතුව ලබා ගැනීම

ඉන්පුව කරනු ලබන අප විසින් නොදන්නා සංඛ්‍යා කීපයක එකතුව ලබා ගැනීමට ලූප් එකක් යොදා ගන්නට පුළුවන. මෙහිදී පරිශීලකයා හෙවත් යුසර් විසින් යම් කිසි විශේෂ දත්තයක් ඇතුළු කරන තෙක්,

ඇතුළු කරනු ලබන සංඛ්‍යා සියල්ලේ ඵලදායී ලබා ගනී. මෙය හේරුම් ගැනීම පිණිස පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න. (13 රූපය)

```
int a = 0;
int b = 0;
cin >> a;
do{
    b = b + a;
    cin >> a;
} while (a != 0);
cout << b << endl;
```

මෙහිදී පළමු පේළි දෙකෙන් a සහ b ලෙස වේරියබල් දෙකක් සකස් කර ඒවාට 0 ඇතුළු කර තිබේ. මෙහි ඇති තුන් වැනි පේළියෙන් පරිශීලක ආදානයක් හෙවත් යුසර් ඉන්පුට් එකක් a වේරියබල් එක වෙත ලබා ගනී. ඒ අනුව මෙම කෝඩ් ක්‍රියාත්මක වෙත විට තුන් වැනි පේළියේදී යුසර් ඉන්පුට් එකක් බලාපොරොත්තු වේ. එය ලබාදුන් පසුව 4 වැනි පේළියේදී ලුප් එක තුළට යොමු වේ.

ඉන්පසුව 5 වැනි පේළිය මගින් පරිශීලකයා විසින් ඉන්පුට් කර a වේරියබල් එක තුළ දී තිබෙන දත්තය b වේරියබල් එක වෙත එකතු කරනු ලැබේ. ඒ අනුව පරිශීලකයා 10 යන අගය ලබාදී තිබුණේ නම් එම අගය b වෙත ඇතුළු වෙයි.

මෙහි ඇති 6 වැනි පේළිය මගින් නැවතත් යුසර් ඉන්පුට් එකක් ලබා ගනී. එම ඉන්පුට් එක ලුප් එක තුළදී ලබා ගන්නා බැවින් ලුප් එක ක්‍රියාත්මක වෙත සෑම විටයකදීම 6 වැනි පේළිය මගින් එම ඉන්පුට් එක ලබා ගනී.

මෙහි ඇති 7 වැනි පේළියේදී ලුප් එකට අදාළව කන්ඩිෂන් එක පරීක්ෂා කරයි. ඒ අනුව පරීක්ෂා කරන්නේ පරිශීලකයා විසින් ලබාදී තිබෙන අගය 0 ට අසමානද යන්නයි. ඒ අනුව මෙම ලුප් එක පරිශීලකයා 0 ලබා දෙන තෙක් ක්‍රියාත්මකවී, 0 ලබාදුන් පසුව නතර වෙයි. එතෙක් පරිශීලකයා විසින් ලබා දෙන ලබන අගය a වේරියබල් එක වෙත ලබා ගෙන එය b තුළට එකතු කරමින් යුසර් ඉන්පුට් සියල්ල එකතු කරනු ලබයි. ඒ සියල්ල අවුට්පුට් කරනු ලබන්නේ ලුප් එකෙන් ඉවත්වූ පසු ක්‍රියාත්මක වෙත අවසන් පේළිය මගිනි. (14 රූපය)

## හය වැනි පරිච්ඡේදය

### නෙස්ටඩ් කන්ට්‍රෝල් ස්ට්‍රක්චර්ස් භාවිතා කරන ආකාරය

කන්ඩිෂනල් ලොජික් සහ ලුප්ස් එකිනෙකට වෙනස් කෝඩ් ලිවීමට යොදා ගන්නා කන්ඩිෂනල් ස්ට්‍රක්චර්ස් දෙකක් වුවද, යම් යම් වැඩසටහන්වල ඇති අවශ්‍යතාවලට ගැළපෙන සේ කෝඩ් ලිවීම පිණිස කන්ඩිෂනල් ලොජික් එකක් තුළ ලුප් එකක් යෙදීමට හෝ ලුප් එකක් තුළ කන්ඩිෂනල් ලොජික් එකක් යෙදීමට හෝ පුළුල් වන. මෙවැනි කෝඩ් ලියන සැලැස්මක් නෙස්ටඩ් කන්ට්‍රෝල් ස්ට්‍රක්චර්ස් එකක් (nested control structures) ලෙස හැඳින්වේ.

#### කන්ඩිෂනල් ලොජික් එකක් තුළදී තවත් කන්ඩිෂනල්

##### ලොජික් එකක් භාවිතා කිරීම

මෙසේ කන්ඩිෂනල් ලොජික් එකක් තුළට තවත් කන්ඩිෂනල් ලොජික් එකක් යොදා කෝඩ් ලියා ඇති කෝඩ් සටහනක් පහත දක්වා තිබේ. මෙම ක්‍රමය හේරුම් ගැනීම පිණිස එය අධ්‍යයනය කරන්න.

(01 රූපය)

```
#include <iostream>
using namespace std;
int main () {
    int a = 100;
    int b = 200;
    if( a == 100 ) {
        if( b == 200 ) {
            cout << "a = 100 and b = 200" << endl;
        }
    }
    cout << "Exact value of a is : " << a << endl;
    cout << "Exact value of b is : " << b << endl;
    return 0;
}
```

මෙහි නිල් පැහැයෙන් ඇති කෝඩ් කොටස් සෑම වැඩසටහනක් සඳහාම අන්තර්ගත විය යුතු ඒවාය. එමෙන්ම ඒවා කෝඩ් ලිවීමට

යොදා ගන්නා කෝඩ් බලාත්මක වැඩසටහන විසින් ලබා දෙන බැවින් අමුතුවෙන් ලිවිය යුතු නැත. මෙහි විස්තර කෙරෙන්නේ තමා විසින් ලිවිය යුතු කෝඩ් කොටස් පිළිබඳව පමණි. මෙම සටහන තුළ කළ පැහැයෙන් දක්වා ඇත්තේ ඒවායි.

මෙහි කළ පැහැයෙන් ඇති පළමු පේළි දෙක තුළදී පිළිවෙලින් a සහ b ලෙස වේරියබල් දෙකක් සකස් කර, ඒවාට 100 සහ 200 යන අගයන් ඇතුළු කර තිබේ. මෙහි තුන් වැනි පේළියේදී if කන්ඩිෂනල් ලොජික් එක මගින් a වේරියබල් එක තුළ ඇති අගය 100 දැයි පරීක්ෂා කර බලයි. එය එසේ නම් පමණක් මෙහි 4 වැනි පේළියේ ඇති දෙවැනි if කන්ඩිෂනල් ලොජික් එකෙහි කන්ඩිෂන් එක පරීක්ෂා කර බලනු ලැබේ. මෙහිදී පරීක්ෂා කර බලන්නේ b වේරියබල් එකෙහි ඇති අගය 200 ද යන්නයි. එය එසේ නම් පමණක් 5 වැනි පේළිය ක්‍රියාත්මකවී a = 100 and b = 200 යන්න අවුට්පුට් වේ.

මෙහි 6 වැනි පේළියේ ඇති වැසුණු සහල වරහන මගින් දෙවැනි if එක අවසන් වෙයි. පළමු if කන්ඩිෂනල් ලොජික් එක අවසන් වෙන්නේ 7 වැනි පේළියෙනි. මේ අනුව 8 සහ 9 යන පේළි ක්‍රියාත්මක වෙන්නේ කන්ඩිෂනල් ලොජික් දෙකම අවසන්වූ පසුවය. ඒ අනුව එහිදී කන්ඩිෂනල් ලොජික් දෙකට අදාළ කන්ඩිෂන් සහය වුවද, අසහය වුවද එම පේළි දෙක ක්‍රියාත්මක වෙයි.

මේ අනුව වැඩසටහන ක්‍රියාත්මක වෙත විට දෙවැනි කන්ඩිෂනල් ලොජික් එක තුළ ඇති කෝඩ් ක්‍රියාත්මක වෙන්නේ පළමු වැනි කන්ඩිෂනල් ලොජික් එක සහ දෙවැනි කන්ඩිෂනල් ලොජික් එක යන දෙකෙහිම ස්ටේට්මන්ට් සහය නම් පමණි. ඉන් එක් ස්ටේට්මන්ට් එකක් හෝ අසහය නම් දෙවැනි කන්ඩිෂනල් ලොජික් එක තුළ ඇති කෝඩ් ක්‍රියාත්මක වෙන්නේ නැත. මෙහිදී දී ඇති ස්ටේට්මන්ට්වලට අනුව එම කන්ඩිෂනල් ලොජික් දෙකම සහය වෙත බැවින් දෙවැනි කන්ඩිෂනල් ලොජික් එක තුළ ඇති කෝඩ් කොටස් ක්‍රියාත්මක වෙයි. (02 රූපය)



```

1 #include <iostream>
2 using namespace std;
3 int main () {
4     int a = 100;
5     int b = 200;
6     if( a == 100 ) {
7         if( b == 200 ) {
8             cout << "a = 100 and b = 200" << endl;
9         }
10    }
11    cout << "Exact value of a is : " << a << endl;
12    cout << "Exact value of b is : " << b << endl;
13    return 0;
14 }
15

```

01

```

1 #include <iostream>
2 using namespace std;
3 int main () {
4     int i = 0;
5     while(i < 3)
6     {
7         int j = 0;
8         while(j < 5)
9         {
10            cout << "i = " << i << " and j = " << j << endl;
11            j++;
12        }
13        i++;
14    }
15    return 0;

```

03

```

a = 100 and b = 200
Exact value of a is : 100
Exact value of b is : 200

Process returned 0 (0x0)   execution time : 0.021 s
Press any key to continue.

```

02

```

i = 0 and j = 0
i = 0 and j = 1
i = 0 and j = 2
i = 0 and j = 3
i = 0 and j = 4
i = 1 and j = 0
i = 1 and j = 1
i = 1 and j = 2
i = 1 and j = 3
i = 1 and j = 4
i = 2 and j = 0
i = 2 and j = 1

```

04

### වයිල් ලූප් එකක් තුළ තවත් වයිල් ලූප් එකක් භාවිතා කිරීම

කලින් විස්තර කළ ආකාරයට ෆෝ ලූප් එක වෙනුවට වයිල් ලූප් එක හෝ ඩූ වයිල් ලූප් එක හෝ යොදා ඒ තුළ තවත් ලූප් එකක් ක්‍රියාත්මක කරවන්නට පුළුවන. මෙය තේරුම් ගැනීම පිණිස පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න. (03 රූපය)

```

int i = 0;
while(i < 3)
{
    int j = 0;
    while(j < 5)
    {
        cout << "i = " << i << " and j = " << j << endl;
        j++;
    }
    i++;
}

```

- මෙහි පළමු පේළියේදී i යන නමින් වේරියබල් එකක් සකස් කර

තිබේ. දෙවැනි පේළියේදී වයිල් ලූප් එක මගින් පරීක්ෂා කර බලන්නේ එම i වේරියබල් එකෙහි අගය 3 ට වඩා කුඩාද යන්නයි. එය එසේ නම් එම ලූප් එක තුළට ඇතුළුවී 4 වැනි පේළිය මගින් j නම් වේරියබල් එක සකස් කර ඊට 0 යන අගය ඇතුළු කර තිබේ.

- ඉන්පසුව 5 වැනි පේළිය මගින් ආරම්භ වෙන ලූප් එක ක්‍රියාත්මක වෙන්නේ j වේරියබල් එකෙහි තිබෙන අගය 5 ට වඩා අඩුව තිබෙන තාක්ය. මේ අනුව 7 වැනි පේළිය මගින් i වේරියබල් එකෙහි තිබෙන අගය සහ j වේරියබල් එකෙහි තිබෙන අගය අඩුවිටුම් කරනු ලැබේ.

මෙහි 6 වැනි පේළිය මගින් j වේරියබල් එකෙහි දෘට ඇති අගයට 1ක් එකතු කරනු ලැබේ. ඉන්පසුව 9 වැනි පේළියෙන් දෙවැනි ලූප් එක අවසන් වෙයි. මෙහි 10 වැනි පේළියේදී i වේරියබල් එකෙහි අගයට 1ක් එකතු කරනු ලැබේ. එසේ කරන්නේ පළමු ලූප් එක වට තුනක් ක්‍රියාත්මකවීමෙන් පසුව එය අවසන් විය යුතු නිසාය. මෙහි අවසන් පේළියෙන් පළමු ලූප් එකද අවසන් වෙයි. මේ අනුව i වේරියබල් එකෙහි අගය 0,1 සහ 2 ලෙස තිබෙන විට දෙවැනි ලූප් එක ක්‍රියාත්මක වෙයි. (04 රූපය)

### නන් වැනි පරිච්ඡේදය

#### ඇරේස් භාවිතා කරන ආකාරය

රිගණක වැඩසටහන් සකසන විට මතකයේ තාවකාලිකව දත්ත රැඳවා තැබීමට වේරියබල් උපයෝගී කර ගන්නා බව දැනිමු. ඒවායේ ඇති දත්ත ඕනෑම විටක වෙනස් කරන්නට පුළුවන. එහි විවිධ දත්ත වැඩි ප්‍රමාණයක් තාවකාලික මතකයේ රැඳවිය යුත්තේ

නම් වේරියබල් වැඩි ගණනක් සැකසිය යුතු බවත් දැනිමු.

එහෙත් වේරියබල් ගණනාවක් අවශ්‍ය නැතකට, වේරියබල් භාවිතා කිරීම වෙනුවට ඇරේ (array) භාවිතා කළ හැක. මෙහිදී වේරියබල්ස් බොහෝ ගණනක් සැකසීම වෙනුවට සැකසිය යුතු වෙන්නේ එක්

```

1 #include <iostream>
2 using namespace std;
3 int main ()
4 {
5     int intPariganaka[3];
6     intPariganaka[0] = 10;
7     intPariganaka[1] = 20;
8     intPariganaka[2] = 30;
9     int intT = intPariganaka[0] + intPariganaka[1];
10    cout << intT;
11    return 0;
12 }
13

```

01

```

1 #include <iostream>
2 using namespace std;
3 int main ()
4 {
5     int numbers[5], sum = 0;
6     cout << "Enter 5 numbers: ";
7     for (int i = 0; i < 5; ++i)
8     {
9         cin >> numbers[i];
10        sum += numbers[i];
11    }
12    cout << "Sum = " << sum << endl;
13    return 0;
14 }

```

03

```

60
Process returned 0 (0x0)   execution time : 0.025 s
Press any key to continue.

```

02

```

Enter 5 numbers: 10
20
30
40
50
Sum = 150

Process returned 0 (0x0)   execution time : 19.958 s
Press any key to continue.

```

04

ඇරේ එකක් පමණක් නිසා පහසුවෙන් කෝඩ් ලිවීමට පුළුවන. එමෙන්ම ඇතුළු කෝඩ්වලදී වේරියබල්ස් නොව ඇරේස්ම සැකසිය යුතු වේ.

මෙහි විස්තර කරන්නේ ඇරේස් සැකසීම සහ කෝඩ් සටහනක් තුළ ඒවා භාවිතා කරන ආකාරය ගැනයි. මෙවැනි ඇරේ එකක පොදු ආකෘතිය **[වේරියබල් එක][ඇරේ එකෙහි නම][කොටස් ගණන]**; ලෙස වෙයි.

ඇරේ එකක් සැකසිය යුත්තේද වේරියබල් එකක් සකස්න්නාක් මෙනි. එහෙත් වෙනස වන්නේ ඇරේ එකකදී එහි ඇති කොටස් ගණන හෙවත් එලිමන්ට් (elements) ගණන දැක්විය යුතුවීමයි. එම එලිමන්ට් ගණන ඇරේ එකට ඇතුළු කරනු ලබන වෙනස් දත්ත ප්‍රමාණය මත වෙනස් කර ගත යුතුය. ඒ අනුව මෙහිදී තනි එලිමන්ට් එකක් තනි වේරියබල් එකක් සේ භාවිතා කරන්නට පුළුවන.

ඇරේ එක සඳහා ලබා දෙන ඩේටා ටයිප් එක අනුව ඒ තුළට ඇතුළු කරනු ලබන දත්ත මොනවාද යන්න තීරණය වෙයි. **int intData [5];** යනු එසේ සකස් කර ඇති ඇරේ එකකි. මෙහි int යනු ඇරේ එකෙහි දත්ත වර්ගයයි. intData යනු, ඇරේ එක සඳහා ලබා දී තිබෙන නමයි. මෙම ඇරේ එකෙහි එලිමන්ට් ගණන ලෙස ලබාදී ඇත්තේ 5 යන්නයි. මේ අනුව 0 සිට 4 දක්වා එලිමන්ට් 5 ක් සහිත ඇරේ එකක් මෙහිදී නිර්මාණය වෙයි. මෙසේ සකස් කරන ඇරේ එක සඳහා එකිනෙකට වෙනස් දත්ත පහක් ඇතුළු කළ හැකිය. මෙහිදී ඇරේ එක සකස් වෙන ආකාරය පහත දක්වා ඇති පරිදි ආදර්ශය කළ හැක.

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |

වම් පස සිට දකුණු පසට යොදන අංකවලින් මෙම එලිමන්ට් හැඳින්වේ. එම අංක යොදන්නේ 0 සිට ය. මේ අනුව මෙහි අංක 1 න් හැඳින්වෙන්නේ වම් පස සිට දෙවැනියට ඇති එලිමන්ට් එකයි. මෙසේ එකිනෙකට වෙනස් දත්ත 10 ක් ඇරේ එක සඳහා ඇතුළු කිරීමට අවශ්‍ය නම් මෙම ඇරේ එක **int intData [9];** යන ආකාරයට සැකසිය යුතුය. එවිට 0 සිට 9 දක්වා එලිමන්ට් 10කින් යුත් ඇරේ එකක් නිර්මාණය කර ගත හැකි වෙයි. මෙවැනි ඇරේ එකක් තුළ ඇති එක්

එලිමන්ට් එකක් තනි වේරියබල් එකක් සේ භාවිතා කිරීමට පුළුවන.

#### ඇරේ එකක් සඳහා දත්ත ඇතුළු කිරීම

ඇරේ එකක් සඳහා දත්ත ඇතුළු කරන විට අනුගමනය කළ යුතු පොදු ආකෘතිය **නම[එලිමන්ට් එක]= දත්තය;** ලෙස වෙයි. මේ අනුව ඉහත සකස් කළ ඇරේ එකෙහි එක් කොටසක් සඳහා දත්ත ඇතුළු කිරීම **intData [0] = 75;** ලෙස ලිවිය හැක. මෙහිදී intData නම් ඇරේ එකෙහි 0 වැනි එලිමන්ට් එක සඳහා 75 යන සංඛ්‍යාව ඇතුළු කර ඇත.

මෙවැනි ඇරේ එකක් භාවිතා කරන ආකාරය තේරුම් ගැනීම සඳහා ඇරේ එකක් යොදා සංඛ්‍යා කීපයක් එකතු කළ හැකි වැඩසටහනක කෝඩ් පහත දක්වා තිබේ. (01 රූපය)

මෙහි නිල් පැහැයෙන් ඇති කෝඩ් කොටස් සෑම වැඩසටහනක් සඳහාම අන්තර්ගත විය යුතු ඒවාය. එමෙන්ම ඒවා කෝඩ් ලිවීමට යොදා ගන්නා කෝඩ්බ්ලොක්ස් වැඩසටහන විසින් ලබා දෙන බැවින් අමුතුවෙන් ලිවිය යුතු නැත. මෙහි විස්තර කෙරෙන්නේ තමා විසින් ලිවිය යුතු කෝඩ් කොටස් පිළිබඳව පමණි. මෙම සටහන තුළ කළ පැහැයෙන් දක්වා ඇත්තේ ඒවායි. කොළ පැහැති කෝඩ් තනි පෙළට ලිවිය යුතුය.

```

#include <iostream>
using namespace std;
int main ()
{
    int intPariganaka[3];
    intPariganaka[0] = 10;
    intPariganaka[1] = 20;
    intPariganaka[2] = 30;
    int intT = intPariganaka[0] +
    intPariganaka[1] + intPariganaka[2];
    cout << intT;
    return 0;
}

```



● මෙහි කළ පැහැයෙන් ඇති පළමු පේළියේදී intPariganaka යන නමින් ඇරේ එකක් සකස් කර ඇත. එහිදී කොටස් ගණන සඳහා 3 යන්න ලබාදී ඇති බැවින් 0 සිට 2 දක්වා කොටස් තුනක් සහිත ඇරේ එකක් නිර්මාණය වේ.

● මෙහි දෙවැනි පේළියේදී intPariganaka ඇරේ එකෙහි 0 වැනි කොටස තුළට අංක 10 ඇතුළු කර ඇත. මේ ආකාරයෙන්ම තුන් සහ 4 යන පේළිවලදී intPariganaka ඇරේ එකෙහි 1,2 යන කොටස් සඳහා පිළිවෙළින් 20 සහ 30 යන අගයන් ඇතුළු කර තිබේ.

● මෙහි 5 වැනි පේළියේ ඇති කෝඩ් මගින් ඇරේ එකෙහි කොටස් තුළ ඇති අගයන් එකතු කොට, ලැබෙන පිළිතුර intT යන නමින් සකස් කරන වේරියබල් තුළ ගබඩා කරනු ලැබේ. අවසාන පේළියේදී එම වේරියබල් තුළ ගබඩා කළ දත්ත අවුච්ඡුටි කරනු ලැබේ. (02 රූපය)

#### ඇරේ එකක් නිර්මාණය කිරීමේදී දත්ත ඇතුළු කිරීම

වේරියබල් එකක් නිර්මාණය කරන විට වේරියබල් වෙත දත්ත ඇතුළු කරන ආකාරයට ඇරේ එකක් නිර්මාණය කරන විටද අදාළ ඇරේ එක වෙත දත්ත ඇතුළු කරන්නට පුළුවන. මෙය තේරුම් ගැනීම සඳහා int a [5] = { 16, 2, 77, 40, 12071 }; යන කෝඩ් විමසා බලන්න. මෙහිදී එළිමන්ටි පහක් ඇති ඇරේ එකක් සකස් කර තිබේ. එමෙන්ම ඉදිරියෙන් ඇති අගයන් පහ පිළිවෙළින් එම ඇරේ එක තුළට ඇතුළු කර තිබේ. මෙහිදී ඇරේ එක සකස් වෙන ආකාරය පහත දක්වා ඇති පරිදි ආදර්ශනය කළ හැක.

| 0  | 1 | 2  | 3  | 4     |
|----|---|----|----|-------|
| 16 | 2 | 77 | 40 | 12071 |

මෙම ඇරේ එක තුළ ඇති එළිමන්ටි මුලින් විස්තර කළ ආකාරයටම භාවිතා කරන්නට පුළුවන. එමෙන්ම එළිමන්ටි එකක් තුළ ඇති දත්තයක් වෙනුවට වෙනත් දත්තයක් ඇතුළු කිරීමද කලින් කළ විදියටම කළ හැක.

#### ලුප් එකක් හරහා ඇරේ එකක් වෙත දත්ත ඇතුළු කිරීම

ඇරේ එකෙහි එක් එක් එළිමන්ටි වෙත දත්ත ඇතුළු කිරීම ලුප් එකක් තුළදී වුවද කළ හැක. පහත දක්වා ඇත්තේ මෙසේ ලුප් එකක් භාවිතා කරමින් ඇරේ එකක් තුළට දත්ත ඇතුළු කර තිබෙන කෝඩ් එකකි. මෙම කෝඩ් යොදා ගෙන ඇත්තේ පරිශීලකයෙක් ඉන්පුට් කරන අගය පහක් ඇරේ එකක් වෙත ලබා ගෙන එම අගය සියල්ල එකතු

කළ විට ලැබෙන පිළිතුර අවුච්ඡුටි කිරීමටය. (03 රූපය)

```
int numbers[5], sum = 0;
cout << "Enter 5 numbers: ";
for (int i = 0; i < 5; ++i)
{
    cin >> numbers[i];
    sum += numbers[i];
}
cout << "Sum = " << sum << endl;
```

● මෙම කෝඩ් සටහනේ ඇති පළමු පේළියේදී numbers යන නමින් int වර්ගයේ ඇරේ එකක් සකස් කර තිබේ. එම ඇරේ එකෙහි එළිමන්ටි ගණන ලෙස ලබාදී තිබෙන්නේ 5 ය. ඊට අමතරව එම පේළියේදීම sum වේරියබල් සකස් කර ඒ සඳහා 0 යන අගයද ඇතුළු කර තිබේ.

● මෙහි දෙවැනි පේළිය ලියා තිබෙන්නේ Enter 5 numbers: යන්න අවුච්ඡුටි කිරීමටය. පසුව මෙහි තුන් වැනි පේළියෙන් ලුප් එක ආරම්භ වෙයි. එම ලුප් එක ක්‍රියාත්මක වෙන්නේ i වේරියබල් එකෙහි ඇති අගය 0 සිට 5 ට වඩා කුඩාව ඇති තෙක්ය. එනම් 0 සිට 4 දක්වා වට 5ක් එම ලුප් එක ක්‍රියාත්මක වෙයි.

● ලුප් එක ක්‍රියාත්මක වෙන විට 5 වැනි පේළියෙන්, පරිශීලකයා විසින් ලබා දෙනු ලබන ඉන්පුට් එක numbers ඇරේ එකෙහි දූතට සිලෙක්ට්වී ඇති එළිමන්ටි එක වෙත ලබා ගනී. ඒ අනුව ලුප් එක ක්‍රියාත්මක වෙන පළමු වටයේදී i වේරියබල් එකෙහි අගය 0 බැවින්, පස් වැනි පේළියේ ඇති numbers[i] යන කෝඩ් කොටසින් numbers ඇරේ එකෙහි 0 වැනි එළිමන්ටි එක වෙත පරිශීලකයා විසින් ලබා දෙනු ලබන ඉන්පුට් එක ලබා ගනී.

මේ ආකාරයට ලුප් එක ක්‍රියාත්මක වෙන අනෙක් වට 4 තුළදී පරිශීලකයා විසින් ලබා දෙනු ලබන ඉන්පුට් සියල්ල අනෙක් එළිමන්ටි 4 වෙතද ලබා ගනී. මෙහි ඇති 6 වැනි පේළිය ලියා තිබෙන්නේ numbers ඇරේ එකෙහි දූතට සිලෙක්ට්වී ඇති එළිමන්ටි එකෙහි ඇති සංඛ්‍යාව sum වේරියබල් වෙත එකතු කිරීමටය. එයද කලින් කෝඩ් පේළිය මෙන් එක් එක් වටයේදී ක්‍රියාත්මක වෙයි. මෙහි ඇති අවසන් පේළිය ලියා තිබෙන්නේ sum වේරියබල් වෙත එකතු කර ගත් ඇරේ එකෙහි එක් එක් එළිමන්ටිවල ඇති අගයන්ගේ ඵලය අවුච්ඡුටි කිරීමටය. (04 රූපය)

## අව වැනි පරිච්ඡේදය

### ෆන්ක්ෂන් නිර්මාණය කරන ආකාරය

යම් කෝඩ් ආකෘතියක් සැකසීම පිණිස පරිගණක භාෂා තුළදී ශ්‍රිත හෙවත් ෆන්ක්ෂන් (function) භාවිතා කරයි. මෙවිට එම ආකෘතිය අවශ්‍ය තැන්වලදී නැවත නැවත ක්‍රියාත්මක කර ගැනීමට හැකි නිසා වැඩසටහනක් ලිවීම සඳහා අවශ්‍ය වෙන කෝඩ් ගණන අඩු වේ. එමෙන්ම පරිගණක වැඩසටහනක් කොටස්වලට බෙදා වෙන් කිරීම මෙමගින් කරනු ලැබේ. එයින් අදාළ වැඩසටහන ලිවීම මෙන්ම

වටහා ගැනීමද පහසු වෙනු ඇත.

මෙසේ C++ කෝඩ් ලියන විට කලින් සකස් කර ඇති ෆන්ක්ෂන් භාවිතා කරයි. මේවා අතරින් නිතරම භාවිතා කරන ෆන්ක්ෂන් එක, සෑම කෝඩ් සටහනකම ඉහළින් ලියනු ලබන main() යන්නයි. මේ අනුව සෑම කෝඩ් සටහනකම ලියන්නේ එම මූලික ෆන්ක්ෂන් එක තුළය. එහිදී එම ෆන්ක්ෂන් එකෙහි නම main යන්නයි. එය එම නමින්

```

1 #include <iostream>
2 using namespace std;
3 void print();
4 int main ()
5 {
6     print();
7 }
8 void print()
9 {
10     cout << "Wijeya Pariganaka" << endl;
11 }
12

```

01

සකස් කර ඇත්තේ පරිගණක වැඩසටහනේ මූලිකම ක්‍රියාත්මක වෙන ගන්නා ලද එක එය නිසාය.

මේ ආකාරයෙන් මූලිකව තිබෙන ගන්නා ලද අවසරය කෝඩ් තුළ ක්‍රියාත්මක කළ හැකි ගන්නා ලද අවසරය ආකාරයට නිර්මාණය කර ගන්නටද පුළුවන. ගන්නා ලද එකක් සකසන ආකාරය අවබෝධ කර ගැනීම සඳහා පහත දක්වා ඇති කෝඩ් අධ්‍යයනය කරන්න. (01 රූපය)

```

#include <iostream>
using namespace std;
void print();
int main ()
{
    print();
}
void print()
{
    cout << "Wijeya Pariganaka" << endl;
}

```

● මෙහි තුන් වැනි පේළියේදී ගන්නා ලද එක ඩික්ලේරා (declare) කිරීම හෙවත් ප්‍රකාශ කර තිබේ. ගන්නා ලද එකක සකසන විට මූලිකම කරන්නේ මේ ආකාරයෙන් ගන්නා ලද එක ඩික්ලේරා කිරීමය. මෙහිදී print යනු ගන්නා ලද එක සඳහා ලබාදී ඇති නමයි.

● මෙහි නම ලෙස ඕනෑම විස්තරයක් ලබා දෙන්නට පුළුවන. එහෙත් එසේ ලබා දෙන නමක කොටස් දෙකක් වෙන් කිරීම සඳහා හිස්තැන් තැබීම නොකළ යුතුය. එමෙන්ම ගන්නා ලද එකක නමක් සඳහා ඉලක්කම් භාවිතා කළ හැකි නමුත් නම ආරම්භ විය යුත්තේ අකුරකින්ම පමණි.

● ගන්නා ලද එකක නම යොදන විට කෙටි ඉරි ලකුණ ( ) හැර වෙනත් සංකේත භාවිතා නොකළ යුතුය. මේ ආකාරයෙන් දෙනු ලබන ගන්නා ලද එකක නමක් එය ක්‍රියාත්මක කරන විට භාවිතා කළ යුතුය. මේ නිසා ගන්නා ලද එකක් සඳහා නමක් ලබා දෙන විට අදාළ ගන්නා ලද එකෙහි අන්තර්ගතය හඳුනා ගත හැකි වෙන ආකාරයෙන් නමක් ලබාදී එය භාවිතා කිරීම පහසු වේ.

● ගන්නා ලද එකෙහි නමට ඉදිරියෙන් වරහන් යොදන්නේ පරාමිතින් හෙවත් පැරාමිටර්ස් (parameters) දක්වීමටය. මූලික වශයෙන් මෙම වරහන් තුළ පැරාමිටර්ස් දක්වා නැති වුවද අවශ්‍ය කාර්ය අනුව ඒවා යොදන ආකාරය පසුව විස්තර කෙරේ.

● මේ ආකාරයට තුන් වැනි පේළියේදී ඩික්ලේරා කළ ගන්නා ලද එක ඩිෆයින් කිරීම (define) හෙවත් අර්ථ දැක්වීම ඊළඟට කළ යුතුය. ඉහත කෝඩ් සටහනේ එම කාර්ය කර තිබෙන්නේ 8 වැනි පේළියේදීය. ගන්නා ලද එක තුළ විය යුතු කාර්ය ලබා දෙන්නේ මෙලෙස එය ඩිෆයින් කරන විටය. එහිදී 9 වැනි පේළියේ විවෘත කර ඇති සහල

```

Wijeya Pariganaka

Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.

```

02

වරහන් සහ 11 වැනි පේළියේ වසා ඇති සහල වරහන් අතර ඇති කොටස අයත් වෙන්නේ ගන්නා ලද එකටය. ඒ අනුව මෙහිදී 10 වැනි පේළියේ දී Wijeya Pariganaka යන්න අඩුවිටුම් කිරීමට අදාළ කෝඩ් ලියා ඇත. එය ගන්නා ලද එක තුළ ඇති කෝඩ් කොටසයි.

● මෙලෙස ගන්නා ලද එකක් ගොඩනැගීමෙන් පසුව එය ක්‍රියාත්මක කරන තුරු අක්‍රියව පවතී. එබැවින් අවශ්‍ය විටෙක ගන්නා ලද එක ක්‍රියාත්මක කරවා ගැනීම හෙවත් කෝල් (call) කිරීම කළ යුතුය. ඉහත කෝඩ් සටහනේ ගන්නා ලද එක කෝල් කර තිබෙන්නේ 6 වැනි පේළියේදීය. එහිදී print() යන්න දක්වා තිබෙන නිසා එම නමින් ඇති ගන්නා ලද එක ක්‍රියාත්මක වෙයි. ඒ අනුව මෙහිදී Wijeya Pariganaka යන්න අඩුවිටුම් වෙයි. (02 රූපය)

● ඉහත කෝඩ් සටහනේ C++ තුළ භාවිතා වෙන ප්‍රධාන ගන්නා ලද එක වෙන main යන්න භාවිතයට කලින් print ගන්නා ලද එක ඩික්ලේරා කර ඇති ආකාරයත් main ගන්නා ලද එකෙහි පිටතදී print ගන්නා ලද එක ඩිෆයින් කර ඇති ආකාරයත් දැක ගන්නට පුළුවන. ඒ අනුව print ගන්නා ලද එක කෝල් කිරීම පමණක් main ගන්නා ලද එක තුළදී කර තිබේ.

#### ගන්නා ලද එකක අන්තර්ගතය

සෑම ගන්නා ලද එකකම කොටස් තුනකින් යුක්ත වේ. මෙම කොටස් තුන විටත් ටයිප් (return type) ගන්නා ලද එකෙහි නම හෙවත් ගන්නා ලද එකෙහි නම (function name) සහ පැරාමිටර් ලැයිස්තුව (parameter list) ලෙස හැඳින්වේ. මෙහි විටත් ටයිප් එක යනු ගන්නා ලද එකෙහි අඩුවිටුම් කරනු ලබන දත්තයට අදාළ ඩේටා ටයිප් (data type) එක හෙවත් දත්ත වර්ගයයි.

සෑම වේරියබල් (variable) එකකටම ඩේටා ටයිප් එකක් භාවිතා කරන ආකාරයෙන්ම ගන්නා ලද එක සඳහාද ඩේටා ටයිප් එකක් භාවිතා කරයි. මෙම විටත් ටයිප් එක ගන්නා ලද එකෙහි ක්‍රියාකාරීත්වය අනුව වෙනස් කළ යුතුය. ගන්නා ලද එකක ඇති පැරාමිටර් ලැයිස්තුව යනු ගන්නා ලද එක වෙත ඉන්පුට් ලබා ගන්නා වේරියබල් ලැයිස්තුවක් වැනිය. එසේ පැරාමිටර් ලැයිස්තුව හරහා ලබා ගන්නා ඉන්පුට් ගන්නා ලද එක තුළදී සකස් කොට විටත් ටයිප් එක හරහා ගන්නා ලද එකෙහි අඩුවිටුම් දෙන බව පැහැදිලි වෙනු ඇත.

ඉහත විස්තර කළ මූලික ගන්නා ලද එක තුළදී මෙම අන්තර්ගත කොටස් තුනෙන් භාවිතා කරන ලද්දේ ගන්නා ලද එකෙහි නම පමණි. මෙසේ විටත් ටයිප් එකක් සහ පැරාමිටර් ලැයිස්තුවක් නොමැතිව සකසන ගන්නා ලද එක සඳහා පිටතින් ඉන්පුට් ලබා ගන්නට බැරිය. එබැවින් ඉහත සකස් කළ ගන්නා ලද එක ක්‍රියාත්මක වූයේ එය ඩිෆයින් කරන අවස්ථාවේ ඒ තුළට ඇතුළු කර ඇති දත්තවලින් පමණි. එමෙන්ම ගන්නා ලද එක තුළ අඩුවිටුම් කිරීමට අදාළව යමක් සැකසුණේද නැත.

#### විටත් ටයිප් සහ පැරාමිටර් ලැයිස්තුව සහිතව

##### ගන්නා ලද එකක නිර්මාණය කිරීම

ඉන්පුට් කරනු ලබන සංඛ්‍යා දෙකක් එකතු කර පිළිතුර අඩුවිටුම්



```

1 #include <iostream>
2 using namespace std;
3 int mysum(int num1, int num2);
4 int main () {
5     int a = 100;
6     int b = 200;
7     int ret;
8
9     ret = mysum(a, b);
10    cout << ret << endl;
11    return 0;
12 }
13 int mysum(int num1, int num2)
14 {
15     int result = num1+num2;
16     return result;
17 }
18

```

03

```

300
Process returned 0 (0x0)   execution time : 0.035 s
Press any key to continue.

```

04

කරන ෆන්ක්ෂන් එකක් සකස් කළ හැක්කේ පහත දක්වා ඇති ආකාරයට කෝඩ් ලියමිනි. (03 රූපය)

```

#include <iostream>
using namespace std;
int mysum(int num1, int num2);
int main () {
    int a = 100;
    int b = 200;
    int ret;
    ret = mysum(a, b);
    cout << ret << endl;
    return 0;
}
int mysum(int num1, int num2)
{
    int result = num1+num2;
    return result;
}

```

● මෙහි 3 වැනි පේළියේදී mysum යන නමින් ෆන්ක්ෂන් එකක් සකස් කොට තිබේ. එහිදී වරහන් තුළ num1 සහ num2 යන නමින් පැරාමීටර් දෙකක් සකස් කොට තිබේ. ඒවා සඳහා int වේරා ටයිප් එක යොදා ඇති බැවින් ෆන්ක්ෂන් එක සඳහා ඉන්පුට් කළ හැක්කේ දශමස්ථාන නැති පූර්ණ සංඛ්‍යාය.

● මෙම කෝඩ් තුළ 12 වැනි පේළියේ ෆන්ක්ෂන් එක ඩිෆයින් කර තිබේ. ඒ අනුව 14 වැනි පේළියෙන් num1 සහ num2 පැරාමීටර්ස් දෙක එකතු කර අලුතින් සකස් කරන result වේරියබල් එක වෙත අගය ලබා ගෙන ඇත. මෙහි 15 වැනි පේළියේ ලියා ඇති return කෝඩ් එකෙහිදී අවුට්පුට් කිරීමට ලබාදී ඇත්තේ result වේරියබල් එකය. මේ අනුව ෆන්ක්ෂන් එකෙහි අවුට්පුට් වෙන්නේ මෙම result වේරියබල් එක තුළ ඇති එකතු කළ අගයයි.

● මෙම ෆන්ක්ෂන් එක කෝල් කිරීම සඳහා කෝඩ් ලියා ඇත්තේ 8 වැනි පේළියේදීය. එහිදී mysum ෆන්ක්ෂන් එකෙහි පැරාමීටර්ස් දෙක වෙත පිළිවෙළින් a සහ b වේරියබල්වල ඇති අගයන් ආදේශ කර තිබේ. මෙහිදී එම ෆන්ක්ෂන් එකෙහි පිළිතුර ලබා ගෙන ඇත්තේ ret වේරියබල් එක වෙතටය. පසුව 9 වැනි පේළියෙන් එම ret වේරියබල් එකෙහි අගය අවුට්පුට් කර තිබේ. (04 රූපය) ■

## නව වැනි පරිච්ඡේදය

### ටෙක්ස්ට් ඩොකියුමන්ට් එකක් දත්ත ගබඩාවක් සේ යොදා ගන්නා ආකාරය

පිගණකයේ යම් තැනක ඇති ටෙක්ස්ට් ඩොකියුමන්ට් එකක තොරතුරු සටහන් කිරීම, එම තොරතුරු නැවත ලබා ගැනීම වැනි දේ කළ හැකි වැඩසටහන් C++ ක්‍රමලේඛ භාෂාව භාවිතා කොට සකස් කළ හැක. එවිට මෙම වැඩසටහනේ දත්ත ගබඩාව සේ යොදවන්නේ ටෙක්ස්ට් ඩොකියුමන්ට් එකකි. පොදුවේ මේවා බාහිර ටෙක්ස්ට් ෆයිල් ලෙසත් හඳුන්වයි.

මෙසේ ටෙක්ස්ට් ෆයිල් එකක් සකස් කර ඒ තුළ යම් විස්තරයක් ඇතුළු කිරීම සඳහා කෝඩ් ලිවිය යුත්තේ පහත දක්වා ඇති සේය. (01 රූපය)

```

#include <fstream>
using namespace std;
int main () {

```

```

main.cpp
1  #include <fstream>
2  using namespace std;
3  int main () {
4      ofstream myfile;
5      myfile.open ("example.txt");
6      myfile << "Wijeya Pariganaka.\n";
7      myfile.close();
8      return 0;
9  }
10

```

01

```

ofstream myfile;
myfile.open ("example.txt");
myfile << "Wijeya Pariganaka.\n";
myfile.close();
return 0;
}

```

● මෙහි පළමු පේළිය ලියා තිබෙන්නේ fstream නම් ලයිබ්‍රරි එක ඉම්පෝට් කිරීමටය. fstream යනු ෆයිල් ස්ට්‍රීම් ලයිබ්‍රරි (file stream library) එකයි. අප විසින් ලියනු ලබන C++ වැඩසටහනක සිට හාඩ් ඩිස්ක් එකෙහි ඇති ෆයිල් හැසිරවීම පිණිස මෙම ලයිබ්‍රරි එක භාවිතා කළ යුතුය. මෙම ලයිබ්‍රරි එක තුළ ෆයිල් ස්ට්‍රීම් (file stream) නම් ක්ලාස් එක ofstream ලෙස හැඳින්වෙන අවුට්පුට් ෆයිල් ස්ට්‍රීම් (output file stream) ක්ලාස් එක සහ ifstream ලෙස හැඳින්වෙන ඉන්පුට් ෆයිල් ස්ට්‍රීම් ක්ලාස් එක අඩංගුව තිබේ.

● මෙහි ඇති 2 සහ 3 යන පේළි කෝඩ් බලාත් වැඩසටහන මගින් ලබා දෙන නිසා ටයිප් කළ යුතු නැත. මෙම 3 වැනි පේළියෙන් C++ තුළ ක්‍රියාත්මක වෙන ප්‍රධාන ක්ලාස් එක ආරම්භ වෙයි. වැඩසටහන තුළ ක්‍රියාත්මක වෙන දෑ ලියා තිබෙන්නේ මේ තුළය.

● මෙහි 4 වැනි පේළියේදී fstream තුළ ඇති ofstream ක්ලාස් එක උපයෝගී කර ගෙන myfile යන නමින් ඔබ්ජෙක්ට් එකක් සකස් කර තිබෙයි. මෙය, ෆයිල් එකට අදාළව යම් අවුට්පුට් එකක් ලබා දීමට භාවිතා කරයි. මේ අනුව 5 වැනි පේළිය මගින් example.txt යන නමින් ෆයිල් එකක් සකස් කර එය විවෘත කරනු ලැබේ. එය ක්‍රියාත්මක වුවත් මෙහිදී තිරයේ නොපෙන්වයි. ඒ වෙනුවට ෆයිල් එක සකස්වී එය රැම් මතකය තුළ විවෘත කරනු ලැබේ. එසේ විවෘත කරනු ලැබූ ෆයිල් එක තුළ යමක් ලියන්නේ මෙහි ඇති 6 වැනි පේළියේ ඇති කෝඩ් එකෙනි. මෙහිදී Wijeya Pariganaka යන්න ෆයිල් එක තුළ ලියා ඇත. එය \n මගින් පහළ පේළිය වෙත යොමු වෙයි.

● විවෘත කර ගත් ෆයිල් එක වසා දමන්නේ මෙහි 7 වැනි පේළියේ ඇති කෝඩ් මගිනි. මේ ආකාරයෙන් සකස් කළ කෝඩ් සටහන ක්‍රියාත්මක කළ විට තිරයේ කිසිවක් පෙන්වන්නේ නැත. මෙහිදී ෆයිල් එක තුළ ලිවීම පමණක් කරන බැවිනි. එහෙත් මෙය ක්‍රියාත්මක කළ විට C++ කෝඩ් සහිත ෆයිල් සේව් කර ඇති ෆෝල්ඩර් එක තුළ example නමින් ටෙක්ස්ට් ෆයිල් එකක් සකස්වී එතුළ Wijeya Pariganaka යන්න සටහන්ව තිබෙනු පෙනේවි. (02 රූපය)

#### නව පෙළට හෝ පේළි නිවසකට හෝ දත්ත ඇතුළු කිරීම

ඉහත විස්තර කළ කෝඩ්වලදී Wijeya Pariganaka යන්න ටෙක්ස්ට් ෆයිල් එක මත සටහන් කිරීමෙන් අනතුරුව \n යන්න ලියා ඇති නිසා එය පහළ පේළියට යොමු වෙන බව දැනිමු. එම නිසා ඉහත වැඩසටහනේ 6 වැනි පේළියෙන් පසුව තවත් එවැනිම පේළියක්

```

main.cpp
C:\Users\Pariganaka\Documents\Code\myCode\bin\Debug\myCode.exe
Process returned 0 (0x0)   execution time : 0.083 s
Press any key to continue.

example.txt - Notepad
File Edit Format View Help
Wijeya Pariganaka.

```

02

යෙදවහොත් එය ටෙක්ස්ට් ෆයිල් එක තුළ පහළ පේළියෙන් සඳහන් වෙයි. මෙය තේරුම් ගැනීමට පහත දැක්වෙන කෝඩ් අධ්‍යයනය කරන්න. (03 රූපය)

```

#include <fstream>
using namespace std;
int main () {
    ofstream myfile;
    myfile.open ("example.txt");
    myfile << "Wijeya Pariganaka \n";
    myfile << "Magazine.\n";
    myfile.close();
    return 0;
}

```

මෙහි Magazine යන්න ටෙක්ස්ට් ෆයිල් එක තුළ ලියාවන්නේ පහළ පේළියෙනි. (04 රූපය)

මෙසේ කොටස් දෙකක් සේ ලියා තිබෙන කෝඩ්වලට අදාළ කොටස් ටෙක්ස්ට් ෆයිල් එක තුළ තනි පෙළට සඳහන් කිරීමට අවශ්‍ය නම් \n යන කොටස නොමැතිව ඊට අදාළ පේළි දෙක පහත දක්වා ඇති සේ ලිවිය යුතුය. මෙහි Wijeya Pariganaka යන්න සහ Magazine යන්න ටෙක්ස්ට් ෆයිල් එක තුළ තනි පෙළට ලියවෙනු පෙනේවි.

```

myfile << "Wijeya Pariganaka ";
myfile << "Magazine.";

```

#### ටෙක්ස්ට් ඩොකියුමන්ට් එකටම නවත් දත්ත ඇතුළු කිරීම

ඉහත දැක්වූ ටෙක්ස්ට් ඩොකියුමන්ට් එකෙහි දැනට ඇතුළු කර ඇති පේළි දෙකට පසුව අලුතින් දත්ත ඇතුළු කිරීමට අවශ්‍ය නම් ඊට කෝඩ් ලියන ආකාරය තරමක් වෙනස්ය. මෙය තේරුම් ගැනීමට පහත දැක්වෙන කෝඩ් අධ්‍යයනය කරන්න.

```

#include <fstream>
using namespace std;
int main () {
    ofstream myfile;
    myfile.open ("example.txt", ios::app);
    myfile << "Sri Lanka \n";
    myfile.close();
    return 0;
}

```

මෙහි myfile.open ලෙස ආරම්භ වෙන කෝඩ් පේළියේ ඉදිරියෙන් ෆයිල් එකෙහි නම පසුව ios::app යන්න යොදා තිබේ. එසේ දක්වා



```

1 #include <fstream>
2 using namespace std;
3 int main () {
4     ofstream myfile;
5     myfile.open ("example.txt");
6     myfile << "Wijeya Pariganaka \n";
7     myfile << "Magazine.\n";
8     myfile.close();
9     return 0;
10 }
11

```

03

```

1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 using namespace std;
5 int main () {
6     string line;
7     ifstream myfile ("example.txt");
8     if (myfile.is_open())
9     {
10         while ( getline (myfile,line) )
11         {
12             cout << line << '\n';
13         }
14         myfile.close();
15     }
16 }

```

05

```

1 #include <fstream>
2 using namespace std;
3 int main () {
4     ofstream myfile;
5     myfile.open ("example.txt");
6     myfile << "Wijeya Pariganaka \n";
7     myfile << "Magazine.\n";
8     myfile.close();
9     return 0;
10 }
11

```

04

```

C:\Users\Pariganaka\Documents\EasyGuide\bin\Debug\EasyGuide.exe
Wijeya Pariganaka
Magazine.

Process returned 0 (0x0)   execution time : 0.015 s
Press any key to continue.

```

06

තිබෙන්නේ ගිවිල් එක විවෘත විය යුතු ආකාරයයි. එහි ඇති app යන්න මගින් කියැවෙන්නේ append යන්නයි. එනම් ටෙක්ස්ට් ගිවිල් එක තුළ දැනට ඇති අකුරුවල අගට අලුත් අකුරු ඇමිණිය හැකි ආකාරයට විවෘත කරන ලෙසයි. ඉන් පසුව 7 වැනි පේළියේදී සාමාන්‍ය ආකාරයට ටෙක්ස්ට් ඩොකියුමන්ට් එක තුළ යමක් ලියනු ලැබේ. දැන් එය සටහන් වෙන්නේ කලින් ඇතුළු කරනු ලැබූ විස්තරයට පසුවයි.

#### ගිවිල් එක විවෘත කරන ක්‍රමය සැකසීම

ඉහත දැක්වූ කෝඩ් සටහනේදී ios::app මගින් ගිවිල් එක විවෘත කර ගන්නා ආකාරය තෝරාදී තිබුණි. එය අලුත් දත්ත ගිවිල් එකෙහි අගට අමුණා තැබිය හැකි අයුරින් ගිවිල් එක විවෘත කරන ක්‍රමයයි. ඊට අමතරව ගිවිල් විවෘත කළ හැකි තවත් විවිධ ක්‍රම වෙයි. ඒවා පිළිබඳ විස්තර මෙසේය.

- **ios::app** - මෙය ඉහත විස්තර කළ ආකාරයයි. අලුත් දත්ත ගිවිල් එකෙහි අගට අමුණා තැබිය හැකි අයුරින් ගිවිල් එක විවෘත කරයි.

- **ios::binary** - මෙහිදී ද්විමය හෙවත් බයිනරි (binary) ආකාරයෙන් ගිවිල් එක විවෘත කරයි.

- **ios::nocreate** - මෙහිදී යම් ගිවිල් එකක් විවෘත කරන්නේ එය දැනට හාඩ් ඩිස්ක් එකෙහි තැන්පත්ව ඇති එකක් නම් පමණි. එනම් කලින් මෙන් ගිවිල් එකක් අලුතින් සකස් කර විවෘත වෙන්නේ නැත.

- **ios::noreplace** - මෙහිදී යම් ගිවිල් එකක් විවෘත කරන්නේ එය දැනට හාඩ් ඩිස්ක් එකෙහි තැන්පත්ව නැත්නම් පමණි.

- **ios::out** - මෙහිදී දත්ත ලිවීම සඳහා පමණක් ගිවිල් එක විවෘත කරයි. එම ගිවිල් එකෙහි කලින් ලියූ දත්ත තිබේ නම් ඒවා මකා දමයි. ගිවිල් එක විවෘත කරන විට ගිවිල් එකෙහි නම අගට මෙම ios::out යන්න නොයෙදුවද ගිවිල් එකක් විවෘත වෙන්නේ මේ ආකාරයෙනි.

#### ගිවිල් එකක් විවෘත කර ඒ තුළ ඇති විස්තර නිරය මත පෙන්වීම

ඉහත ආකාරයට කෝඩ් ලිවීමෙන් පසුව ගිවිල් එක සකස්වී එය සේවිම පමණක් වෙයි. එහිදී ඒ තුළ ඇති අන්තර්ගතය නිරයේ පෙන්වන්නේ නැත. එම නිසා ගිවිල් එක බලා ගන්නට අවශ්‍යනම් එය

සාමාන්‍ය ආකාරයට විවෘත කර බලන්නට වෙයි. මේ වෙනුවට වැඩ සටහන තුළම ගිවිල් එකෙහි අන්තර්ගතය පෙන්වීම සඳහා පහත දක්වා ඇති සේ කෝඩ් ලිවිය යුතුය. (05 රූපය)

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main () {
    string line;
    ifstream myfile ("example.txt");
    if (myfile.is_open())
    {
        while ( getline (myfile,line) )
        {
            cout << line << '\n';
        }
        myfile.close();
    }
    else cout << "Unable to open file";
    return 0;
}

```

මෙහිදී කලින් ඉම්පෝට් කළ ලයිබ්‍රේරිවලට අමතරව string ලයිබ්‍රේරියද ඉම්පෝට් කර තිබේ. එය භාවිතා කරන්නේ මෙහි 7 වැනි පේළියේදීය. එහිදී ගිවිල් එක විවෘත කරනු ලැබේ. 8 වැනි පේළියේ ඇති if කන්ඩිෂනල් ස්ට්‍රක්චර් එක ක්‍රියාත්මක වෙන්නේ අදාළ ගිවිල් එක දෝෂ නොපෙන්වා විවෘතවී තිබෙනමය. එහිදී 10 වැනි පේළියෙන් ආරම්භ වෙන While ලූප් එක මගින් ටෙක්ස්ට් ගිවිල් එක තුළ ඇති පේළියෙන් පේළිය නිරය මත අඩුවිටුවී කර පෙන්වයි. (06 රූපය)



# programming



## ත්‍රෝග්‍රහණයේ පොත්

පැස්කල් | ජාවා | ජාවා ස්ක්‍රිප්ට් | ජිඑච්සී | පයිතන්  
වීබී ඩොට් හෙට් | සී ෂාප් ඩොට් හෙට්

කිට්ටුම පුවත්පත් නියෝජිත  
වෙත ඇනවුම් කර මෙම  
පොත් ගන්නවා ගත හැක.

පොතක මිල රුපියල් 200යි.  
පොත් 07ක කට්ටලයේ මිල  
රුපියල් 900 යි.

**EasyGuide®** සරල සිංහලෙන් පරිගණක  
දැනුම ගෙනෙන පොත් පෙළ

දුරකතන අංක 0112 479 629 හෝ 0112 479 657 හෝ ඔස්සේ අමතා වැඩි විස්තර දැන ගත හැක.

© 2019 Wijaya Newspapers Ltd.





සි ජලස් ජලස් යනු අද ලොව ප්‍රකට පෙළේ පරිගණක ක්‍රමලේඛ භාෂා අතරින් එකකි. එසේම අනාගත යේදී පරිගණක ක්‍රමලේඛ ශිල්පියෙක් වෙන්නට සිතා වැඩ කරන ඕනෑම කෙනෙක් විසින් උගත යුතු මූලික ක්‍රම ලේඛ භාෂා අතරින් මුල් තැනේ තිබෙන ක්‍රමලේඛ භාෂාව සි ජලස් ජලස්ය. මෙම භාෂාව හැසිරවීම ගැන මූලික දැනුමක් නැති අයට වුවත් මෙම පොත පරිහරණය කොට ඒ ගැන ඉගෙන ගන්නට පුළුවන. එසේම දැනට සි ජලස් ජලස් ගැන මූලික දේ දන්නා අයටත් දැනුම වැඩි කර ගන්නට ඉවහල් වෙන කරුණුද මෙම පොතෙහි අඩංගුය. ■

EasyGuide®

ISBN 978-955-1008-60-4



9 789551 008604